CMDBUF RD 场景 波形与 SVA 对照手册

基于 2026 年 10 月 21 日对齐规格与 VM 验证证据

这份手册把当前 156 个场景放到一起：先说明怎样触发、应该看到什么，再给出波形和对应 SVA。读者可以按场景编号检查设计规则，也可以从代码标签追到验证日志。代码保持可编辑，图中的采样值来自保存的实际轨迹。

本模块为三个 XAU 共用九项 CMDBUF 和 OTB。读命令在最老五条待发命令中选择空闲 Bank 的最老项，允许乱序发射；MEM 数据在成功发射后四拍捕获，之后按每个 XAU 的接收顺序返回，三个 XAU 之间用 RR 轮询。Entry 只在 R 握手时回收，AR 可以同拍使用回收的容量。

本文内容	数量或范围
逐项场景	156 项，沿用 AR、CAP、ENT、WIN 等原编号
实际波形	143 项正向见证，4 项环境违规见证；共 147 个场景
边界示意	9 项，涉及物理注错、端口不可观察或配置审计
原生 cover 代码	全部 547 个当前 Corner 分支；正文给代表分支，附录补齐其余分支
断言和约束	正文给主要检查；附录收录当前 assume、assert 与模型自检
不纳入本册	复位场景和 X/Z 场景

应先读第 1 章的记号和采样约定，再看所需场景。正向 cover 命中表示存在对应轨迹，不能替代断言证明。此次 156 个 assert 实例通过 10 步 BMC；只有 req 独热或零、rvalid 独热或零、rvalid 指向 ready 三条属性完成无界证明。

139 项合法业务都有相应 cover 命中；ENT-12 使用单独的合法输入引导作业。576 条正向轨迹经原生 SVA 回放，共 4740 个采样拍，全部新增 547 个 cover 分支命中。这不代表所有文字条件均已闭合：ARB-13、MEM-03、MEM-06 的条件差异见相应场景页和附录 E。原失败、超时与注错边界保留。

版本依据：docs/timing/protocol_corrections_20261021.json；证据依据：verification/corner_20261021。RTL 和验证代码本次没有修改。

阅读导航

下列页码为 Word 域，可点击跳转；在 Word 中更新域后，目录会随排版同步。每个场景标题也出现在 Word 导航窗格中。

类别	场景编号	项数	起始页
AR 接收与 src	AR-01 至 AR-20	20	6
容量与来源保护	CAP-01 至 CAP-24	24	26
Entry 生命周期	ENT-01 至 ENT-12	12	50
流水与乱序窗口	WIN-01 至 WIN-19	19	62
Bank 与仲裁判断	ARB-01 至 ARB-14	14	81
pending 与占用计数	CNT-01 至 CNT-08	8	95
MEM 返回与 OTB	MEM-01 至 MEM-14	14	103
R 保序与 RR	RSP-01 至 RSP-18	18	117
字段与错误组合	ERR-01 至 ERR-08	8	136
同拍事件与冲突	CON-01 至 CON-06	6	144
持续业务与边界	BIZ-01 至 BIZ-10	10	150
接入与验证边界	ENV-06 至 ENV-08	3	160

第 1 章 记号与检查方法 第 3 页

附录 A 公共 assume 与 assert 第 163 页

附录 B 其余 cover 分支 第 170 页

附录 C 源码接入与变量说明 第 203 页

附录 D 关联的基础 cover 第 205 页

附录 E 已识别的覆盖条件差异 第 208 页

场景页如何读

“前提与动作”给出输入和已有状态；“预期”描述设计必须做到的事情；黄色波形列标出该分支的关键采样点。正文 SVA 为本项代表检查和实际命中的 cover。其余属性标签保留在页尾，assert 与 assume 查附录 A，基础 cover 查附录 D。

同一个场景可能有多个来源、Bank、half 或数据位分支。正文只绘一条代表轨迹，来源编号或容量分布可能与说明中的例子不同；实际取值以图和页内关键采样为准。全部分支代码列在正文或附录 B。MEM-12 的 260 个分支包含 128 位 walking-one、walking-zero 及全零、全一等特征。

1 采样记号和事件定义

图中 S0、S1 等是 VCD 中连续上升沿的采样编号，不代表纳秒。每个格子画的是该拍采样值；数字波形的保持画法用于阅读，不承诺两个边沿之间没有组合变化。黄色列是代表 cover 的触发采样点。

记号	含义
A	AR 实际握手：arvalid=1、src<3，且 arready[src]=1。
I	成功发射：req、ar_rd_gnt、rw_rd_gnt 在同一个 Bank 位同时为 1。
M	某条 I 的第 4 拍到期。本接口没有 MEM valid，M 是验证侧日历事件。
R	R 实际握手：rvalid & rready 非零。
N0/N1/N2	拍初各源已接受 AR 数减去已完成 R 数；不是待发数。
Nnext	计入当前 A、R 后的下一状态；I、M 不释放占用。
P 和 pending_cnt	P 为拍初待发数；pending_cnt=P+A−I，本拍组合输出。
T0 T1 等	按 AR 接收先后分配的图示事务编号，不是 arid，也不是 DUT 物理 entry。
带星号的信号	由顶层端口历史推导，未读取 DUT 内部。C2 window 按年龄列出最多五个编号。
总线与状态	ready/valid/grant/req 用二进制；地址、ID 用十六进制；其他计数用十进制。
status Bn[1:0]	固定 Bank n 的奇偶两位，左为 half1，右为 half0；0 空闲、1 占用。
横杠	该图不使用此字段，或该负载无效；不表示 X/Z 测试。

SBY 的 chformal 检查节点比触发它的端口采样晚一步。证据日志的 step 与本文黄色列都予以保留：例如日志 step=13 对应 VCD 采样 S12。这个编号差不改变 I→M 的四拍协议，不能把日志步号当作数据返回延迟。

128 位数据中的 ALL1 表示全一，1<<b 表示只置第 b 位。D0、D1 等仅为本图别名，其完整十六进制值写在图后；数据断言始终比较全部 128 位。

2 容量 窗口和返回时序

容量先扣除本拍已经发生的 R，再判断是否可以接收。每源最多七项；空来源至少保留一项，因此仅检查“总数≤9、每源≤7”还不够，5/4/0 也必须禁止。

```
A = arvalid && (src < 3) && arready[src];
R_s = rvalid[s] && rready[s];
N_s_next = N_s + (A && src == s) - R_s;
M_s = N_s - R_s;
U = M_0 + M_1 + M_2;
S = max(M_0-1,0) + max(M_1-1,0) + max(M_2-1,0);
ready_expected[s] = (U < 9) && ((M_s == 0) || (S < 6));
```

上面的 M_s 是扣除回收后的容量中间量，不是 MEM 到期事件 M。工程代码使用 h_after[s]，避免同名。比如拍初 4/4/1，回收 XAU2 后变为 4/4/0，此拍 ready=100；回收 XAU0 的一项则三源均可接收。

命令最短经过 C1 接收、C2 判断、C3 请求。C2 先从最老五条未成功发射命令中过滤空闲 half，再取最老候选。本拍 I 成功后，下一条预选可以立即使用补入窗口的原第六条；失败项保持年龄，若仍被选中，下一拍可以继续请求。新 AR 不参加本拍 C2。

五项全忙时，即使第六项空闲也不发射。成功才从待发集合移除该项，普通 R 回收不会推动待发窗口。当前 req 来自上一拍预选，因此 C3 的目标状态变忙不能撤销 req；双 grant 成功反而要求当拍目标 half 的状态为忙。

I 出现在采样 N，MEM 数据和 pc_err 在 N+4 捕获，最早 N+5 才能对该命令产生 R。年轻命令即使先完成，也不能越过同源队头。跨源只在“队头已完成且 rready=1”的集合内按 0→1→2 轮询，实际 R 后指向胜者下一源。

rresp 的规则为：该事务 arresp=11 时返回 11；否则保存的 pc_err=1 时返回 01；否则返回 00。越界命令仍发给 MEM。无 req 或无 rvalid 的负载不检查。

3 SVA 的责任和代码依赖

assume 规定环境允许出现的输入关系，assert 检查设计输出，cover 寻找具体场景。不能把正确的 ready 或 R 顺序写进 assume，再宣布该行为被证明。

标记	用途和限制
NO_REF_MODEL_PORTS	仅使用当前顶层端口，例如 req 独热与 rvalid⊆rready。
NO_REF_MODEL_HISTORY	使用端口的历史计数或快照，例如 h_occ、last_pending。没有事务存储模型。
NO_REF_MODEL_ENV	环境义务；本册正常作业只有 E03、E04、E05、E06，展开共 11 个实例。
REF_MODEL	按端口事件保存事务、窗口、MEM 日历和各源 FIFO，用于比较预期输出。
MODEL_SELF_CHECK	参考记录自身的守恒和一致性检查，不证明 DUT 的物理槽位或原始位清零。

所有原生属性使用 @(posedge clk) disable iff (!rst_n)。正文是从工程提取的属性片段，依赖同一检查器中的辅助变量及 generate 作用域，不能把单个片段直接当作完整模块编译。附录 C 给出接入文件和变量入口。

对于 AR-09、AR-10、ARB-12、ERR-05，单独移除对应环境 assume，保留其余契约，用 V_* 断言监测违规、N_* cover 获取违规见证。V_* 失败是预期检出，不是正常 DUT 验证失败。

CAP-20、CAP-21、ENT-10、ENT-11、CON-05 的物理注错尚未执行；MEM-14 缺少独立数据真值；AR-15 涉及边沿之间变化；ENV-07 是结构可观察性边界；ENV-08 是约束集合审计。这九项配示意图，不补造 cover 命中。

证明范围：10 步 BMC 共 156 个 assert 实例，其中 51 个检查 DUT 行为、105 个检查模型自洽；三条端口协议性质有无界结果。12 步全量 BMC、容量无界 PDR 和 ENT-12 自由搜索的超时记录仍保留。有限轨迹不用于声称永久等待或无条件排空。

实际工具为 VM 上的 SBY；本手册没有新增 FormalMC 2024 执行、加密 RTL 等价检查或 UCAgent 全流程验收结论。

AR 01 空表首次接收

AR 接收与 src | 合法业务

前提与动作 N=(0,0,0)，无 R；arvalid=1，依次覆盖 src=0/1/2。

预期 aready=111；仅 src 对应计数加 1；一次只分配一项。

要拦住的错误 把 ready 三位当成三个并行 AR；一次分配多项。

实际 Formal 见证 AR-01 上升沿采样值

	S0
arvalid	1
src	0
aready	111
rvalid	000
rready	000
A	1
R	0
N0/N1/N2*	0/0/0
Nnext*	1/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-01 实际正向见证；黄色为关键采样点。

关键采样 S0 A/I/M/R=1000，N=0/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    aready==h_ready
);
```

```
C_CC_AR_01_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==0 && !r_fire && ar_fire && src==0 && aready==7
);
```

证据 build/formal/ar/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, M_ALLOC, M_RANGE。

分支 3 个；其余分支完整代码见附录 B。

AR 02 无请求时切换 src

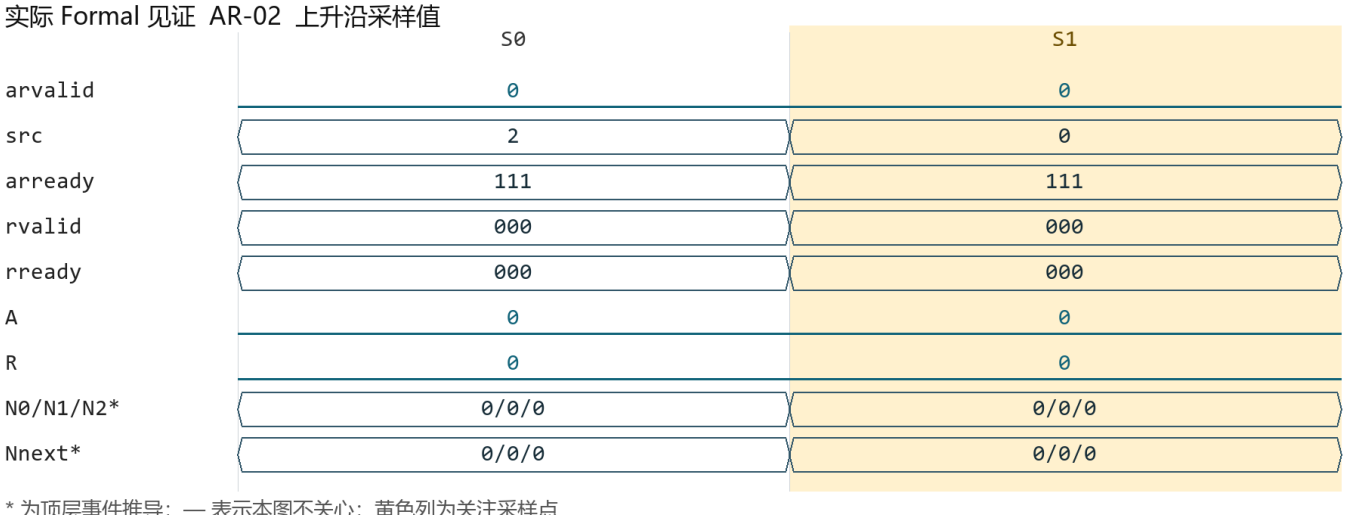
AR 接收与 src | 合法业务

前提与动作 任一合法占用，无 R；arvalid=0；src 在 0/1/2/3 之间切换。

预期 不接收命令；占用不变；ready 不应由 src 变化而变化。

要拦住的错误 未以 arvalid 门控分配；src=3 越界索引。

实际 Formal 见证 AR-02 上升沿采样值



* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-02 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=0/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_02_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    !h_prev_valid && !arvalid && !h_prev_r && !r_fire && h_prev_n==used && src==0 &&
        src!=h_prev_src && arready==h_prev_ready
);
```

证据 build/formal/ar/run/engine_0/trace3.vcd；日志 step=2，图示采样 S1。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_PENDING, A_READY, M_RANGE。

分支 4 个；其余分支完整代码见附录 B。

AR 03 选中可接收来源

AR 接收与 src | 合法业务

前提与动作 aready 非零，所选 src 的位为 1；arvalid=1，src=0/1/2。

预期 A=1；只采样该拍共享负载并归属 src。

要拦住的错误 只检查 ready 非零，未检查对应 bit。

实际 Formal 见证 AR-03 上升沿采样值

	S0
arvalid	1
src	0
aready	111
rvalid	000
rready	000
A	1
R	0
N0/N1/N2*	0/0/0
Nnext*	1/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-03 实际正向见证；黄色为关键采样点。

关键采样 S0 A/I/M/R=1000，N=0/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    aready==h_ready
);
```

```
C_CC_AR_03_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && src==0
);
```

证据 build/formal/ar/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_ID_ORDER, M_ALLOC。

分支 3 个；其余分支完整代码见附录 B。

AR 04 选中被阻塞来源

AR 接收与 src | 合法业务

前提与动作 例如 N=(7,0,0)，无 R，ready=110；arvalid=1，src=0。

预期 A=0；不分配、不增加占用、不把请求加入年龄序列。

要拦住的错误 以任意 ready=1 错收请求。

实际 Formal 见证 AR-04 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	110
rvalid	000	000	000	000	000	000	000	000
rready	000	000	111	111	111	111	101	101
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
Nnext*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-04 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0000，N=7/0/0，ready=110，pending_cnt=7。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==7 && occ[1]==0 && occ[2]==0 && !r_fire && arvalid && src==0 && !ar_fire
        && arready==6
);
```

证据 build/formal/ar/run/engine_0/trace28.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_PENDING, A_READY, M_RANGE。

分支 1 个；本页已列出该场景 cover。

AR 05 被阻塞后切换来源

AR 接收与 src | 合法业务

前提与动作 N=(7,0,0)，src0 未被接收；下一拍换 src1；可同时更换地址、ID 和 port_id。

预期 新拍若 ready[1]=1 则只接收新请求；前一拍不形成事务。

要拦住的错误 把未握手请求锁住；错误加入 AXI hold 约束。

实际 Formal 见证 AR-05 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	1
arready	111	111	111	111	111	111	110	110
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	111	111	011	111	100
A	1	1	1	1	1	1	0	1
R	0	0	0	0	0	0	0	0
N0/N1/N2*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0
Nnext*	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0	7/1/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-05 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=1000，N=7/0/0，ready=110，pending_cnt=7。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_05_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_n==7 && h_prev_valid && !h_prev_a && h_prev_src==0 && h_prev_ready==6 &&
        ar_fire && src==1 && araddr!=h_prev_addr && arid!=h_prev_id &&
        ar_port_id!=h_prev_port
);
```

证据 build/formal/ar/run/engine_0/trace30.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_R_METADATA。

分支 1 个；本页已列出该场景 cover。

AR 06 同源连续接收

AR 接收与 src | 合法业务

前提与动作 有容量且每拍所选 ready=1；连续数拍 arvalid=1、src 相同，负载每拍变化。

预期 每拍独立接收；年龄按握手拍排序；无 valid 下降沿要求。

要拦住的错误 只识别 valid 上升沿导致漏单。

实际 Formal 见证 AR-06 上升沿采样值

	S0	S1	S2
arvalid	1	1	1
src	2	2	2
arready	111	111	111
rvalid	000	000	000
rready	000	000	000
A	1	1	1
R	0	0	0
N0/N1/N2*	0/0/0	0/0/1	0/0/2
Nnext*	0/0/1	0/0/2	0/0/3

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-06 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=1000，N=0/0/2，ready=111，pending_cnt=3。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_same_ar_run>=2 && ar_fire && src==h_prev_src && arid!=h_prev_id &&
        ar_port_id!=h_prev_port
);
```

证据 build/formal/ar/run/engine_0/trace11.vcd；日志 step=3，图示采样 S2。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_XAU_R_ID_ORDER, A_XAU_R_PORT_ORDER。

分支 1 个；本页已列出该场景 cover。

AR 07 不同源连续切换

AR 接收与 src | 合法业务

前提与动作 有容量；src 按 0→2→1→0 切换，每拍实际握手。

预期 三路分别计数与保序；共享 AR 每拍仍最多一单。

要拦住的错误 以固定轮询顺序代替实际 src 归属。

实际 Formal 见证 AR-07 上升沿采样值

	S0	S1	S2	S3
arvalid	1	1	1	1
src	0	2	1	0
arready	111	111	111	111
rvalid	000	000	000	000
rready	000	000	111	101
A	1	1	1	1
R	0	0	0	0
N0/N1/N2*	0/0/0	1/0/0	1/0/1	1/1/1
Nnext*	1/0/0	1/0/1	1/1/1	2/1/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-07 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=1000，N=1/1/1，ready=111，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_a_history[2:0]==3'b111 && h_src_history==6'b001001 && ar_fire && src==0
);
```

证据 build/formal/ar/run/engine_0/trace12.vcd；日志 step=4，图示采样 S3。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_ID_ORDER。

分支 1 个；本页已列出该场景 cover。

AR 08 全不 ready 时空拍

AR 接收与 src | 合法业务

前提与动作 N=9 且无 R；aready=000，环境令 arvalid=0，src 任意。

预期 不接收；src 不决定何时恢复 ready；恢复取决于实际 R。

要拦住的错误 误要求 src 变化能解除满载。

实际 Formal 见证 AR-08 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	0
src	2	1	0	0	0	0	1	0
aready	111	111	111	111	111	111	111	000
rvalid	000	000	000	000	000	000	000	000
rready	111	111	101	111	011	111	111	000
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/1/1	0/1/2	0/2/2	1/2/2	2/2/2	3/2/2	4/2/2	4/3/2
Nnext*	0/1/2	0/2/2	1/2/2	2/2/2	3/2/2	4/2/2	4/3/2	4/3/2

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-08 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0000，N=4/3/2，ready=000，pending_cnt=9。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    aready==h_ready
);
```

```
C_CC_AR_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && !r_fire && aready==0 && !arvalid
);
```

证据 build/formal/ar/run/engine_0/trace31.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, E05_ZERO_READY。

分支 1 个；本页已列出该场景 cover。

AR 09 全不 ready 仍拉 valid

AR 接收与 src | 环境违约

前提与动作 aready=000；注入 arvalid=1。

预期 违反输入约束 E05；不能作为合法业务反例；单独检查约束监测。

要拦住的错误 在保留 E05 时求该 cover，导致不可达。

环境违规见证 AR-09 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	1
src	1	2	1	2	2	1	0	0
aready	111	111	111	111	111	111	001	000
rvalid	000	000	000	000	000	000	000	000
rready	111	011	111	111	111	101	101	101
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/2/0	0/3/0	0/3/1	0/4/1	0/4/2	0/4/3	0/5/3	1/5/3
Nnext*	0/3/0	0/3/1	0/4/1	0/4/2	0/4/3	0/5/3	1/5/3	1/5/3

* 为顶层事件推导；— 表示本图不关心；蓝色列为关注采样点

图 AR-09 实际负向输入轨迹，监测断言按预期失败。

关键采样 S9 A/I/M/R=0000，N=1/5/3，ready=000，pending_cnt=8。

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 全不 ready 时环境仍发送有效请求；本项改为监测断言，预期检出。

```
V_AR_09: assert property (
    @(posedge clk) disable iff (!rst_n)
    (aready==0) |-> (!arvalid)
);
```

```
N_AR_09: cover property (
    @(posedge clk) disable iff (!rst_n)
    aready==0 && arvalid
);
```

证据 build/negative/ar_09_cover/run/engine_0/trace0.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, E05_ZERO_READY。

边界 关闭 E05 后，用 N_AR_09 cover / V_AR_09 assert 检查环境违规见证。

AR 10 有效请求 src=3

AR 接收与 src | 环境违约

前提与动作 arvalid=1；src=2'b11。

预期 违反 E03；不能分配给不存在的第四路；无合法业务承诺。

要拦住的错误 无效索引导致误分配或污染计数。

环境违规见证 AR-10 上升沿采样值

	S0
arvalid	1
src	3
arready	111
rvalid	000
rready	000
A	0
R	0
N0/N1/N2*	0/0/0
Nnext*	0/0/0

* 为低层事件推导；— 表示本图不关心；蓄色列为关注采样点

图 AR-10 实际负向输入轨迹，监测断言按预期失败。

关键采样 S0 A/I/M/R=0000，N=0/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 有效请求使用不存在的第四个来源；本项改为监测断言，预期检出。

```
V_AR_10: assert property (
    @(posedge clk) disable iff (!rst_n)
    arvalid |-> (src<3)
);
```

```
N_AR_10: cover property (
    @(posedge clk) disable iff (!rst_n)
    arvalid && src==3
);
```

证据 build/negative/ar_10_cover/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, E03_SRC。

边界 关闭 E03 后，用 N_AR_10 cover / V_AR_10 assert 检查环境违规见证。

AR 11 AR 无效负载任意变化

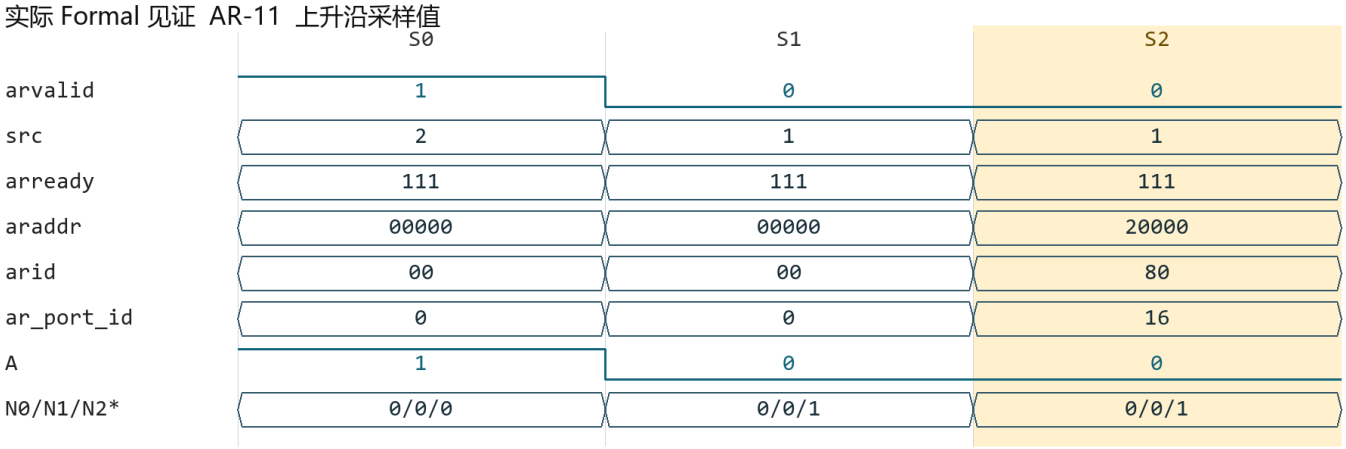
AR 接收与 src | 合法业务

前提与动作 arvalid=0；地址、arid、port_id、arresp 改变。

预期 不采样这些字段；不得改变当前事务内容。

要拦住的错误 无 arvalid 时刷新已有 entry。

实际 Formal 见证 AR-11 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-11 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=0/0/1，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_11_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    !h_prev_valid && !arvalid && used>0 && araddr!=h_prev_addr && arid!=h_prev_id &&
        ar_port_id!=h_prev_port
);
```

证据 build/formal/ar/run/engine_0/trace9.vcd；日志 step=3，图示采样 S2。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_METADATA, A_XAU_R_DATA_ORDER。

分支 1 个；本页已列出该场景 cover。

AR 12 相同负载重复握手

AR 接收与 src | 合法业务

前提与动作 两次 AR 地址、ID、port_id 完全相同；两拍分别 A=1。

预期 是两条不同事务；占两项；每条分别发射和响应。

要拦住的错误 以地址/ID 去重；用 arid 代替年龄。

实际 Formal 见证 AR-12 上升沿采样值

	S0	S1
arvalid	1	1
src	2	2
arready	111	111
araddr	00000	00000
arid	00	00
ar_port_id	0	0
A	1	1
N0/N1/N2*	0/0/0	0/0/1

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-12 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=1000，N=0/0/1，ready=111，pending_cnt=2。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_12_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_a && ar_fire && src==h_prev_src && araddr==h_prev_addr && arid==h_prev_id
        && ar_port_id==h_prev_port
);
```

证据 build/formal/ar/run/engine_0/trace5.vcd；日志 step=2，图示采样 S1。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_XAU_R_HAS_PENDING, A_XAU_R_ID_ORDER。

分支 1 个；本页已列出该场景 cover。

AR 13 本拍 src 不直接决定 ready

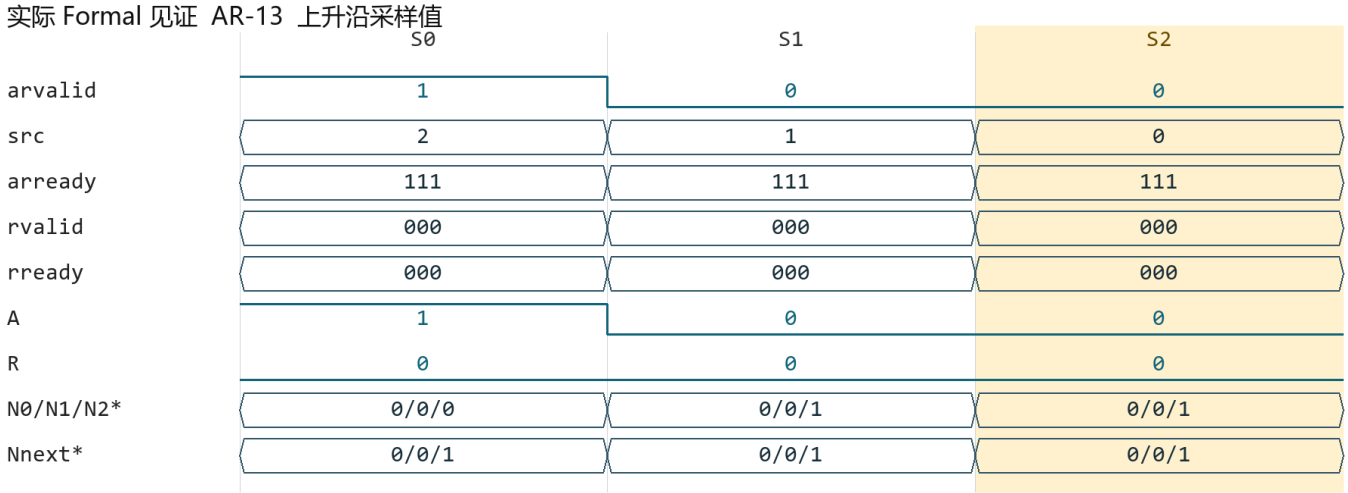
AR 接收与 src | 合法业务

前提与动作 固定 N、R，选择不同合法 src；仅切换 src，保持 rready 与完成状态不变。

预期 ready 三位保持相同；只有 A 的来源和下一拍占用可变。

要拦住的错误 把 src 编入 ready 的组合反馈。

实际 Formal 见证 AR-13 上升沿采样值



* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-13 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=0/0/1，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_13_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    !h_prev_a && !ar_fire && !h_prev_r && !r_fire && h_prev_n==used &&
        src!=h_prev_src && arready==h_prev_ready && used>0
);
```

证据 build/formal/ar/run/engine_0/trace10.vcd；日志 step=3，图示采样 S2。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

AR 14 本拍 valid 不直接决定 ready

AR 接收与 src | 合法业务

前提与动作 固定 N 与 R；arvalid 在 0/1 间改变，遵守全零 ready 约束。

预期 ready 本拍不变；不允许由 A 反推 ready 构成环路。

要拦住的错误 使用加上本拍 A 的占用计算本拍 ready。

实际 Formal 见证 AR-14 上升沿采样值

	S0	S1
arvalid	0	1
src	0	0
arready	111	111
rvalid	000	000
rready	000	000
A	0	1
R	0	0
N0/N1/N2*	0/0/0	0/0/0
Nnext*	0/0/0	1/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-14 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=1000，N=0/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_14_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    !h_prev_a && !h_prev_r && !r_fire && used==h_prev_n && arvalid!=h_prev_valid &&
        arready==h_prev_ready && arready!=0
);
```

证据 build/formal/ar/run/engine_0/trace6.vcd；日志 step=2，图示采样 S1。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

AR 15 非采样瞬间的组合变化

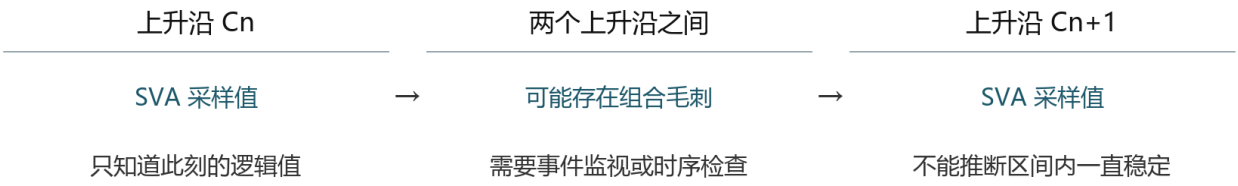
AR 接收与 src | 集成边界

前提与动作 时钟边沿前后分别观察；rready 变化使 R 胜者改变，再观察 ready 与 AR 握手。

预期 用上升沿稳定值定义 A/R；ready 是组合输出，不要求整拍恒定。

要拦住的错误 用波形毛刺当成多次接收；误按注册 ready 采样。

注错或观测边界示意 未作为已命中的 Formal 波形



本图描述检查责任或拟注错路径，不声称该非法内部状态已经被注入。

图 AR-15 未执行注错或不可直接观察的边界示意。

检查边界 posedge SVA 只检查采样值；边沿之间的组合毛刺/稳定时间需要时序或仿真检查。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_R_READY。

AR 16 有效时字段的边界值

AR 接收与 src | 合法业务

前提与动作 每个 src 有容量；araddr=0/18'h3ffff，arid=0/255，port_id=0/31，并遍历 raddr 0..7。

预期 有效负载位宽不截断；返回所属事务字段；地址越界由 arresp 指示。

要拦住的错误 有符号扩展、位切片错误；仅凭地址自行判越界。

实际 Formal 见证 AR-16 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	–	T2	T3	T4	T5	–
araddr	00000	00000	00000	00000	00000	00000	00000	20000
arid	00	00	00	00	00	00	00	80
ar_port_id	0	0	0	0	0	0	0	16
rvalid	000	000	000	000	000	000	000	001
raddr	0	0	0	0	0	0	0	0
rid	00	00	00	00	00	00	00	00
r_port_id	0	0	0	0	0	0	0	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-16 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=2/0/4，ready=111，pending_cnt=5。

[REF_MODEL] R 握手时，ID、port_id、地址低三位必须属于同一预期事务。

```
A_R_METADATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> ((rid == m_id[rsp_e] && r_port_id == m_port[rsp_e] && raddr ==
        m_addr[rsp_e][2:0]))
);
```

```
C_CC_AR_16_S0_EXT0: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_src[rsp_e]==0 && m_addr[rsp_e]==18'h00000 && rid==0 &&
        r_port_id==0
);
```

证据 build/formal/ar/run/engine_0/trace13.vcd；日志 step=8，图示采样 S7。

关联属性 A_ARB_ADDR, A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_XAU_R_ADDR_ORDER。

分支 14 个；其余分支完整代码见附录 B。

AR 17 AR 与 R 来源相同

AR 接收与 src | 合法业务

前提与动作 该源旧队头可响应，当前有容量或可回收；同拍 R_s=1、A_s=1。

预期 N_s 不变；旧头弹出，新命令放到该源末尾。

要拦住的错误 把同拍新 AR 当作刚返回的旧命令。

实际 Formal 见证 AR-17 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	0	1	1	1	1	1
src	0	0	1	2	2	2	0	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	001
rready	000	000	111	111	111	111	111	101
A	1	1	0	1	1	1	1	1
R	0	0	0	0	0	0	0	1
N0/N1/N2*	0/0/0	1/0/0	2/0/0	2/0/0	2/0/1	2/0/2	2/0/3	3/0/3
Nnext*	1/0/0	2/0/0	2/0/0	2/0/1	2/0/2	2/0/3	3/0/3	3/0/3

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-17 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=3/0/3，ready=111，pending_cnt=6。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_17_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && r_fire && src==winner && h_response_matches
);
```

证据 build/formal/ar/run/engine_0/trace23.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_XAU_R_ID_ORDER, C_XAU_SAME_CYCLE_POP_PUSH。

分支 1 个；本页已列出该场景 cover。

AR 18 AR 与 R 来源不同

AR 接收与 src | 合法业务

前提与动作 源 s 响应，源 t 的 ready[t]=1；同拍 R_s=1、A_t=1，s≠t。

预期 总 N 不变；N_s 减 1、N_t 加 1；逐源顺序不串扰。

要拦住的错误 总量正确但分路计数错。

实际 Formal 见证 AR-18 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	0	1	1	1	1	1
src	0	0	1	2	2	2	2	2
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	001
rready	000	000	111	111	111	111	111	101
A	1	1	0	1	1	1	1	1
R	0	0	0	0	0	0	0	1
N0/N1/N2*	0/0/0	1/0/0	2/0/0	2/0/0	2/0/1	2/0/2	2/0/3	2/0/4
Nnext*	1/0/0	2/0/0	2/0/0	2/0/1	2/0/2	2/0/3	2/0/4	1/0/5

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-18 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=2/0/4，ready=111，pending_cnt=6。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_18_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && r_fire && src!=winner && h_response_matches
);
```

证据 build/formal/ar/run/engine_0/trace21.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_ID_ORDER, M_RANGE。

分支 1 个；本页已列出该场景 cover。

AR 19 R 已发生但所选源不能接收

AR 接收与 src | 合法业务

前提与动作 例如 (4,4,1) 本拍 R2，ready=100；arvalid=1、src0 或 src1。

预期 R 仍正常回收，AR 不接收，总占用由 9 降到 8。

要拦住的错误 因 AR 阻塞取消 R；认为有 R 则任何源可接收。

实际 Formal 见证 AR-19 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	1
src	2	0	0	1	1	1	0	0
arready	111	111	111	111	111	111	111	100
rvalid	000	000	000	000	000	000	000	100
rready	111	111	101	111	111	101	110	110
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	1
N0/N1/N2*	1/1/0	1/1/1	2/1/1	3/1/1	3/2/1	3/3/1	3/4/1	4/4/1
Nnext*	1/1/1	2/1/1	3/1/1	3/2/1	3/3/1	3/4/1	4/4/1	4/4/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-19 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0001，N=4/4/1，ready=100，pending_cnt=7。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_19_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==4 && occ[1]==4 && occ[2]==1 && rvalid==4 && arvalid && src<2 && !ar_fire
        && arready==4
);
```

证据 build/formal/ar/run/engine_0/trace32.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_R_READY。

分支 1 个；本页已列出该场景 cover。

AR 20 没有上游公平保证

AR 接收与 src | 集成边界

前提与动作 ready 至少一位有效，某一路持续有需求；上游长期只选另一路或被阻塞的 src。

预期 DUT 保留容量许可，但不能保证未被送到共享 AR 的请求最终接收。

要拦住的错误 把容量保护误写成无条件接收活性。

实际 Formal 见证 AR-20 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
arvalid	1	1	1	1	1	1	1	1
src	2	2	2	2	2	2	2	2
arready	111	111	111	111	111	111	011	011
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	101	111	111	001	010
A	1	1	1	1	1	1	0	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/1	0/0/2	0/0/3	0/0/4	0/0/5	0/0/6	0/0/7	0/0/7
Nnext*	0/0/2	0/0/3	0/0/4	0/0/5	0/0/6	0/0/7	0/0/7	0/0/7

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 AR-20 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0000，N=0/0/7，ready=011，pending_cnt=5。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_AR_20_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_valid && !h_prev_a && arvalid && !ar_fire && arready!=0 &&
        src==h_prev_src
);
```

证据 build/formal/ar/run/engine_0/trace29.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

边界 只见证连续阻塞选择；共享端口无各源 pending 信号，不能证明上游公平性。

CAP 01 总量 0 到 6

容量与来源保护 | 合法业务

前提与动作 任意合法分配，N≤6；无 R，查询三个 ready。

预期 ready=111；所有来源均可再收一项。

要拦住的错误 错设每源固定三项额度。

实际 Formal 见证 CAP-01 上升沿采样值

	S0
arvalid	0
src	0
arready	111
rvalid	000
rready	000
A	0
R	0
N0/N1/N2*	0/0/0
Nnext*	0/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-01 实际正向见证；黄色为关键采样点。

关键采样 S0 A/I/M/R=0000，N=0/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_01_N0: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total==0 && !r_fire && arready==7
);
```

证据 build/formal/cap/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 7 个；其余分支完整代码见附录 B。

CAP 02 单源从 6 到 7

容量与来源保护 | 合法业务

前提与动作 N=(6,0,0)，无 R；src0 握手一次。

预期 本拍 ready=111；下一拍 (7,0,0)、ready=110。

要拦住的错误 提前一拍阻塞；延后一拍允许第 8 项。

实际 Formal 见证 CAP-02 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	1
arready	111	111	111	111	111	111	111	110
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	111	111	111	111	110
A	1	1	1	1	1	1	1	1
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
Nnext*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/1/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-02 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1010，N=7/0/0，ready=110，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_occ[0]==6 && h_prev_occ[1]==0 && h_prev_occ[2]==0 && h_prev_a &&
        h_prev_src==0 && !h_prev_r && occ[0]==7 && !r_fire && arready==6
);
```

证据 build/formal/cap/run/engine_0/trace13.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, C_CAPACITY700。

分支 1 个；本页已列出该场景 cover。

CAP 03 单源七项被限制

容量与来源保护 | 合法业务

前提与动作 N=(7,0,0)，无 R；持续向 src0 请求。

预期 不得接收第 8 项；src1/2 仍各可接收首项。

要拦住的错误 只检查总数<9 导致一源占满。

实际 Formal 见证 CAP-03 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	110
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	111	111	111	111	110
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
Nnext*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-03 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0010，N=7/0/0，ready=110，pending_cnt=3。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_03_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==7 && occ[1]==0 && occ[2]==0 && !r_fire && arvalid && src==0 && !ar_fire
        && arready==6
);
```

证据 build/formal/cap/run/engine_0/trace14.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, M_QUOTA。

分支 1 个；本页已列出该场景 cover。

CAP 04 总量七项但多源有数据

容量与来源保护 | 合法业务

前提与动作 例如 (6,1,0)、(5,1,1)；无 R。

预期 ready=111；不能把所有 N=7 都按单源七项处理。

要拦住的错误 只依总量判断额度。

实际 Formal 见证 CAP-04 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	2	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	111	111	111	110	011
A	1	1	1	1	1	1	1	1
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/0	0/0/1	1/0/1	2/0/1	3/0/1	4/0/1	5/0/1	6/0/1
Nnext*	0/0/1	1/0/1	2/0/1	3/0/1	4/0/1	5/0/1	6/0/1	7/0/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-04 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1010，N=6/0/1，ready=111，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==7 && ((occ[0]>0)+(occ[1]>0)+(occ[2]>0))>=2 && !r_fire && arready==7
);
```

证据 build/formal/cap/run/engine_0/trace11.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

CAP 05 总量八项且一源为空

容量与来源保护 | 合法业务

前提与动作 例如 (7,1,0)、(4,4,0)；无 R，各 src 尝试。

预期 ready=100；空源的保留首项可接收，已有源不可增长。

要拦住的错误 只做每源≤7；误接 (5,4,0)。

实际 Formal 见证 CAP-05 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
arvalid	1	1	1	1	1	1	1	0
src	1	1	1	1	0	1	0	1
arready	111	111	111	111	111	111	111	100
rvalid	000	000	000	000	000	000	000	000
rready	111	111	111	111	111	111	100	101
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/1/0	0/2/0	0/3/0	0/4/0	0/5/0	1/5/0	1/6/0	2/6/0
Nnext*	0/2/0	0/3/0	0/4/0	0/5/0	1/5/0	1/6/0	2/6/0	2/6/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-05 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0000，N=2/6/0，ready=100，pending_cnt=6。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_05_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==8 && occ[0]>0 && occ[1]>0 && occ[2]==0 && !r_fire && arready==4
);
```

证据 build/formal/cap/run/engine_0/trace24.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, C_CAPACITY440。

分支 1 个；本页已列出该场景 cover。

CAP 06 总量八项且三源非空

容量与来源保护 | 合法业务

前提与动作 例如 (6,1,1)、(3,3,2)；无 R。

预期 ready=111；任何一源可使用最后一项。

要拦住的错误 把 N=8 一律只开放某个源。

实际 Formal 见证 CAP-06 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
arvalid	1	1	1	1	1	1	1	0
src	0	1	0	0	1	0	2	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	000
rready	111	111	111	111	111	111	100	101
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/1/0	1/1/0	1/2/0	2/2/0	3/2/0	3/3/0	4/3/0	4/3/1
Nnext*	1/1/0	1/2/0	2/2/0	3/2/0	3/3/0	4/3/0	4/3/1	4/3/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-06 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0000，N=4/3/1，ready=111，pending_cnt=6。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==8 && occ[0]>0 && occ[1]>0 && occ[2]>0 && !r_fire && arready==7
);
```

证据 build/formal/cap/run/engine_0/trace21.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

CAP 07 总量九项无回收

容量与来源保护 | 合法业务

前提与动作 任意合法满载分布，三源均非空；R=0。

预期 ready=000；arvalid 必须为 0。

要拦住的错误 满表仍接受；超出 9 项。

实际 Formal 见证 CAP-07 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	0
src	1	1	1	0	1	0	2	0
arready	111	111	111	111	111	111	100	000
rvalid	000	000	000	000	000	000	000	000
rready	111	111	111	111	111	110	101	111
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/2/0	0/3/0	0/4/0	0/5/0	1/5/0	1/6/0	2/6/0	2/6/1
Nnext*	0/3/0	0/4/0	0/5/0	1/5/0	1/6/0	2/6/0	2/6/1	2/6/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-07 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0010，N=2/6/1，ready=000，pending_cnt=8。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && !r_fire && arready==0 && !arvalid
);
```

证据 build/formal/cap/run/engine_0/trace28.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, E05_ZERO_READY, M_RANGE。

分支 1 个；本页已列出该场景 cover。

CAP 08 满表回收多项源

容量与来源保护 | 合法业务

前提与动作 (7,1,1) 中 R0，或 (4,4,1) 中 R0/R1；本拍 R=1。

预期 回收后共享额度减少 1，本拍 ready=111。

要拦住的错误 等待下一拍才释放 ready，造成气泡。

实际 Formal 见证 CAP-08 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	2	1	0	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	001
rready	111	111	111	111	111	111	111	111
A	1	1	1	1	1	1	1	1
R	0	0	0	0	0	0	0	1
N0/N1/N2*	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	6/0/1	6/1/1	7/1/1
Nnext*	3/0/0	4/0/0	5/0/0	6/0/0	6/0/1	6/1/1	7/1/1	7/1/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-08 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=1001，N=7/1/1，ready=111，pending_cnt=9。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && r_fire && occ[winner]>1 && arready==7
);
```

证据 build/formal/cap/run/engine_0/trace31.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, C_FULL_REUSE。

分支 1 个；本页已列出该场景 cover。

CAP 09 满表回收唯一项源

容量与来源保护 | 合法业务

前提与动作 满表，回收来源只占一项；例如 4/4/1 回收 XAU2；分别观察三个来源的本拍接收资格；代表波形采用来源对称的另一组取值。

预期 本拍仅刚变空的来源可接收；例如回收 XAU2 则 ready=100，回收 XAU0 则 ready=001。

要拦住的错误 按物理空闲数接收 src0/1。

实际 Formal 见证 CAP-09 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	1
src	0	1	1	1	1	1	2	0
arready	111	111	111	111	111	111	100	001
rvalid	000	000	000	000	000	000	000	001
rready	111	111	110	111	111	111	111	111
A	1	1	1	1	1	1	1	1
R	0	0	0	0	0	0	0	1
N0/N1/N2*	0/2/0	1/2/0	1/3/0	1/4/0	1/5/0	1/6/0	1/7/0	1/7/1
Nnext*	1/2/0	1/3/0	1/4/0	1/5/0	1/6/0	1/7/0	1/7/1	1/7/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-09 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=1001，N=1/7/1，ready=001，pending_cnt=9。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_09_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && r_fire && winner==0 && occ[0]==1 && arready==(3'b1<<0)
);
```

证据 build/formal/cap/run/engine_0/trace32.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, M_QUOTA。

分支 3 个；其余分支完整代码见附录 B。

CAP 10 满表同源出入保持九项

容量与来源保护 | 合法业务

前提与动作 有效 R 且 AR 选择 ready 源；R 与 AR 同拍。

预期 N_next=9；满表必须复用刚退役物理项，旧响应仍正确。

要拦住的错误 先写新字段污染旧 R；先清空覆盖新写入。

实际 Formal 见证 CAP-10 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	1	0	2
arready	111	111	111	111	111	111	111	100
rvalid	000	000	000	000	000	000	000	100
rready	111	111	111	111	111	100	110	110
A	1	1	1	1	1	1	1	1
R	0	0	0	0	0	0	0	1
N0/N1/N2*	1/0/1	2/0/1	3/0/1	4/0/1	5/0/1	6/0/1	6/1/1	7/1/1
Nnext*	2/0/1	3/0/1	4/0/1	5/0/1	6/0/1	6/1/1	7/1/1	7/1/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-10 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=1001，N=7/1/1，ready=100，pending_cnt=8。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_10_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && ar_fire && r_fire && src==winner && h_response_matches
);
```

证据 build/formal/cap/run/engine_0/trace30.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_DATA, A_R_METADATA, C_FULL_REUSE。

分支 1 个；本页已列出该场景 cover。

CAP 11 单源七项回收并重填

容量与来源保护 | 合法业务

前提与动作 N=(7,0,0)，本拍 R0；同拍 AR src0。

预期 ready=111；N_next=(7,0,0)，下一拍无 R 时回到 110。

要拦住的错误 忽略 R 的前瞻释放。

实际 Formal 见证 CAP-11 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	001
rready	000	111	111	111	111	111	110	011
A	1	1	1	1	1	1	1	1
R	0	0	0	0	0	0	0	1
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
Nnext*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-11 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1011，N=7/0/0，ready=111，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_11_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==7 && occ[1]==0 && occ[2]==0 && rvalid==1 && ar_fire && src==0 &&
        arready==7
);
```

证据 build/formal/cap/run/engine_0/trace10.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

CAP 12 八项两源回收多项源

容量与来源保护 | 合法业务

前提与动作 N=(7,1,0)，本拍 R0；三种 src 分别尝试。

预期 ready=111；释放一个共享名额，可由任一源使用。

要拦住的错误 继续只允许原空源。

实际 Formal 见证 CAP-12 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
arvalid	1	1	1	1	1	1	1	0
src	0	0	0	1	0	0	0	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	001
rready	111	111	111	111	111	111	010	111
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	1
N0/N1/N2*	1/0/0	2/0/0	3/0/0	4/0/0	4/1/0	5/1/0	6/1/0	7/1/0
Nnext*	2/0/0	3/0/0	4/0/0	4/1/0	5/1/0	6/1/0	7/1/0	6/1/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-12 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=7/1/0，ready=111，pending_cnt=5。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_12_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==7 && occ[1]==1 && occ[2]==0 && rvalid==1 && arready==7
);
```

证据 build/formal/cap/run/engine_0/trace25.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

CAP 13 八项两源回收单项源

容量与来源保护 | 合法业务

前提与动作 N=(7,1,0)，本拍 R1；三种 src 分别尝试。

预期 回收后 (7,0,0)，ready=110；src0 仍不可加项。

要拦住的错误 任意 R 都将 ready 设成 111。

实际 Formal 见证 CAP-13 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
arvalid	1	1	1	1	1	1	1	0
src	0	0	0	0	0	0	0	1
arready	111	111	111	111	111	111	111	110
rvalid	000	000	000	000	000	000	000	010
rready	111	111	111	111	111	111	101	011
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	1
N0/N1/N2*	0/1/0	1/1/0	2/1/0	3/1/0	4/1/0	5/1/0	6/1/0	7/1/0
Nnext*	1/1/0	2/1/0	3/1/0	4/1/0	5/1/0	6/1/0	7/1/0	7/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-13 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=7/1/0，ready=110，pending_cnt=6。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_13_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==7 && occ[1]==1 && occ[2]==0 && rvalid==2 && arready==6
);
```

证据 build/formal/cap/run/engine_0/trace26.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

CAP 14 总量不变但 ready 改变

容量与来源保护 | 合法业务

前提与动作 N=(6,1,1)，R1 与 AR0 同拍；从 (6,1,1) 变为 (7,0,1)。

预期 两拍总量均为 8；下一拍无 R 时 ready 从 111 变 010。

要拦住的错误 用总计数缓存 ready，忽略来源分布。

实际 Formal 见证 CAP-14 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	1
src	0	2	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	010
rvalid	000	000	000	000	000	000	010	000
rready	111	111	111	111	111	101	110	010
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	1	0
N0/N1/N2*	1/1/0	2/1/0	2/1/1	3/1/1	4/1/1	5/1/1	6/1/1	7/0/1
Nnext*	2/1/0	2/1/1	3/1/1	4/1/1	5/1/1	6/1/1	7/0/1	7/0/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-14 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0000，N=7/0/1，ready=010，pending_cnt=6。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_14_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_occ[0]==6 && h_prev_occ[1]==1 && h_prev_occ[2]==1 && h_prev_r &&
        h_prev_rsrc==1 && h_prev_a && h_prev_src==0 && occ[0]==7 && occ[1]==0 &&
        occ[2]==1 && !r_fire && arready==2
);
```

证据 build/formal/cap/run/engine_0/trace34.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

CAP 15 总量下降但原源仍被挡

容量与来源保护 | 合法业务

前提与动作 N=(7,1,0)，R1，AR0 未接收；下一拍继续 src0 请求。

预期 总量从 8 降到 7，src0 ready 仍为 0。

要拦住的错误 认为总量下降就全部解除阻塞。

实际 Formal 见证 CAP-15 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	110	110
rvalid	000	000	000	000	000	000	010	000
rready	111	111	111	111	111	110	011	010
A	1	1	1	1	1	1	0	0
R	0	0	0	0	0	0	1	0
N0/N1/N2*	1/1/0	2/1/0	3/1/0	4/1/0	5/1/0	6/1/0	7/1/0	7/0/0
Nnext*	2/1/0	3/1/0	4/1/0	5/1/0	6/1/0	7/1/0	7/0/0	7/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-15 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0000，N=7/0/0，ready=110，pending_cnt=7。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_15_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_occ[0]==7 && h_prev_occ[1]==1 && h_prev_occ[2]==0 && h_prev_r &&
        h_prev_rsrc==1 && !h_prev_a && occ[0]==7 && used==7 && !r_fire && arvalid &&
        src==0 && !ar_fire
);
```

证据 build/formal/cap/run/engine_0/trace35.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

CAP 16 最后一项回收后可换源

容量与来源保护 | 合法业务

前提与动作 总量只剩一项，可位于任一 XAU，本拍返回该旧命令；同拍接收另一来源的新命令。

预期 ready=111；旧源归零，新源一项，总量保持 1。

要拦住的错误 清空时误拒新来源。

实际 Formal 见证 CAP-16 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	0	0	0	0	0	0	1
src	1	3	2	2	0	2	2	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	000	010
rready	000	111	111	111	111	111	111	010
A	1	0	0	0	0	0	0	1
R	0	0	0	0	0	0	0	1
N0/N1/N2*	0/0/0	0/1/0	0/1/0	0/1/0	0/1/0	0/1/0	0/1/0	0/1/0
Nnext*	0/1/0	0/1/0	0/1/0	0/1/0	0/1/0	0/1/0	0/1/0	1/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-16 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=0/1/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_16_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==1 && r_fire && ar_fire && src!=winner && arready==7
);
```

证据 build/formal/cap/run/engine_0/trace19.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_ID_ORDER。

分支 1 个；本页已列出该场景 cover。

CAP 17 排空到零

容量与来源保护 | 合法业务

前提与动作 N=1，有效 R，无 AR；R 完成后观察下一拍。

预期 N_next=0，各源占用全零；释放后 ready=111。

要拦住的错误 计数下溢或有幽灵 entry。

实际 Formal 见证 CAP-17 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
arvalid	0	0	0	0	0	0	0	0
src	2	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	111
rvalid	000	000	000	000	000	000	001	000
rready	111	111	111	111	111	111	011	111
A	0	0	0	0	0	0	0	0
R	0	0	0	0	0	0	1	0
N0/N1/N2*	1/0/0	1/0/0	1/0/0	1/0/0	1/0/0	1/0/0	1/0/0	0/0/0
Nnext*	1/0/0	1/0/0	1/0/0	1/0/0	1/0/0	1/0/0	0/0/0	0/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-17 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0000，N=0/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_17_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_n==1 && h_prev_r && !h_prev_a && used==0 && arready==7
);
```

证据 build/formal/cap/run/engine_0/trace27.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_HAS_PENDING, M_RANGE。

分支 1 个；本页已列出该场景 cover。

CAP 18 八种 ready 码全部出现

容量与来源保护 | 合法业务

前提与动作 合法 Ni，允许有效 R；遍历容量状态与 R 来源。

预期 000、001、010、011、100、101、110、111 均有合法容量实例；bit 顺序为 [2:0]。

要拦住的错误 错把 110 解释为 XAU0/1 ready。

实际 Formal 见证 CAP-18 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
arvalid	1	1	1	1	1	1	1	0
src	1	1	1	0	1	0	2	0
arready	111	111	111	111	111	111	100	000
rvalid	000	000	000	000	000	000	000	000
rready	111	111	111	111	111	110	101	111
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/2/0	0/3/0	0/4/0	0/5/0	1/5/0	1/6/0	2/6/0	2/6/1
Nnext*	0/3/0	0/4/0	0/5/0	1/5/0	1/6/0	2/6/0	2/6/1	2/6/1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-18 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0010，N=2/6/1，ready=000，pending_cnt=8。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_18_READY000: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==0 && used>0
);
```

证据 build/formal/cap/run/engine_0/trace28.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 8 个；其余分支完整代码见附录 B。

CAP 19 来源对称性

容量与来源保护 | 合法业务

前提与动作 任一合法场景；把 0/1/2 的计数、AR 与 R 来源一起置换。

预期 ready 置换后一致；容量规则不偏袒编号；R 轮询顺序另受指针影响。

要拦住的错误 硬编码某一路额度与别路不同。

实际 Formal 见证 CAP-19 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	1
arready	111	111	111	111	111	111	111	110
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	111	111	111	111	110
A	1	1	1	1	1	1	1	1
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
Nnext*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/1/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-19 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1010，N=7/0/0，ready=110，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_19_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==7 && used==7 && !r_fire && arready==(3'b111^(3'b1<<0))
);
```

证据 build/formal/cap/run/engine_0/trace13.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 3 个；其余分支完整代码见附录 B。

CAP 20 不合法分布 5/4/0

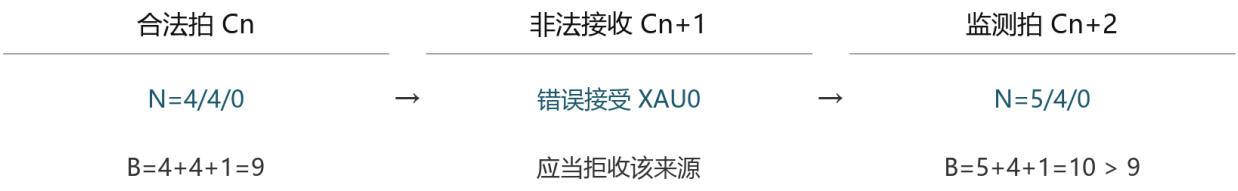
容量与来源保护 | 内部注错

前提与动作 总数 9，各源都不超过 7，但空源无首项余量；内部状态强制为 (5,4,0)，或注错让 (4,4,0) 再收 src0。

预期 违反 sum(max(Ni,1))≤9；正常 admission 应阻止形成。

要拦住的错误 只校验总量≤9 和单源≤7。

注错或观测边界示意 未作为已命中的 Formal 波形



本图描述检查责任或拟注错路径，不声称该非法内部状态已经被注入。

图 CAP-20 未执行注错或不可直接观察的边界示意。

检查边界 A_CC_READY 与 A_CC_BUDGET 检查错误接收后果；物理强制 5/4/0 需内部注错。

[NO_REF_MODEL_HISTORY] 空来源按 1 项保留计算，三源总预算不得超过 9。

```
A_CC_BUDGET: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_budget<=9
);
```

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

关联属性 A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, M_QUOTA。

CAP 21 不合法单源八项

容量与来源保护 | 内部注错

前提与动作 强制 N=(8,0,0) 或错误接收第八条同源命令；检查容量及响应记账。

预期 正常协议下不可达；需捕获超额或异常接收。

要拦住的错误 门限比较使用≤7 而非允许后的合法性。

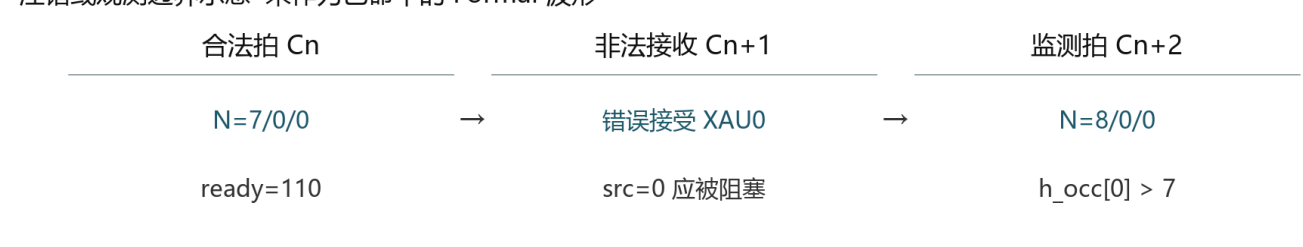


图 CAP-21 未执行注错或不可直接观察的边界示意。

检查边界 A_CC_READY/A_CC_SRC_LIMIT 检查第八项错误接收；物理强制占用需内部注错。

[NO_REF_MODEL_HISTORY] 任一 XAU 尚未返回的命令不能超过 7 条。

```
A_CC_SRC_LIMIT: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_occ[cs]<=7
);
```

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_TOTAL, A_READY, M_QUOTA。

CAP 22 非空并不代表可回收

容量与来源保护 | 合法业务

前提与动作 容量表某源 Ni>0，但队头未完成或 rready=0；试图选该源作为 R。

预期 实际 RTL 不得 R；枚举里的 R 选择仅在队头完成且 ready 时成立。

要拦住的错误 将容量枚举的抽象 R 选择当成任意可达时刻。

实际 Formal 见证 CAP-22 上升沿采样值

	S0	S1
arvalid	1	0
src	0	0
arready	111	111
rvalid	000	000
rready	000	001
A	1	0
R	0	0
N0/N1/N2*	0/0/0	1/0/0
Nnext*	1/0/0	1/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-22 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_22_UNDONE0: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[0] && !m_done[head[0]] && rready[0] && !rvalid[0]
);
```

证据 build/formal/cap/run/engine_0/trace2.vcd；日志 step=2，图示采样 S1。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_READY, A_XAU_R_HEAD_COMPLETE。

分支 6 个；其余分支完整代码见附录 B。

CAP 23 发射不释放 entry

容量与来源保护 | 合法业务

前提与动作 达到容量限制；待发命令仲裁成功；无 R，可有 req_success。

预期 N 不变，容量不会因 issued 而释放。

要拦住的错误 在发射成功时减占用或归还 entry。

实际 Formal 见证 CAP-23 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	110
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	111	111	111	111	110
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
Nnext*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-23 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0110，N=7/0/0，ready=110，pending_cnt=3。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_23_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used>=7 && issue_fire && !r_fire && !ar_fire && arready!=7
);
```

证据 build/formal/cap/run/engine_0/trace15.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, M_STATE。

分支 1 个；本页已列出该场景 cover。

CAP 24 MEM 返回不释放 entry

容量与来源保护 | 合法业务

前提与动作 达到容量限制；本拍 MEM 返回；无 R，允许 pc_err 任意。

预期 N 不变；最早下一拍才可能 R 并释放。

要拦住的错误 在 MEM 返回拍清 valid 或开放容量。

实际 Formal 见证 CAP-24 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
arvalid	1	1	1	1	1	1	1	1
src	0	0	0	0	0	0	0	0
arready	111	111	111	111	111	111	111	110
rvalid	000	000	000	000	000	000	000	000
rready	000	111	111	111	111	111	111	110
A	1	1	1	1	1	1	1	0
R	0	0	0	0	0	0	0	0
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
Nnext*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CAP-24 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0010，N=7/0/0，ready=110，pending_cnt=3。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CAP_24_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used>=7 && mem_capture_due && !r_fire && !ar_fire && arready!=7
);
```

证据 build/formal/cap/run/engine_0/trace14.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_NO_EARLY_BYPASS。

分支 1 个；本页已列出该场景 cover。

ENT 01 一次 AR 同时占两表

Entry 生命周期 | 合法业务

前提与动作 一次 A=1，有一项可用；观察最终响应及内部结构。

预期 CMDBUF/OTB 一一对应预留；不能只有一张表分配；端口模型只能查可观察后果。

要拦住的错误 双表索引不同、漏预留。

实际 Formal 见证 ENT-01 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	–	T2	–	T3	T4	–
arready	111	111	111	111	111	111	111	111
req token*	–	–	T0	–	T1	T1	T1	T1
I	0	0	1	0	0	0	0	0
MEM token*	–	–	–	–	–	–	T0	–
R token*	–	–	–	–	–	–	–	T0
N0/N1/N2*	0/0/0	0/0/1	0/1/1	0/1/1	0/2/1	0/2/1	0/3/1	0/4/1
pending_cnt	1	2	1	2	2	3	4	4
A/I/M/R*	1000	1000	0100	1000	0000	1000	1010	0001

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-01 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=0/4/1，ready=111，pending_cnt=4。

[NO_REF_MODEL_HISTORY] AR 累计接收减 R 累计返回：全局最多仍在途 9 条。

```
A_CC_TOTAL: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_total<=9
);
```

```
C_CC_ENT_01_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches
);
```

证据 build/formal/ent/run/engine_0/trace3.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_R_DATA, M_ALLOC, M_STATE。

分支 1 个；本页已列出该场景 cover。

观测范围 该见证检查端口事务的占用与复用；图示 token 是验证记录，不能据此认定 DUT 选用了某个物理槽位。

ENT 02 待发项无进展仍保留

Entry 生命周期 | 合法业务

前提与动作 Bank 忙，未选中，R 不涉及此项；多拍等待。

预期 地址/ID/源/port/异常与年龄保持；不因等待覆盖。

要拦住的错误 无效选择把待发项清除。

实际 Formal 见证 ENT-02 上升沿采样值

	S0	S1	S2	S3	S4
AR token*	T0	–	–	–	–
arready	111	111	111	111	111
req token*	–	–	–	T0	T0
I	0	0	0	0	0
MEM token*	–	–	–	–	–
R token*	–	–	–	–	–
N0/N1/N2*	0/0/0	0/1/0	0/1/0	0/1/0	0/1/0
pending_cnt	1	1	1	1	1
A/I/M/R*	1000	0000	0000	0000	0000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-02 实际正向见证；黄色为关键采样点。

关键采样 S4 A/I/M/R=0000，N=0/1/0，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_ENT_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used>0 && waiting>0 && h_quiet_run>=3 && !select_valid && !ar_fire && !r_fire &&
        !issue_fire
);
```

证据 build/formal/ent/run/engine_0/trace2.vcd；日志 step=5，图示采样 S4。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_METADATA。

分支 1 个；本页已列出该场景 cover。

ENT 03 仲裁失败保留年龄

Entry 生命周期 | 合法业务

前提与动作 当前 req 对应命令失败；后续和新老命令竞争。

预期 entry 仍待发，年龄不重置；不重新分配 OTB。

要拦住的错误 重试当成新命令排队。

实际 Formal 见证 ENT-03 上升沿采样值

	S0	S1	S2	S3
AR token*	T0	T1	–	–
arready	111	111	111	111
req token*	–	–	T0	T0
I	0	0	0	0
MEM token*	–	–	–	–
R token*	–	–	–	–
N0/N1/N2*	0/0/0	0/0/1	0/0/2	0/0/2
pending_cnt	1	2	2	2
A/I/M/R*	1000	1000	0000	0000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-03 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=0000，N=0/0/2，ready=111，pending_cnt=2。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_ENT_03_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    retry_selected && req==last_retry_req && arb_addr==last_retry_addr && req!=0
);
```

证据 build/formal/ent/run/engine_0/trace0.vcd；日志 step=4，图示采样 S3。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, C_RETRY_NEXT_CYCLE。

分支 1 个；本页已列出该场景 cover。

ENT 04 已发未返回继续占用

Entry 生命周期 | 合法业务

前提与动作 I=1 后尚未到 N+4；发生其他 AR/R。

预期 原项保持 reserved，不参与重发，不可分给新命令。

要拦住的错误 issued 项被视为空闲。

实际 Formal 见证 ENT-04 上升沿采样值

	S0	S1	S2	S3
AR token*	T0	T1	–	T2
arready	111	111	111	111
req token*	–	–	T0	T1
I	0	0	1	0
MEM token*	–	–	–	–
R token*	–	–	–	–
N0/N1/N2*	0/0/0	0/0/1	0/0/2	0/0/2
pending_cnt	1	2	1	2
A/I/M/R*	1000	1000	0100	1000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-04 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=1000，N=0/0/2，ready=111，pending_cnt=2。

[NO_REF_MODEL_HISTORY] AR 累计接收减 R 累计返回：全局最多仍在途 9 条。

```
A_CC_TOTAL: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_total<=9
);
```

```
C_CC_ENT_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    (m_live&m_sent&~m_done)!=0 && ar_fire && !r_fire
);
```

证据 build/formal/ent/run/engine_0/trace1.vcd；日志 step=4，图示采样 S3。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_REQ, A_R_DATA, M_STATE。

分支 1 个；本页已列出该场景 cover。

ENT 05 已返回未响应继续占用

Entry 生命周期 | 合法业务

前提与动作 MEM 已完成，源头阻塞或 rready=0；多拍延迟后 R。

预期 缓存数据和 pc_err 保持直到该事务实际 R。

要拦住的错误 MEM 后自动释放，数据被新 AR 覆盖。

实际 Formal 见证 ENT-05 上升沿采样值

	S3	S4	S5	S6	S7	S8	S9	S10
AR token*	–	T2	–	T3	–	–	T4	T5
arready	111	111	111	111	111	111	111	111
req token*	T1	–	T1	T1	T1	T1	T1	T1
I	0	0	0	0	0	0	0	0
MEM token*	–	–	–	T0	–	–	–	–
R token*	–	–	–	–	–	–	–	T0
N0/N1/N2*	0/0/2	0/0/2	0/0/3	0/0/3	0/0/4	0/0/4	0/0/4	1/0/4
pending_cnt	1	2	2	3	3	3	4	5
A/I/M/R*	0000	1000	0000	1010	0000	0000	1000	1001

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-05 实际正向见证；黄色为关键采样点。

关键采样 S10 A/I/M/R=1001，N=1/0/4，ready=111，pending_cnt=5。

[NO_REF_MODEL_HISTORY] AR 累计接收减 R 累计返回：全局最多仍在途 9 条。

```
A_CC_TOTAL: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_total<=9
);
```

```
C_CC_ENT_05_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && h_done_wait[rsp_e]>=3
);
```

证据 build/formal/ent/run/engine_0/trace7.vcd；日志 step=11，图示采样 S10。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_XAU_R_DATA_ORDER, A_XAU_R_RESP_ORDER。

分支 1 个；本页已列出该场景 cover。

ENT 06 空闲物理洞再次使用

Entry 生命周期 | 合法业务

前提与动作 多项中间某项先 R，其他项存活；随后 AR 使用空闲位置。

预期 新项年龄为最新，与物理索引大小无关。

要拦住的错误 按物理 entry 编号判断最老。

实际 Formal 见证 ENT-06 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	T2	–	–	T3	T4	–	T5
arready	111	111	111	111	111	111	111	111
req token*	–	T0	T1	–	T2	T2	T2	T2
I	0	1	1	0	0	0	0	0
MEM token*	–	–	–	–	–	T0	T1	–
R token*	–	–	–	–	–	–	T0	–
N0/N1/N2*	0/0/1	0/0/2	0/0/3	0/0/3	0/0/3	0/0/4	0/0/5	0/0/4
pending_cnt	2	2	1	1	2	3	3	4
A/I/M/R*	1000	1100	0100	0000	1000	1010	0011	1000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-06 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=1000，N=0/0/4，ready=111，pending_cnt=4。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_ENT_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && h_hole_reused && h_total_r>0
);
```

证据 build/formal/ent/run/engine_0/trace5.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_REQ, A_XAU_R_ID_ORDER, M_AGE_UNIQUE。

分支 1 个；本页已列出该场景 cover。

观测范围 该见证检查端口事务的占用与复用；图示 token 是验证记录，不能据此认定 DUT 选用了某个物理槽位。

ENT 07 满表同项退役与分配

Entry 生命周期 | 合法业务

前提与动作 9 项有效，本拍 R+A；同一物理槽交接旧/新事务。

预期 旧 R 读旧字段；时钟后新项 valid=1、issued/done=0；数据错误位清理。

要拦住的错误 退役清除优先级压过新分配。

实际 Formal 见证 ENT-07 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
AR token*	T2	T3	T4	T5	T6	T7	T8	T9
arready	111	111	111	111	111	111	111	111
req token*	T0	T1	T1	T0	T0	T0	T0	T0
I	0	0	1	0	0	0	0	0
MEM token*	–	–	–	–	–	–	T1	–
R token*	–	–	–	–	–	–	–	T1
N0/N1/N2*	0/1/1	0/1/2	0/1/3	1/1/3	2/1/3	2/2/3	2/3/3	3/3/3
pending_cnt	3	4	4	5	6	7	8	9
A/I/M/R*	1000	1000	1100	1000	1000	1000	1010	1001

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-07 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=1001，N=3/3/3，ready=111，pending_cnt=9。

[NO_REF_MODEL_HISTORY] AR 累计接收减 R 累计返回：全局最多仍在途 9 条。

```
A_CC_TOTAL: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_total<=9
);
```

```
C_CC_ENT_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && ar_fire && r_fire && alloc_e==rsp_e && h_response_matches
);
```

证据 build/formal/ent/run/engine_0/trace6.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_R_DATA, C_FULL_REUSE, M_STATE。

分支 1 个；本页已列出该场景 cover。

观测范围 该见证检查端口事务的占用与复用；图示 token 是验证记录，不能据此认定 DUT 选用了某个物理槽位。

ENT 08 非满表同拍出入

Entry 生命周期 | 合法业务

前提与动作 旧空闲项和本拍退役项都存在；R+A。

预期 允许实现选择任一合法空闲位置；不能把指定槽号作为外部协议要求。

要拦住的错误 顶层检查器绑定实现分配索引。

实际 Formal 见证 ENT-08 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	–	T2	–	T3	T4	T5
arready	111	111	111	111	111	111	111	111
req token*	–	–	T0	–	T1	T1	T1	T1
I	0	0	1	0	0	0	0	0
MEM token*	–	–	–	–	–	–	T0	–
R token*	–	–	–	–	–	–	–	T0
N0/N1/N2*	0/0/0	0/0/1	0/1/1	0/1/1	0/2/1	0/2/1	0/3/1	0/4/1
pending_cnt	1	2	1	2	2	3	4	5
A/I/M/R*	1000	1000	0100	1000	0000	1000	1010	1001

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-08 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=0/4/1，ready=111，pending_cnt=5。

[NO_REF_MODEL_HISTORY] AR 累计接收减 R 累计返回：全局最多仍在途 9 条。

```
A_CC_TOTAL: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_total<=9
);
```

```
C_CC_ENT_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used<9 && used>1 && ar_fire && r_fire && h_response_matches
);
```

证据 build/formal/ent/run/engine_0/trace4.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_READY, A_R_METADATA, M_ALLOC。

分支 1 个；本页已列出该场景 cover。

ENT 09 错误位与数据残留

Entry 生命周期 | 合法业务

前提与动作 旧事务 pc_err=1、数据全 1，随后退役；同槽新事务正常数据全 0。

预期 新 R 为新值及 rresp=00；不能继承旧 done/pc_err。

要拦住的错误 仅清 valid 未清错误完成状态。

实际 Formal 见证 ENT-09 上升沿采样值

	S7	S8	S9	S10	S11	S12	S13	S14
req token*	T5	T6	T7	–	T8	T8	T8	T8
I	1	1	1	0	0	0	0	0
MEM token*	T1	T2	T3	T4	T5	T6	T7	–
mem_rdata	0	0	D2	D2	D2	0	0	–
pc_err	0	0	0	0	0	0	0	0
R token*	T0	–	T2	T1	T4	T5	T6	T7
rdata	ALL1	–	0	0	D2	D2	0	0
rresp	01	–	00	00	00	00	00	00

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-09 实际正向见证；黄色为关键采样点。

关键采样 S14 A/I/M/R=1001，N=3/1/1，ready=111，pending_cnt=4。

[REF_MODEL] R 错误状态来自预期事务；地址越界优先于奇偶校验异常。

```
A_R_RESP: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rresp == expected_resp)
);
```

```
C_CC_ENT_09_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && h_clean_reuse[rsp_e] && m_err[rsp_e]==0 && !m_pc[rsp_e] &&
        m_data[rsp_e]==0 && rresp==0
);
```

D2 = 128'hFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFE

证据 build/formal/ent/run/engine_0/trace8.vcd；日志 step=15，图示采样 S14。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_DATA。

分支 1 个；本页已列出该场景 cover。

ENT 10 双重分配

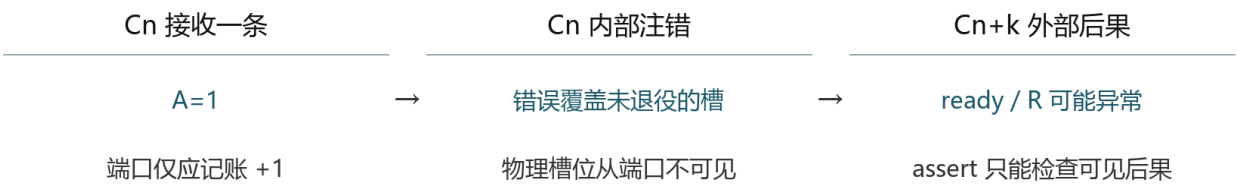
Entry 生命周期 | 内部注错

前提与动作 内部强制 alloc 指向尚未退役的有效项；继续该项及新项的响应流程。

预期 注错应触发结构诊断或丢单/字段断言；不能以模型自检替代物理状态证明。

要拦住的错误 误把软件参考模型 M_ALLOC 当成 RTL 内部断言。

注错或观测边界示意 未作为已命中的 Formal 波形



本图描述检查责任或拟注错路径，不声称该非法内部状态已经被注入。

图 ENT-10 未执行注错或不可直接观察的边界示意。

检查边界 双重分配的丢单/串单由 A_REQ/A_R_*/A_XAU_R_* 检查；物理 alloc 冲突不是端口可观察事件。

[REF_MODEL] 有效 R 的 128 位数据必须来自同一个队头事务的 MEM 捕获记录。

```
A_XAU_R_DATA_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rdata == m_data[oldest]))
);
```

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_DATA, A_XAU_R_ID_ORDER。

ENT 11 重复完成或重复响应

Entry 生命周期 | 内部注错

前提与动作 对同一已完成命令注入重复 done/retire；检查 R 计数与逐源队头。

预期 每次 AR 只能产生一次 R；不得使占用下溢或重复交付。

要拦住的错误 已退役项仍留 valid 或 calendar 标记。

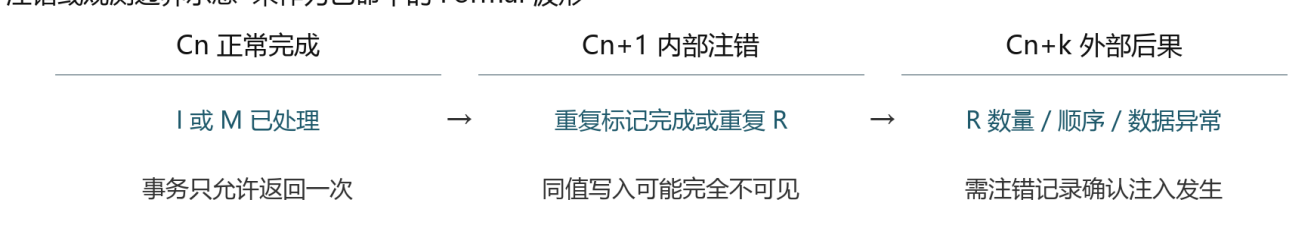


图 ENT-11 未执行注错或不可直接观察的边界示意。

检查边界 A_CC_R_NONEMPTY/A_XAU_R_* 检查重复响应；重复 done 若未改变外部行为不保证检出。

[NO_REF_MODEL_HISTORY] 只要该源 rvalid 有效，就必须有之前接受而尚未返回的命令。

```
A_CC_R_NONEMPTY: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid[cs] |-> (h_occ[cs]!=0)
);
```

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_XAU_R_HAS_PENDING, A_XAU_R_ID_ORDER。

ENT 12 长期槽位复用

Entry 生命周期 | 合法业务

前提与动作 多次填满、部分回收，再填满；累计事务远多于 9，ID 也重复。

预期 年龄关系不受物理槽位和 ID 回绕影响；响应数守恒。

要拦住的错误 年龄矩阵旧位残留。

实际 Formal 见证 ENT-12 引导输入作业 原生回放通过

	S24	S25	S26	S27	S28	S29	S30	S31	S32
AR token*	T13	T14	T15	T16	T17	T18	T19	T20	T21
arready	111	111	111	111	111	111	111	111	111
req token*	T11	T12	T13	T14	T15	T16	T17	T18	T19
I	1	1	1	1	1	1	1	1	1
MEM token*	–	–	T9	T10	T11	T12	T13	T14	T15
R token*	T4	T5	T6	T7	T8	T10	T11	T9	T13
N0/N1/N2*	2/3/4	3/2/4	3/3/3	2/3/4	3/2/4	3/3/3	2/3/4	3/2/4	3/3/3
pending_cnt	2	2	2	2	2	2	2	2	2
A/I/M/R*	1101	1101	1111	1111	1111	1111	1111	1111	1111

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ENT-12 实际正向见证；黄色为关键采样点。

关键采样 S32 A/I/M/R=1111，N=3/3/3，ready=111，pending_cnt=2。

[REF_MODEL] 有效 R 的 rid 必须属于队头；不按 ID 数值排序，也不要求 ID 唯一。

```
A_XAU_R_ID_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rid == m_id[oldest]))
);
```

```
C_CC_ENT_12_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total_a>=20 && h_total_r>=12 && h_ever_full && ar_fire && h_duplicate_id
);
```

证据 build/formal_guided/ent_long_guided/run/engine_0/trace0.vcd；日志 step=33，图示采样 S32。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_REQ, M_AGE_UNIQUE。

分支 1 个；本页已列出该场景 cover。 本场景为单独引导见证，额外约束未进入正常证明。

观测范围 该见证检查端口事务的占用与复用；图示 token 是验证记录，不能据此认定 DUT 选用了某个物理槽位。

WIN 01 空表单命令最短路径

流水与乱序窗口 | 合法业务

前提与动作 空表；Bank 可按协议空闲/变忙；仲裁允许；C1 AR，C2 预选，C3 req 并成功。

预期 C1/C2 不得提前发射；C3 地址为该命令。

要拦住的错误 AR 直接旁路到 req，少一流水级。

实际 Formal 见证 WIN-01 上升沿采样值

	S0	S1	S2
AR token*	T0	–	–
pending_cnt	1	1	0
C2 window*	–	0	–
idle rank 1→5*	-----	1----	-----
6th outside*	–	–	–
C2 chosen*	–	T0	–
req token*	–	–	T0
req	00000000	00000000	00000001
I	0	0	1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-01 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0100，N=0/1/0，ready=111，pending_cnt=0。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_01_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_a_history[1] && h_prev_n==1 && used==1 && h_selected_prev && issue_fire &&
        h_total_a==1 && h_total_r==0
);
```

证据 build/formal/win/run/engine_0/trace5.vcd；日志 step=3，图示采样 S2。

关联属性 A_ARB_ADDR。

分支 1 个；本页已列出该场景 cover。

WIN 02 连续两条流水

流水与乱序窗口 | 合法业务

前提与动作 A、B 连续 AR，目标条件允许；A 在 C3 req 时同拍预选 B。

预期 B 可于 C4 req，不应强制预选空拍。

要拦住的错误 把预选和发射串行化。

实际 Formal 见证 WIN-02 上升沿采样值

	S0	S1	S2	S3
AR token*	T0	T1	–	–
pending_cnt	1	2	1	0
C2 window*	–	0	1	–
idle rank 1→5*	-----	1----	1----	-----
6th outside*	–	–	–	–
C2 chosen*	–	T0	T1	–
req token*	–	–	T0	T1
req	00000000	00000000	00000010	00000001
I	0	0	1	1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-02 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=0100，N=0/0/2，ready=111，pending_cnt=0。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_i && issue_fire && h_a_history[2:1]==2'b11
);
```

证据 build/formal/win/run/engine_0/trace11.vcd；日志 step=4，图示采样 S3。

关联属性 C_FOUR_SUCCESS。

分支 1 个；本页已列出该场景 cover。

WIN 03 窗口不足五项

流水与乱序窗口 | 合法业务

前提与动作 待发数分别 0/1/2/3/4；逐项改变 Bank 空闲状态。

预期 只在存在的待发项中选；空表无 req；不得读无效槽。

要拦住的错误 未屏蔽无效 entry。

实际 Formal 见证 WIN-03 上升沿采样值

	S0
AR token*	–
pending_cnt	0
C2 window*	–
idle rank 1→5*	-----
6th outside*	–
C2 chosen*	–
req token*	–
req	00000000
I	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-03 实际正向见证；黄色为关键采样点。

关键采样 S0 A/I/M/R=0000，N=0/0/0，ready=111，pending_cnt=0。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_03_P0: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==0 && !issue_fire && (h_rank_valid>>0)==0
);
```

证据 build/formal/win/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 M_ATTEMPT。

分支 5 个；其余分支完整代码见附录 B。

WIN 04 正好五项全部可用

流水与乱序窗口 | 合法业务

前提与动作 A..E 按年龄排列且目标均空闲；C2 选择。

预期 发最老 A；跨源也按全局 AR 年龄。

要拦住的错误 优先级编码器按槽号或 src 选中。

实际 Formal 见证 WIN-04 上升沿采样值

	S0	S1	S2	S3	S4	S5
AR token*	T0	T1	T2	T3	T4	—
pending_cnt	1	2	3	4	5	5
C2 window*	—	0	0/1	0/1/2	0/1/2/3	0/1/2/3/4
idle rank 1→5*	-----	0----	00---	001--	0001-	11111
6th outside*	—	—	—	—	—	—
C2 chosen*	—	—	—	T2	T3	T0
req token*	—	—	—	—	T2	T3
req	00000000	00000000	00000000	00000000	00000001	00001000
I	0	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-04 实际正向见证；黄色为关键采样点。

关键采样 S5 A/I/M/R=0000，N=2/2/1，ready=111，pending_cnt=5。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==5 && !issue_fire && h_rank_idle[4:0]==5'b11111 && select_valid &&
        ranks[select_e]==0
);
```

证据 build/formal/win/run/engine_0/trace13.vcd；日志 step=6，图示采样 S5。

关联属性 A_ARB_ADDR。

分支 1 个；本页已列出该场景 cover。

WIN 05 最老忙而次老空闲

流水与乱序窗口 | 合法业务

前提与动作 窗口 A..E，A 忙，B 空闲；C2 选择。

预期 选择 B；先按空闲过滤再选最老。

要拦住的错误 先找全局最老再检查空闲，造成队头阻塞。

实际 Formal 见证 WIN-05 上升沿采样值

	S0	S1	S2
AR token*	T0	T1	–
pending_cnt	1	2	2
C2 window*	–	0	0/1
idle rank 1→5*	-----	1----	01---
6th outside*	–	–	–
C2 chosen*	–	T0	T1
req token*	–	–	T0
req	00000000	00000000	00010000
I	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-05 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=0/1/1，ready=111，pending_cnt=2。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_05_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting>=2 && !issue_fire && !h_rank_idle[0] && h_rank_idle[1] && select_valid
        && ranks[select_e]==1
);
```

证据 build/formal/win/run/engine_0/trace6.vcd；日志 step=3，图示采样 S2。

关联属性 A_ARB_ADDR。

分支 1 个；本页已列出该场景 cover。

WIN 06 多个非连续候选

流水与乱序窗口 | 合法业务

前提与动作 窗口 A..E，只有 B/D/E 空闲；C2 选择。

预期 选择 B；不能跳到 D/E。

要拦住的错误 从可用候选选最新或固定 bank。

实际 Formal 见证 WIN-06 上升沿采样值

	S0	S1	S2	S3	S4	S5
AR token*	T0	T1	T2	T3	T4	–
pending_cnt	1	2	3	4	5	5
C2 window*	–	0	0/1	0/1/2	0/1/2/3	0/1/2/3/4
idle rank 1→5*	-----	0----	00---	001--	0001-	01011
6th outside*	–	–	–	–	–	–
C2 chosen*	–	–	–	T2	T3	T1
req token*	–	–	–	–	T2	T3
req	00000000	00000000	00000000	00000000	00000001	00001000
I	0	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-06 实际正向见证；黄色为关键采样点。

关键采样 S5 A/I/M/R=0000，N=2/2/1，ready=111，pending_cnt=5。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting>=5 && !issue_fire && h_rank_idle[4:0]==5'b11010 && select_valid &&
        ranks[select_e]==1
);
```

证据 build/formal/win/run/engine_0/trace14.vcd；日志 step=6，图示采样 S5。

关联属性 A_ARB_ADDR。

分支 1 个；本页已列出该场景 cover。

WIN 07 只有第五项可用

流水与乱序窗口 | 合法业务

前提与动作 A..D 忙、E 空闲；F 更年轻且也空闲；C2 选择。

预期 选择 E；窗口上界包含第 5 项。

要拦住的错误 rank<4 导致漏掉第 5 项。

实际 Formal 见证 WIN-07 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6
AR token*	T0	T1	T2	T3	T4	T5	–
pending_cnt	1	2	3	4	5	6	6
C2 window*	–	0	0/1	0/1/2	0/1/2/3	0/1/2/3/4	0/1/2/3/4
idle rank 1→5*	-----	0----	11---	000--	0011-	00000	00001
6th outside*	–	–	–	–	–	–	T5 idle
C2 chosen*	–	–	T0	–	T2	–	T4
req token*	–	–	–	T0	–	T2	–
req	00000000	00000000	00000000	00000001	00000000	00000100	00000000
I	0	0	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-07 实际正向见证；黄色为关键采样点。

关键采样 S6 A/I/M/R=0000，N=2/0/4，ready=111，pending_cnt=6。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting>=6 && !issue_fire && h_rank_idle[5:0]==6'b110000 && select_valid &&
        ranks[select_e]==4
);
```

证据 build/formal/win/run/engine_0/trace16.vcd；日志 step=7，图示采样 S6。

分支 1 个；本页已列出该场景 cover。

WIN 08 前五项全忙但第六项空闲

流水与乱序窗口 | 合法业务

前提与动作 已接收 A..F，A..E 目标忙，F 目标空闲；状态保持多拍，未发生成功发射。

预期 不下发 F；窗口不因空闲等待而向后滑动。

要拦住的错误 在全 9 项中过滤后取最老；定时滑窗。

实际 Formal 见证 WIN-08 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
AR token*	T2	T3	T4	T5	—	—	T6	—
pending_cnt	3	4	5	6	6	6	7	7
C2 window*	0/1	0/1/2	0/1/2/3	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4
idle rank 1→5*	00---	010--	1000-	00000	00000	00000	00000	00000
6th outside*	—	—	—	—	T5 idle	T5 idle	T5 idle	T5 idle
C2 chosen*	—	T1	T0	—	—	—	—	—
req token*	—	—	T1	T0	—	—	—	—
req	00000000	00000000	01000000	00100000	00000000	00000000	00000000	00000000
I	0	0	0	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-08 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0000，N=3/0/4，ready=111，pending_cnt=7。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    blocked_six && blocked_streak>=3 && req==0 && h_status==h_prev_status
);
```

证据 build/formal/win/run/engine_0/trace24.vcd；日志 step=10，图示采样 S9。

关联属性 C_BLOCKED_SIX。

分支 1 个；本页已列出该场景 cover。

WIN 09 六到九项窗口仍只取五

流水与乱序窗口 | 合法业务

前提与动作 已接收 6/7/8/9 项，合法来源分布；第 6..9 项目标空闲，前五不可用。

预期 这些项都不能越窗；占用多不扩大窗口。

要拦住的错误 窗口参数误设为 9。

实际 Formal 见证 WIN-09 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	T5	–	–
pending_cnt	1	2	3	4	5	6	6	6
C2 window*	–	0	0/1	0/1/2	0/1/2/3	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4
idle rank 1→5*	-----	1----	00---	001--	0000-	00000	00000	00000
6th outside*	–	–	–	–	–	–	T5 idle	T5 idle
C2 chosen*	–	T0	–	T2	–	–	–	–
req token*	–	–	T0	–	T2	–	–	–
req	00000000	00000000	00000001	00000000	00000010	00000000	00000000	00000000
I	0	0	0	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-09 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0000，N=3/0/3，ready=111，pending_cnt=6。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_09_P6: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==6 && blocked_six && blocked_streak>=1 && req==0
);
```

证据 build/formal/win/run/engine_0/trace19.vcd；日志 step=8，图示采样 S7。

关联属性 C_BLOCKED_SIX。

分支 4 个；其余分支完整代码见附录 B。

WIN 10 已发项不占待发窗口

流水与乱序窗口 | 合法业务

前提与动作 若干最老项已 issued，仍保留 entry；检查剩余未 issued 命令的排名。

预期 取最老五条未成功发射命令，不是最老五条有效项。

要拦住的错误 issued 长时间占据窗口造成停顿。

实际 Formal 见证 WIN-10 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6
AR token*	T0	T1	T2	T3	T4	T5	–
pending_cnt	1	2	3	4	5	5	5
C2 window*	–	0	0/1	0/1/2	0/1/2/3	0/1/3/4	0/1/3/4/5
idle rank 1→5*	-----	0----	11---	000--	0011-	0000-	00001
6th outside*	–	–	–	–	–	–	–
C2 chosen*	–	–	T0	–	T2	–	T5
req token*	–	–	–	T0	–	T2	–
req	00000000	00000000	00000000	00000001	00000000	00000100	00000000
I	0	0	0	0	0	1	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-10 实际正向见证；黄色为关键采样点。

关键采样 S6 A/I/M/R=0000，N=2/0/4，ready=111，pending_cnt=5。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_10_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    (m_live&m_sent)!=0 && select_valid && m_age[select_e]>=5 && ranks[select_e]<5
);
```

证据 build/formal/win/run/engine_0/trace15.vcd；日志 step=7，图示采样 S6。

关联属性 M_STATE。

分支 1 个；本页已列出该场景 cover。

WIN 11 窗口队头成功后补入

流水与乱序窗口 | 合法业务

前提与动作 A..E 在窗内，F 在窗外；当前 A req 成功；同拍进行下一次预选。

预期 新窗 B/C/D/E/F，F 可当拍参与、下一拍 req。

要拦住的错误 补窗延迟一拍；先选后扣 success。

实际 Formal 见证 WIN-11 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	T5	T6	–
pending_cnt	1	2	3	4	5	6	6	6
C2 window*	–	0	0/1	0/1/2	0/1/2/3	0/1/2/3/4	1/2/3/4/5	1/2/3/4/5
idle rank 1→5*	-----	0----	11---	110--	0010-	11000	00001	00011
6th outside*	–	–	–	–	–	–	–	T6 busy
C2 chosen*	–	–	T0	T0	T2	T0	T5	T4
req token*	–	–	–	T0	T0	T2	T0	T5
req	00000000	00000000	00000000	00000001	00000001	00000001	00000001	00000100
I	0	0	0	0	0	0	1	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-11 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0000，N=3/0/4，ready=111，pending_cnt=6。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_11_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    refill_selected && !nonhead_refill_selected && req==last_refill_req &&
        arb_addr==last_refill_addr && req!=0
);
```

证据 build/formal/win/run/engine_0/trace21.vcd；日志 step=8，图示采样 S7。

关联属性 C_REFILL_NEXT_CYCLE。

分支 1 个；本页已列出该场景 cover。

WIN 12 窗口中间项成功后补入

流水与乱序窗口 | 合法业务

前提与动作 窗口 A/B/C/D/E，F 在窗外，C 成功；当拍更新候选。

预期 新窗 A/B/D/E/F；其余相对年龄不变，F 可参与。

要拦住的错误 用整体头指针滑一格丢 A/B。

实际 Formal 见证 WIN-12 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	T5	–	–
pending_cnt	1	2	3	4	5	6	5	5
C2 window*	–	0	0/1	0/1/2	0/1/2/3	0/1/2/3/4	0/1/2/3/5	0/1/2/3/5
idle rank 1→5*	-----	1----	00---	001--	0000-	00001	00001	00000
6th outside*	–	–	–	–	–	–	–	–
C2 chosen*	–	T0	–	T2	–	T4	T5	–
req token*	–	–	T0	–	T2	–	T4	T5
req	00000000	00000000	00000001	00000000	00000010	00000000	00000100	00000001
I	0	0	0	0	0	0	1	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-12 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0000，N=3/0/3，ready=111，pending_cnt=5。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_12_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    nonhead_refill_selected && req==last_refill_req && arb_addr==last_refill_addr &&
        req!=0
);
```

证据 build/formal/win/run/engine_0/trace20.vcd；日志 step=8，图示采样 S7。

关联属性 C_NONHEAD_REFILL。

分支 1 个；本页已列出该场景 cover。

WIN 13 仲裁失败窗口不后移

流水与乱序窗口 | 合法业务

前提与动作 当前窗内命令 req，双 grant 未同位为 1；当拍预选及下一拍请求。

预期 失败项仍在原年龄位置，第六项不因失败进入。

要拦住的错误 仅看到 req 就出窗。

实际 Formal 见证 WIN-13 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6
AR token*	T0	T1	T2	T3	T4	T5	–
pending_cnt	1	2	3	4	5	6	6
C2 window*	–	0	0/1	0/1/2	0/1/2/3	0/1/2/3/4	0/1/2/3/4
idle rank 1→5*	-----	0----	11---	001--	0001-	01100	00000
6th outside*	–	–	–	–	–	–	T5 idle
C2 chosen*	–	–	T0	T2	T3	T1	–
req token*	–	–	–	T0	T2	T3	T1
req	00000000	00000000	00000000	00001000	00000010	00000001	00010000
I	0	0	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-13 实际正向见证；黄色为关键采样点。

关键采样 S6 A/I/M/R=0000，N=2/0/4，ready=111，pending_cnt=6。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_13_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting>=6 && req!=0 && !issue_fire && !h_rank_idle[4:0] && h_rank_idle[5] &&
        !select_valid
);
```

证据 build/formal/win/run/engine_0/trace17.vcd；日志 step=7，图示采样 S6。

关联属性 C_RETRY_NEXT_CYCLE。

分支 1 个；本页已列出该场景 cover。

WIN 14 单命令失败后连续重试

流水与乱序窗口 | 合法业务

前提与动作 只有 A 待发，C3 失败且目标在 C3 仍空闲；C3 同拍重新选中 A。

预期 C4 仍有 A 的 req；不应固定插空拍。

要拦住的错误 失败项被临时排除一次选择。

实际 Formal 见证 WIN-14 上升沿采样值

	S0	S1	S2	S3
AR token*	T0	–	–	–
pending_cnt	1	1	1	1
C2 window*	–	0	0	0
idle rank 1→5*	-----	1----	1----	1----
6th outside*	–	–	–	–
C2 chosen*	–	T0	T0	T0
req token*	–	–	T0	T0
req	00000000	00000000	00000010	00000010
I	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-14 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=0000，N=0/0/1，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_14_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==1 && retry_selected && req==last_retry_req && arb_addr==last_retry_addr
        && req!=0
);
```

证据 build/formal/win/run/engine_0/trace8.vcd；日志 step=4，图示采样 S3。

关联属性 C_RETRY_NEXT_CYCLE。

分支 1 个；本页已列出该场景 cover。

WIN 15 失败后目标变忙

流水与乱序窗口 | 合法业务

前提与动作 A 当前失败；C3 status 指示 A 忙；当拍没有其他候选。

预期 C4 无 req；A 保持待发直到以后空闲。

要拦住的错误 失败必定重发而不再检查预选状态。

实际 Formal 见证 WIN-15 上升沿采样值

	S0	S1	S2	S3
AR token*	T0	–	–	–
pending_cnt	1	1	1	1
C2 window*	–	0	0	0
idle rank 1→5*	-----	1----	0----	1----
6th outside*	–	–	–	–
C2 chosen*	–	T0	–	T0
req token*	–	–	T0	–
req	00000000	00000000	00000010	00000000
I	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-15 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=0000，N=0/0/1，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_15_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_failed_none && req==0 && waiting>0
);
```

证据 build/formal/win/run/engine_0/trace9.vcd；日志 step=4，图示采样 S3。

分支 1 个；本页已列出该场景 cover。

WIN 16 失败后改选其他命令

流水与乱序窗口 | 合法业务

前提与动作 A 当前失败且忙，B 在窗内且空闲；C3 预选 B。

预期 C4 req=B，A 年龄仍旧；不锁死在 A。

要拦住的错误 将重试优先级硬锁为上一次项。

实际 Formal 见证 WIN-16 上升沿采样值

	S0	S1	S2	S3
AR token*	T0	T1	–	–
pending_cnt	1	2	2	2
C2 window*	–	0	0/1	0/1
idle rank 1→5*	-----	1----	01---	11---
6th outside*	–	–	–	–
C2 chosen*	–	T0	T1	T0
req token*	–	–	T0	T1
req	00000000	00000000	00000010	00000001
I	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-16 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=0000，N=0/0/2，ready=111，pending_cnt=2。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_16_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_failed_other && req==h_selected_req && arb_addr==h_selected_addr && req!=0
);
```

证据 build/formal/win/run/engine_0/trace10.vcd；日志 step=4，图示采样 S3。

关联属性 C_FAILURE_SELECT_OTHER。

分支 1 个；本页已列出该场景 cover。

WIN 17 成功项不能再次预选

流水与乱序窗口 | 合法业务

前提与动作 当前 req=A 成功，A 仍有效且可能状态误解；同拍构造多个候选。

预期 排除 A；仅其他未成功项参与下一拍。

要拦住的错误 issued 标记次拍才更新而重复发射。

实际 Formal 见证 WIN-17 上升沿采样值

	S0	S1	S2
AR token*	T0	T1	–
pending_cnt	1	2	1
C2 window*	–	0	1
idle rank 1→5*	-----	1----	1----
6th outside*	–	–	–
C2 chosen*	–	T0	T1
req token*	–	–	T0
req	00000000	00000000	00000001
I	0	0	1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-17 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0100，N=0/1/1，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_17_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && select_valid && select_e!=m_attempt_e
);
```

证据 build/formal/win/run/engine_0/trace4.vcd；日志 step=3，图示采样 S2。

关联属性 M_ATTEMPT。

分支 1 个；本页已列出该场景 cover。

WIN 18 新 AR 不加入当前预选

流水与乱序窗口 | 合法业务

前提与动作 本拍 A 接收且另有已存命令可选；对新旧命令设置空闲状态。

预期 本拍 C2 仅看已寄存命令；新 AR 最早下拍判断、再下拍 req。

要拦住的错误 接收旁路跳过 C2。

实际 Formal 见证 WIN-18 上升沿采样值

	S0	S1
AR token*	T0	T1
pending_cnt	1	2
C2 window*	–	0
idle rank 1→5*	-----	1----
6th outside*	–	–
C2 chosen*	–	T0
req token*	–	–
req	00000000	00000000
I	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-18 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=1000，N=1/0/0，ready=111，pending_cnt=2。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_18_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && select_valid && alloc_e!=select_e && h_rank_valid!=0
);
```

证据 build/formal/win/run/engine_0/trace2.vcd；日志 step=2，图示采样 S1。

关联属性 A_ARB_ADDR。

分支 1 个；本页已列出该场景 cover。

WIN 19 R 回收不使待发窗前移

流水与乱序窗口 | 合法业务

前提与动作 旧已发项本拍 R，待发五项均忙；第六待发项空闲。

预期 R 只释放容量；不把第六待发项自动拉入窗口。

要拦住的错误 把 R 出队误当成待发队列出队。

实际 Formal 见证 WIN-19 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	T5	T6	–
pending_cnt	1	2	2	3	4	5	6	6
C2 window*	–	0	1	1/2	1/2/3	1/2/3/4	1/2/3/4/5	1/2/3/4/5
idle rank 1→5*	-----	1----	0----	00---	001--	0000-	00000	00000
6th outside*	–	–	–	–	–	–	–	T6 idle
C2 chosen*	–	T0	–	–	T3	–	–	–
req token*	–	–	T0	–	–	T3	–	–
req	00000000	00000000	00000001	00000000	00000000	01000000	00000000	00000000
I	0	0	1	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 WIN-19 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=3/3/1，ready=111，pending_cnt=6。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_WIN_19_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    r_fire && blocked_six && !issue_fire && req==0
);
```

证据 build/formal/win/run/engine_0/trace22.vcd；日志 step=8，图示采样 S7。

关联属性 C_BLOCKED_SIX。

分支 1 个；本页已列出该场景 cover。

ARB 01 八个 Bank 编码边界

Bank 与仲裁判断 | 合法业务

前提与动作 合法命令在窗口中；遍历 araddr[5:3]=0..7。

预期 req[bank] 唯一置位；没有 Bank8；地址最低三位不影响 Bank。

要拦住的错误 切片错一位、最高 Bank 丢失。

实际 Formal 见证 ARB-01 上升沿采样值

	S0	S1	S2
C2 chosen*	–	T0	–
req	00000000	00000000	00000001
arb_addr	–	–	FFE
status B0[1:0]	00	00	01
ar_rd_gnt	11111111	00000000	00000011
rw_rd_gnt	11111111	00000000	00000011
I	0	0	1
pending_cnt	1	1	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-01 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0100，N=1/0/0，ready=111，pending_cnt=0。

[REF_MODEL] req 有效时检查完整下游地址 araddr[17:6]；无效时不检查负载。

```
A_ARB_ADDR: assert property (
    @(posedge clk) disable iff (!rst_n)
    (req != 8'b0)
    |-> (arb_addr == expected_addr)
);
```

```
C_CC_ARB_01_B0: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<0) && arb_addr==m_addr[m_attempt_e][17:6]
);
```

证据 build/formal/arb/run/engine_0/trace22.vcd；日志 step=3，图示采样 S2。

关联属性 A_REQ, A_REQ_ONEHOT。

分支 8 个；其余分支完整代码见附录 B。

ARB 02 奇偶半区独立

Bank 与仲裁判断 | 合法业务

前提与动作 固定 bank，status=01 或 10；命令 araddr[6] 分别为 0/1。

预期 仅对应 half=0 的 bit0、half=1 的 bit1 为 0 时可预选。

要拦住的错误 二维下标转置；奇偶含义反了。

实际 Formal 见证 ARB-02 上升沿采样值

	S0	S1
C2 chosen*	–	T0
req	00000000	00000000
arb_addr	–	–
status B0[1:0]	00	10
ar_rd_gnt	11111111	00000000
rw_rd_gnt	11111111	00000000
I	0	0
pending_cnt	1	1

* 为低层事件推导；— 表示本图不关心；蓄色列为关注采样点

图 ARB-02 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] req 有效时检查完整下游地址 araddr[17:6]；无效时不检查负载。

```
A_ARB_ADDR: assert property (
    @(posedge clk) disable iff (!rst_n)
    (req != 8'b0)
    |-> (arb_addr == expected_addr)
);
```

```
C_CC_ARB_02_B0H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==0 && m_addr[select_e][6]==0 &&
        rd_mem_status[0]==2
);
```

证据 build/formal/arb/run/engine_0/trace16.vcd；日志 step=2，图示采样 S1。

关联属性 A_REQ。

分支 16 个；其余分支完整代码见附录 B。

ARB 03 Bank 两半均空闲/均忙

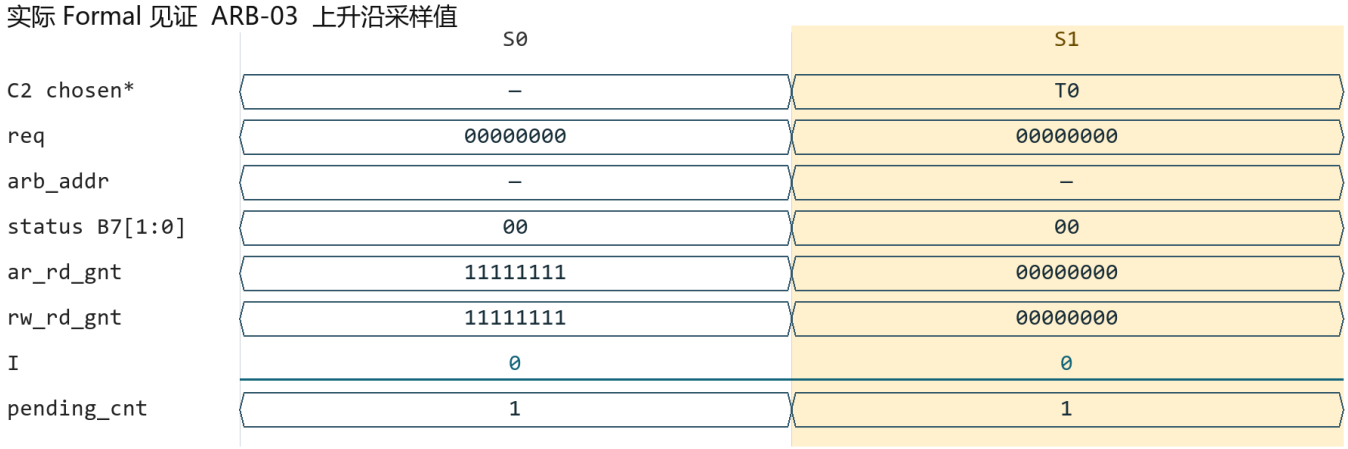
Bank 与仲裁判断 | 合法业务

前提与动作 status[bank]=00 或 11；窗内请求同 bank 两种 half。

预期 00 两半都可候选；11 两半都排除，仍按年龄择一。

要拦住的错误 将 2'b00 解读为占用。

实际 Formal 见证 ARB-03 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-03 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_ARB_03_IDLE: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting>0 && rd_mem_status[m_addr[m_attempt_e][5:3]]==0 && select_valid
);
```

证据 build/formal/arb/run/engine_0/trace4.vcd；日志 step=2，图示采样 S1。

分支 2 个；其余分支完整代码见附录 B。

ARB 04 发送地址保留 half

Bank 与仲裁判断 | 合法业务

前提与动作 araddr[17:6] 边界与相邻变化；命令进入 req 拍。

预期 arb_addr=araddr[17:6]，12 位；arb_addr[0]=half。

要拦住的错误 沿用旧 11 位 [17:7] 地址。

实际 Formal 见证 ARB-04 上升沿采样值

	S0	S1	S2
C2 chosen*	–	T0	–
req	00000000	00000000	00000001
arb_addr	–	–	000
status B0[1:0]	00	00	11
ar_rd_gnt	11111111	00000000	11111110
rw_rd_gnt	11111111	00000000	00000010
I	0	0	0
pending_cnt	1	1	1

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-04 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] req 有效时检查完整下游地址 araddr[17:6]；无效时不检查负载。

```
A_ARB_ADDR: assert property (
    @(posedge clk) disable iff (!rst_n)
    (req != 8'b0)
    |-> (arb_addr == expected_addr)
);
```

```
C_CC_ARB_04_ADDR0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && arb_addr==0 && arb_addr==m_addr[m_attempt_e][17:6]
);
```

证据 build/formal/arb/run/engine_0/trace30.vcd；日志 step=3，图示采样 S2。

分支 4 个；其余分支完整代码见附录 B。

ARB 05 四种双 grant 组合

Bank 与仲裁判断 | 合法业务

前提与动作 当前 req 的同一 bit=i；(ar_gnt[i],rw_gnt[i])=00/01/10/11。

预期 仅 11 成功；其他组合均失败且待发保持。

要拦住的错误 只看其中一种 grant 或使用 OR。

实际 Formal 见证 ARB-05 上升沿采样值

	S0	S1	S2
C2 chosen*	–	T0	–
req	00000000	00000000	00000001
arb_addr	–	–	FFF
status B0[1:0]	00	00	11
ar_rd_gnt	11111111	00000000	11111110
rw_rd_gnt	11111111	00000000	00000010
I	0	0	0
pending_cnt	1	1	1

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-05 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

```
C_CC_ARB_05_G00: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && ((req&ar_rd_gnt)!=0)==0 && ((req&rw_rd_gnt)!=0)==0 && issue_fire==0
);
```

证据 build/formal/arb/run/engine_0/trace28.vcd；日志 step=3，图示采样 S2。

关联属性 A_PENDING, A_REQ, M_SUCCESS。

分支 4 个；其余分支完整代码见附录 B。

ARB 06 grant 出现在非请求 Bank

Bank 与仲裁判断 | 合法业务

前提与动作 req[i]=1，其他 Bank grant=1；请求 bit 双 grant 不同时为 1。

预期 不成功；不能把无关 bit 合并为成功。

要拦住的错误 对两个 grant 分别做 reduction 再 AND。

实际 Formal 见证 ARB-06 上升沿采样值

	S0	S1	S2
C2 chosen*	–	T0	–
req	00000000	00000000	00000001
arb_addr	–	–	FFE
status B0[1:0]	00	00	01
ar_rd_gnt	11111111	00000000	00000010
rw_rd_gnt	11111111	00000000	00000011
I	0	0	0
pending_cnt	1	1	1

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-06 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

```
C_CC_ARB_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && (ar_rd_gnt&rw_rd_gnt&~req)!=0 && !issue_fire
);
```

证据 build/formal/arb/run/engine_0/trace21.vcd；日志 step=3，图示采样 S2。

关联属性 A_PENDING, A_REQ。

分支 1 个；本页已列出该场景 cover。

ARB 07 两个 grant 的有效 bit 不同

Bank 与仲裁判断 | 合法业务

前提与动作 req[i]=1，ar_gnt[i]=1；rw_gnt[j]=1，j≠i；无 requested bit 的双 1。

预期 失败；pending 不因该 grant 减 1。

要拦住的错误 先归约再相与造成假成功。

实际 Formal 见证 ARB-07 上升沿采样值

	S0	S1	S2
C2 chosen*	–	T0	–
req	00000000	00000000	00000001
arb_addr	–	–	FFF
status B0[1:0]	00	00	11
ar_rd_gnt	11111111	00000000	11111111
rw_rd_gnt	11111111	00000000	00000010
I	0	0	0
pending_cnt	1	1	1

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-07 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

```
C_CC_ARB_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && (req&ar_rd_gnt)!=0 && (req&rw_rd_gnt)==0 && rw_rd_gnt!=0 &&
        !issue_fire
);
```

证据 build/formal/arb/run/engine_0/trace27.vcd；日志 step=3，图示采样 S2。

关联属性 A_PENDING, A_REQ。

分支 1 个；本页已列出该场景 cover。

ARB 08 req=0 时有 grant

Bank 与仲裁判断 | 合法业务

前提与动作 当前无下游请求；grant 任意组合，包括多 bit=1。

预期 本模块 I=0，不启动返回日历；无效 arb_addr 不检查。

要拦住的错误 没有与 req 相与导致凭空成功。

实际 Formal 见证 ARB-08 上升沿采样值

	S0
C2 chosen*	–
req	00000000
arb_addr	–
status B0[1:0]	00
ar_rd_gnt	11111111
rw_rd_gnt	11111111
I	0
pending_cnt	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-08 实际正向见证；黄色为关键采样点。

关键采样 S0 A/I/M/R=0000，N=0/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

```
C_CC_ARB_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    req==0 && ar_rd_gnt==8'hff && rw_rd_gnt==8'hff && !issue_fire
);
```

证据 build/formal/arb/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 A_PENDING, M_CALENDAR。

分支 1 个；本页已列出该场景 cover。

ARB 09 多 bit grant 包含请求 bit

Bank 与仲裁判断 | 合法业务

前提与动作 req onehot，请求 bit 两种 grant 都为 1，其他位可为 1；遵守请求 half 必须 busy。

预期 仍只成功一条；无额外多发/多返回。

要拦住的错误 把 grant 位数计成多条命令。

实际 Formal 见证 ARB-09 上升沿采样值

	S0	S1	S2
C2 chosen*	–	T0	–
req	00000000	00000000	00000001
arb_addr	–	–	FFE
status B0[1:0]	00	00	01
ar_rd_gnt	11111111	00000000	11111111
rw_rd_gnt	11111111	00000000	11111111
I	0	0	1
pending_cnt	1	1	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-09 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0100，N=1/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

```
C_CC_ARB_09_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && ar_rd_gnt==8'hff && rw_rd_gnt==8'hff
);
```

证据 build/formal/arb/run/engine_0/trace23.vcd；日志 step=3，图示采样 S2。

关联属性 A_PENDING, A_REQ_ONEHOT, M_CALENDAR。

分支 1 个；本页已列出该场景 cover。

ARB 10 预选空闲，发送拍已忙

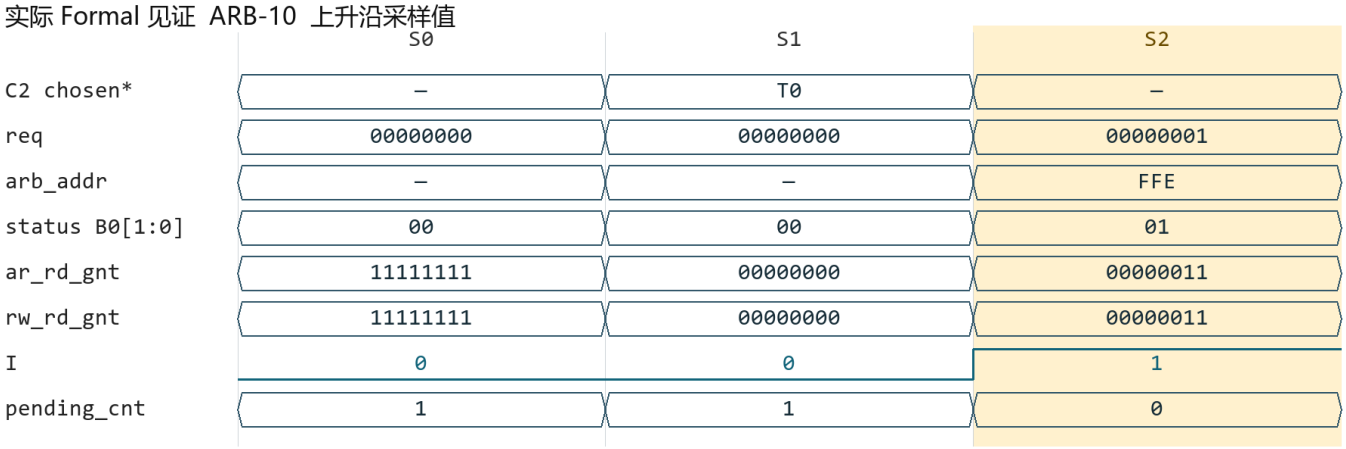
Bank 与仲裁判断 | 合法业务

前提与动作 C2 status 目标 half=0，命令被选中；C3 req 有效且目标 status=1。

预期 req 保持；是否成功由双 grant 决定；不以当前 busy 撤销 req。

要拦住的错误 C3 重新用 status 门控 req。

实际 Formal 见证 ARB-10 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-10 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0100，N=1/0/0，ready=111，pending_cnt=0。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_ARB_10_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_selected_prev && req==h_selected_req && arb_addr==h_selected_addr &&
        rd_mem_status[m_addr[m_attempt_e][5:3]][arb_addr[0]] && issue_fire
);
```

证据 build/formal/arb/run/engine_0/trace22.vcd；日志 step=3，图示采样 S2。

关联属性 E06_SUCCESS_BUSY。

分支 1 个；本页已列出该场景 cover。

ARB 11 成功当拍目标 busy

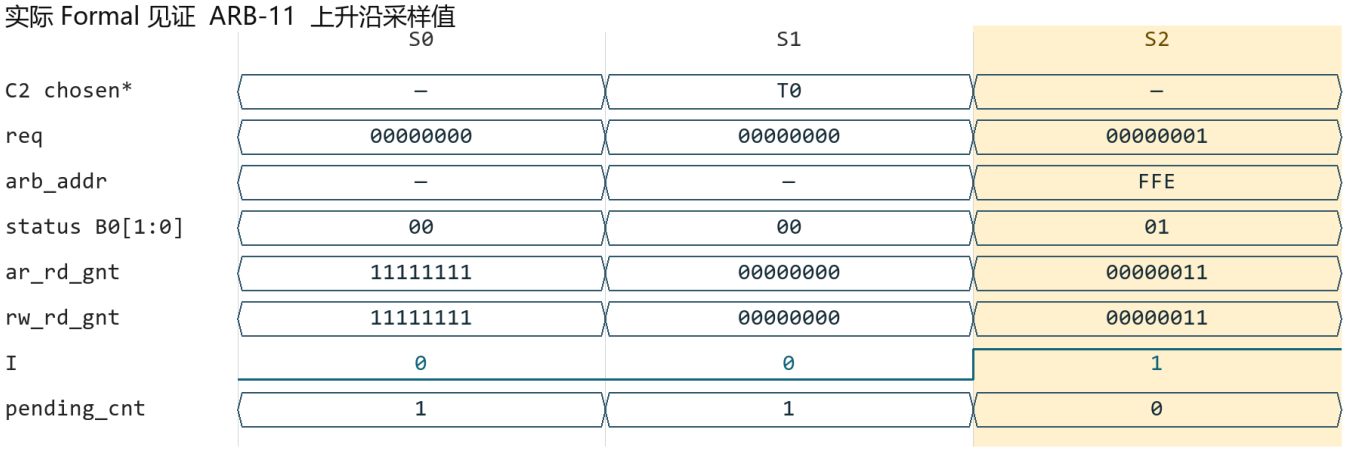
Bank 与仲裁判断 | 合法业务

前提与动作 req[i] 与两个 grant[i] 均为 1；检查 status[i][arb_addr[0]]。

预期 同拍必须为 1；这是一条环境关系，不是 DUT 要拉的输出。

要拦住的错误 将 same-cycle 写成下一拍 busy。

实际 Formal 见证 ARB-11 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-11 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0100，N=1/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 本拍该 Bank 双 grant 成功时，被访问的奇偶半区本拍必须报告占用。

```
E06_SUCCESS_BUSY: assume property (
    @(posedge clk) disable iff (!rst_n)
    (req[bank_env] && ar_rd_gnt[bank_env] && rw_rd_gnt[bank_env])
    |-> (requested_half_busy)
);
```

```
C_CC_ARB_11_B0H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[0] && ar_rd_gnt[0] && rw_rd_gnt[0] && arb_addr[0]==0 && rd_mem_status[0][0]
);
```

证据 build/formal/arb/run/engine_0/trace22.vcd；日志 step=3，图示采样 S2。

关联属性 C_SUCCESS_BUSY_TARGET。

分支 16 个；其余分支完整代码见附录 B。

ARB 12 成功但目标仍空闲

Bank 与仲裁判断 | 环境违约

前提与动作 req 位双 grant=1，同时目标 status=0；独立负向注入。

预期 违反 E06；不能要求正常模型接受该环境并当合法反例。

要拦住的错误 将违反 assume 的波形算作 RTL 错误。

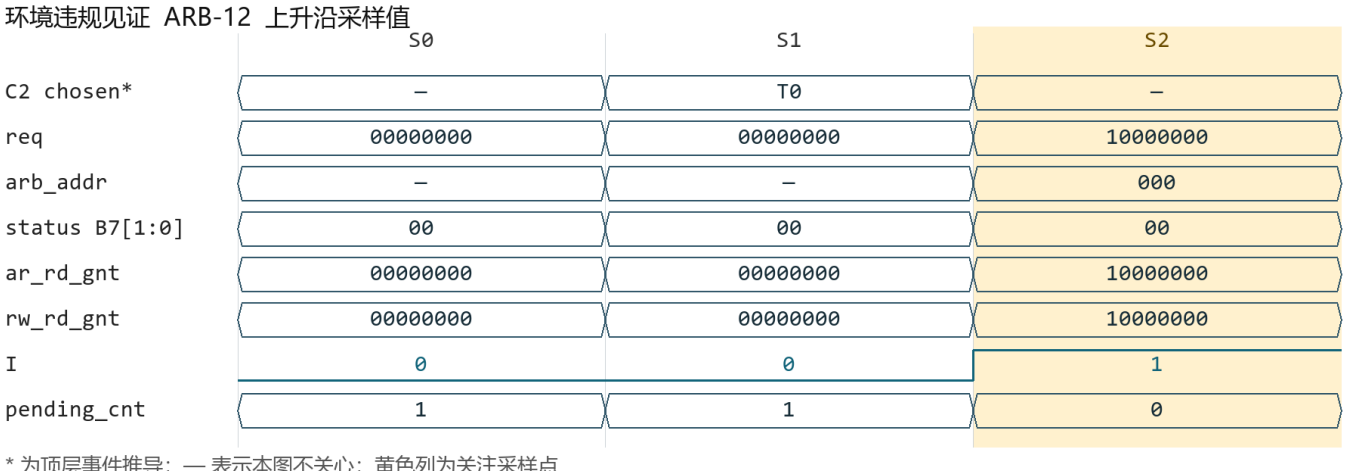


图 ARB-12 实际负向输入轨迹，监测断言按预期失败。

关键采样 S2 A/I/M/R=0100，N=1/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] req 同位双 grant 成功但对应半区仍空闲；本项改为监测断言，预期检出。

```
V_ARB_12: assert property (
    @(posedge clk) disable iff (!rst_n)
    !bad_busy
);
```

```
N_ARB_12: cover property (
    @(posedge clk) disable iff (!rst_n)
    bad_busy
);
```

证据 build/negative/arb_12_cover/run/engine_0/trace0.vcd；日志 step=3，图示采样 S2。

关联属性 E06_SUCCESS_BUSY。

边界 关闭 E06 后，用 N_ARB_12 cover / V_ARB_12 assert 检查环境违规见证。

ARB 13 其他 Bank 状态变化

Bank 与仲裁判断 | 合法业务

前提与动作 固定窗口命令的目标 half 条件；无关 Bank/half 状态任意改变。

预期 不影响该项是否候选；候选排序只受真正相关项影响。

要拦住的错误 bank 下标拼接或位序错误。

实际 Formal 见证 ARB-13 上升沿采样值

	S0	S1	S2
C2 chosen*	–	T0	–
req	00000000	00000000	00000001
arb_addr	–	–	FFE
status B0[1:0]	00	00	01
ar_rd_gnt	11111111	00000000	00000000
rw_rd_gnt	11111111	00000000	00000001
I	0	0	0
pending_cnt	1	1	1

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-13 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_ARB_13_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_selected_prev && req==h_selected_req && arb_addr==h_selected_addr &&
        h_status!=h_prev_status && !issue_fire
);
```

证据 build/formal/arb/run/engine_0/trace20.vcd；日志 step=3，图示采样 S2。

分支 1 个；本页已列出该场景 cover。

覆盖差异 当前 cover 只要求状态向量发生变化，未限定“仅其他 Bank 变化”。图中目标 Bank 也发生变化，不能据此认定其他 Bank 单独干扰场景已命中，见附录 E。

ARB 14 同 Bank 两 half 连续请求

Bank 与仲裁判断 | 合法业务

前提与动作 两条命令访问同一 Bank 的不同 half，且按相应拍的空闲状态具备发送条件；若 B 的 half 仍空闲则同拍预选 B。

预期 B 可下一拍请求；不把一个 half busy 当整个 bank 永久忙。

要拦住的错误 忽略奇偶独立性或额外施加两拍间隔。

实际 Formal 见证 ARB-14 上升沿采样值

	S0	S1	S2	S3
C2 chosen*	–	T0	T1	–
req	00000000	00000000	00000010	00000010
arb_addr	–	–	FC1	FC0
status B1[1:0]	00	00	10	01
ar_rd_gnt	11111111	00000000	11111111	00000010
rw_rd_gnt	11111111	00000000	11111111	00000010
I	0	0	1	1
pending_cnt	1	2	1	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ARB-14 实际正向见证；黄色为关键采样点。

关键采样 S3 A/I/M/R=0100，N=2/0/0，ready=111，pending_cnt=0。

[REF_MODEL] req 有效时检查完整下游地址 araddr[17:6]；无效时不检查负载。

```
A_ARB_ADDR: assert property (
    @(posedge clk) disable iff (!rst_n)
    (req != 8'b0)
    |-> (arb_addr == expected_addr)
);
```

```
C_CC_ARB_14_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_i && issue_fire && req==h_prev_req && arb_addr[0]!=h_prev_arb[0]
);
```

证据 build/formal/arb/run/engine_0/trace45.vcd；日志 step=4，图示采样 S3。

关联属性 A_REQ。

分支 1 个；本页已列出该场景 cover。

CNT 01 四种 A/I 组合

pending 与占用计数 | 合法业务

前提与动作 拍初待发数 P，合法 A/I 条件；分别 A,I=00/10/01/11。

预期 本拍 pending_cnt=P、P+1、P-1、P；以事件布尔值计算。

要拦住的错误 pending 输出滞后一拍。

实际 Formal 见证 CNT-01 上升沿采样值

	S0	S1
P before*	0	1
A	1	0
I	0	0
M	0	0
R	0	0
pending_cnt	1	1
N0/N1/N2*	0/0/0	1/0/0
AR token*	T0	–
req token*	–	–

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-01 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

```
C_CC_CNT_01_A0I0: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire==0 && issue_fire==0 && waiting>0 && pending_cnt==waiting+0-0
);
```

证据 build/formal/cnt/run/engine_0/trace1.vcd；日志 step=2，图示采样 S1。

关联属性 A_PENDING。

分支 4 个；其余分支完整代码见附录 B。

CNT 02 五条只成功一条

pending 与占用计数 | 合法业务

前提与动作 P=5，本拍 I=1、A=0；查看本拍组合输出。

预期 pending_cnt=4；不能等下一拍显示 4。

要拦住的错误 使用注册输出而不含本拍 I。

实际 Formal 见证 CNT-02 上升沿采样值

	S0	S1	S2	S3	S4	S5
P before*	0	1	2	3	4	5
A	1	1	1	1	1	0
I	0	0	0	0	0	1
M	0	0	0	0	0	0
R	0	0	0	0	0	0
pending_cnt	1	2	3	4	5	4
N0/N1/N2*	0/0/0	1/0/0	1/0/1	1/0/2	1/0/3	2/0/3
AR token*	T0	T1	T2	T3	T4	–
req token*	–	–	–	–	T2	T3

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-02 实际正向见证；黄色为关键采样点。

关键采样 S5 A/I/M/R=0100，N=2/0/3，ready=111，pending_cnt=4。

[REF_MODEL] 待发数含本拍新 AR，扣除本拍双 grant 成功；仲裁失败不扣除。

```
A_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (waiting+(ar_fire?1:0)-(issue_fire?1:0))
);
```

```
C_CC_CNT_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==5 && issue_fire && !ar_fire && pending_cnt==4
);
```

证据 build/formal/cnt/run/engine_0/trace6.vcd；日志 step=6，图示采样 S5。

关联属性 C_PENDING5_DEC。

分支 1 个；本页已列出该场景 cover。

CNT 03 五条同拍进一出一

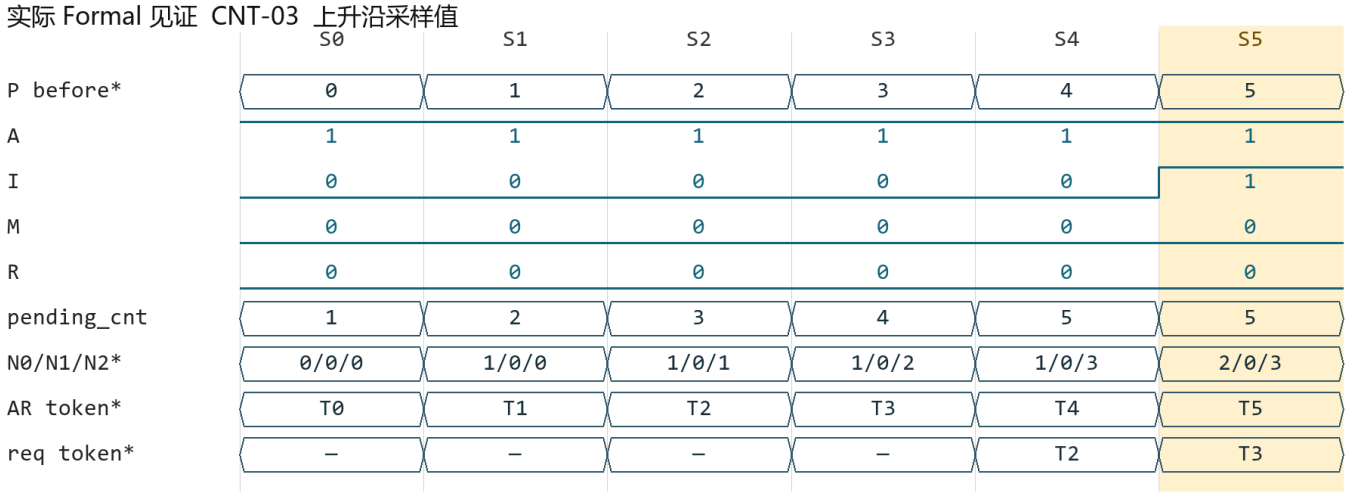
pending 与占用计数 | 合法业务

前提与动作 P=5，A=1、I=1；查看本拍组合输出。

预期 pending_cnt=5；新 AR 当拍计入。

要拦住的错误 只减 I 或把新 AR 延后。

实际 Formal 见证 CNT-03 上升沿采样值



* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-03 实际正向见证；黄色为关键采样点。

关键采样 S5 A/I/M/R=1100，N=2/0/3，ready=111，pending_cnt=5。

[REF_MODEL] 待发数含本拍新 AR，扣除本拍双 grant 成功；仲裁失败不扣除。

```
A_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (waiting+(ar_fire?1:0)-(issue_fire?1:0))
);
```

```
C_CC_CNT_03_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==5 && issue_fire && ar_fire && pending_cnt==5
);
```

证据 build/formal/cnt/run/engine_0/trace7.vcd；日志 step=6，图示采样 S5。

关联属性 C_PENDING5_BOTH。

分支 1 个；本页已列出该场景 cover。

CNT 04 五条只进一条

pending 与占用计数 | 合法业务

前提与动作 P=5，A=1、I=0；查看本拍组合输出。

预期 pending_cnt=6；窗口仍最多 5 条。

要拦住的错误 把 pending_cnt 截成窗口长度。

实际 Formal 见证 CNT-04 上升沿采样值

	S0	S1	S2	S3	S4	S5
P before*	0	1	2	3	4	5
A	1	1	1	1	1	1
I	0	0	0	0	0	0
M	0	0	0	0	0	0
R	0	0	0	0	0	0
pending_cnt	1	2	3	4	5	6
N0/N1/N2*	0/0/0	1/0/0	1/0/1	1/0/2	1/0/3	2/0/3
AR token*	T0	T1	T2	T3	T4	T5
req token*	–	–	–	–	T2	T3

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-04 实际正向见证；黄色为关键采样点。

关键采样 S5 A/I/M/R=1000，N=2/0/3，ready=111，pending_cnt=6。

[REF_MODEL] 待发数含本拍新 AR，扣除本拍双 grant 成功；仲裁失败不扣除。

```
A_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (waiting+(ar_fire?1:0)-(issue_fire?1:0))
);
```

```
C_CC_CNT_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==5 && !issue_fire && ar_fire && pending_cnt==6
);
```

证据 build/formal/cnt/run/engine_0/trace8.vcd；日志 step=6，图示采样 S5。

关联属性 C_PENDING5_ADD。

分支 1 个；本页已列出该场景 cover。

CNT 05 满九项但待发为零

pending 与占用计数 | 合法业务

前提与动作 9 项均已发射，尚有等待 MEM/R 的项；没有新的 AR。

预期 pending_cnt=0；不代表 CMDBUF 空或 ready=111。

要拦住的错误 把 pending 当 entry 总占用。

实际 Formal 见证 CNT-05 上升沿采样值

	S4	S5	S6	S7	S8	S9	S10	S11
P before*	2	2	2	2	2	2	1	0
A	1	1	1	1	1	0	0	0
I	1	1	1	1	1	1	1	0
M	0	0	1	1	1	1	1	1
R	0	0	0	0	0	0	0	0
pending_cnt	2	2	2	2	2	1	0	0
N0/N1/N2*	1/0/3	2/0/3	3/0/3	4/0/3	5/0/3	5/1/3	5/1/3	5/1/3
AR token*	T4	T5	T6	T7	T8	–	–	–
req token*	T2	T3	T4	T5	T6	T7	T8	–

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-05 实际正向见证；黄色为关键采样点。

关键采样 S11 A/I/M/R=0010，N=5/1/3，ready=000，pending_cnt=0。

[REF_MODEL] 待发数含本拍新 AR，扣除本拍双 grant 成功；仲裁失败不扣除。

```
A_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (waiting+(ar_fire?1:0)-(issue_fire?1:0))
);
```

```
C_CC_CNT_05_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && waiting==0 && pending_cnt==0 && !ar_fire && !r_fire && arready==0
);
```

证据 build/formal/cnt/run/engine_0/trace13.vcd；日志 step=12，图示采样 S11。

关联属性 A_READY。

分支 1 个；本页已列出该场景 cover。

CNT 06 pending 上界与下界

pending 与占用计数 | 合法业务

前提与动作 P=0 或 P=9；合法事件组合：空时无 I；P=9 时无直接多收。

预期 pending 始终 0..9；无 4 位回绕；P=0 接收可为 1。

要拦住的错误 无符号下溢变 15。

实际 Formal 见证 CNT-06 上升沿采样值

	S0
P before*	0
A	1
I	0
M	0
R	0
pending_cnt	1
N0/N1/N2*	0/0/0
AR token*	T0
req token*	–

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-06 实际正向见证；黄色为关键采样点。

关键采样 S0 A/I/M/R=1000，N=0/0/0，ready=111，pending_cnt=1。

[REF_MODEL] 待发数含本拍新 AR，扣除本拍双 grant 成功；仲裁失败不扣除。

```
A_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (waiting+(ar_fire?1:0)-(issue_fire?1:0))
);
```

```
C_CC_CNT_06_EMPTY_A: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==0 && ar_fire && !issue_fire && pending_cnt==1
);
```

证据 build/formal/cnt/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 A_PENDING_RANGE。

分支 2 个；其余分支完整代码见附录 B。

CNT 07 失败与 MEM 不减 pending

pending 与占用计数 | 合法业务

前提与动作 req 失败，或仅有 MEM 捕获；无 AR、无成功发射。

预期 pending 保持；失败请求仍待发。

要拦住的错误 req 代替 I；MEM 代替成功发射。

实际 Formal 见证 CNT-07 上升沿采样值

	S0	S1	S2
P before*	0	1	2
A	1	1	0
I	0	0	0
M	0	0	0
R	0	0	0
pending_cnt	1	2	2
N0/N1/N2*	0/0/0	1/0/0	1/0/1
AR token*	T0	T1	–
req token*	–	–	T0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-07 实际正向见证；黄色为关键采样点。

关键采样 S2 A/I/M/R=0000，N=1/0/1，ready=111，pending_cnt=2。

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

```
C_CC_CNT_07_FAIL: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && !issue_fire && !ar_fire && pending_cnt==waiting
);
```

证据 build/formal/cnt/run/engine_0/trace5.vcd；日志 step=3，图示采样 S2。

关联属性 A_PENDING。

分支 2 个；其余分支完整代码见附录 B。

CNT 08 R 仅间接影响 pending

pending 与占用计数 | 合法业务

前提与动作 旧已发项 R，可能释放容量给新 AR；R 无 AR 时 P 不变；R+A 时 P 加 1，另按 I 减 1。

预期 不直接从 P 扣 R；新 A 是否成立取决于本拍 ready。

要拦住的错误 将 AR-R 算法误套 pending。

实际 Formal 见证 CNT-08 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
P before*	0	1	2	2	3	4	4	3
A	1	1	1	1	1	0	0	0
I	0	0	1	0	0	0	1	0
M	0	0	0	0	0	0	1	0
R	0	0	0	0	0	0	0	1
pending_cnt	1	2	2	3	4	4	3	3
N0/N1/N2*	0/0/0	0/1/0	1/1/0	2/1/0	2/2/0	2/3/0	2/3/0	2/3/0
AR token*	T0	T1	T2	T3	T4	–	–	–
req token*	–	–	T0	–	–	–	T2	–

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CNT-08 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=2/3/0，ready=111，pending_cnt=3。

[REF_MODEL] 待发数含本拍新 AR，扣除本拍双 grant 成功；仲裁失败不扣除。

```
A_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (waiting+(ar_fire?1:0)-(issue_fire?1:0))
);
```

```
C_CC_CNT_08_A0: cover property (
    @(posedge clk) disable iff (!rst_n)
    r_fire && ar_fire==0 && !issue_fire && pending_cnt==waiting+0
);
```

证据 build/formal/cnt/run/engine_0/trace10.vcd；日志 step=8，图示采样 S7。

关联属性 A_READY。

分支 2 个；其余分支完整代码见附录 B。

MEM 01 固定四拍返回

MEM 返回与 OTB | 合法业务

前提与动作 命令在 Cn 双 grant 成功；Cn+4 提供 mem_rdata 与 pc_err，仅维持一拍。

预期 在 Cn+4 捕获到该事务；最早 Cn+5 R；不是从 AR 或预选起算。

要拦住的错误 返回日历差一拍。

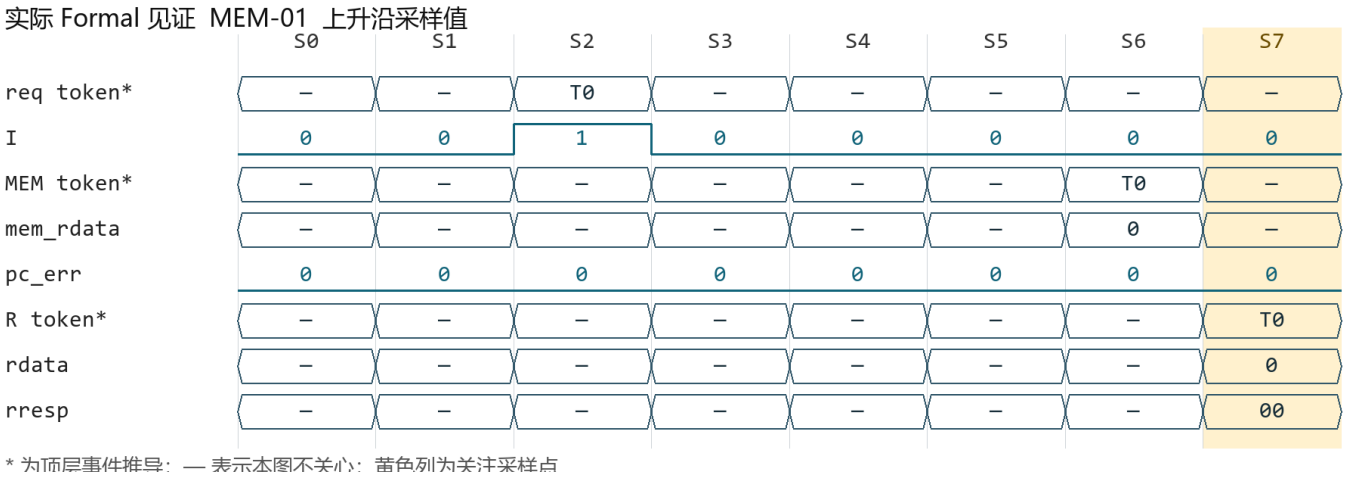


图 MEM-01 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=2/1/0，ready=111，pending_cnt=2。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_01_NEXT_R: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_m && h_response_matches && rsp_e==h_prev_due_slot
);
```

证据 build/formal/mem/run/engine_0/trace4.vcd；日志 step=8，图示采样 S7。

关联属性 A_XAU_R_NO_EARLY_BYPASS, M_DUE。

分支 1 个；本页已列出该场景 cover。

MEM 02 失败请求不启动日历

MEM 返回与 OTB | 合法业务

前提与动作 req 失败后若干拍再成功；在真正成功后第 4 拍返回。

预期 失败那拍+4 的数据不得绑定该事务。

要拦住的错误 用 req 而非 issue_success 启动计时。

实际 Formal 见证 MEM-02 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
req token*	–	–	T0	T0	–	–	–	–
I	0	0	0	1	0	0	0	0
MEM token*	–	–	–	–	–	–	–	T0
mem_rdata	–	–	–	–	–	–	–	1<<127
pc_err	0	0	0	0	0	0	0	0
R token*	–	–	–	–	–	–	–	–
rdata	–	–	–	–	–	–	–	–
rresp	–	–	–	–	–	–	–	–

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-02 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0010，N=0/1/1，ready=111，pending_cnt=1。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_i_history[3:0]==4'b1000 && mem_capture_due &&
        h_failed_before_success[mem_capture_slot]
);
```

证据 build/formal/mem/run/engine_0/trace265.vcd；日志 step=8，图示采样 S7。

关联属性 M_CALENDAR。

分支 1 个；本页已列出该场景 cover。

MEM 03 连续成功对应连续捕获

MEM 返回与 OTB | 合法业务

前提与动作 Cn..Cn+3 每拍成功一条；四拍后逐拍给不同数据和 pc_err。

预期 返回时序与成功顺序一致，逐项分开缓存。

要拦住的错误 日历覆盖、移位暂停、entry 标签错位。

实际 Formal 见证 MEM-03 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
req token*	T0	T1	T2	T3	T4	–	T4	T4
I	1	1	1	1	0	0	0	0
MEM token*	–	–	–	–	T0	T1	T2	T3
mem_rdata	–	–	–	–	0	0	0	1<<127
pc_err	0	0	0	0	0	0	0	1
R token*	–	–	–	–	–	–	T0	–
rdata	–	–	–	–	–	–	0	–
rresp	–	–	–	–	–	–	00	–

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-03 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0010，N=2/0/3，ready=111，pending_cnt=2。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_03_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_m_history[2:0]==3'b111 && mem_capture_due && mem_rdata!=h_prev_mem &&
        pc_err!=h_prev_pc
);
```

证据 build/formal/mem/run/engine_0/trace268.vcd；日志 step=10，图示采样 S9。

关联属性 A_R_RESP, C_FOUR_SUCCESS。

分支 1 个；本页已列出该场景 cover。

覆盖差异 当前 cover 要求连续四拍捕获，但只检查最后相邻两拍的数据与 pc_err 不同；本图前三拍数据相同，尚未见证每拍均不同，见附录 E。

MEM 04 带空洞的返回流水

MEM 返回与 OTB | 合法业务

前提与动作 成功序列 1/0/1/0/0/1；四拍后在对应位置给数据。

预期 捕获掩码同样有空洞；不得把无效拍压缩掉。

要拦住的错误 只在成功时移位，变成变长延迟。

实际 Formal 见证 MEM-04 上升沿采样值

	S4	S5	S6	S7	S8	S9	S10	S11
req token*	T2	T3	T1	T1	T3	T3	T3	T3
I	1	0	0	1	0	0	0	0
MEM token*	–	–	T0	–	T2	–	–	T1
mem_rdata	–	–	0	–	0	–	–	0
pc_err	0	0	0	0	0	0	0	0
R token*	–	–	–	–	T0	–	–	–
rdata	–	–	–	–	0	–	–	–
rresp	–	–	–	–	00	–	–	–

* 为顶层事件推导：— 表示本图不关心；黄色列为关注采样点

图 MEM-04 实际正向见证；黄色为关键采样点。

关键采样 S11 A/I/M/R=0010，N=2/0/3，ready=111，pending_cnt=3。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_m_history[4:0]==5'b10100 && mem_capture_due
);
```

证据 build/formal/mem/run/engine_0/trace270.vcd；日志 step=12，图示采样 S11。

关联属性 M_CALENDAR。

分支 1 个；本页已列出该场景 cover。

MEM 05 没有到期命令的数据扰动

MEM 返回与 OTB | 合法业务

前提与动作 返回日历该拍为空；mem_rdata/pc_err 任意变化。

预期 忽略该拍输入；不产生 done 或 R。

要拦住的错误 每拍无条件写 OTB。

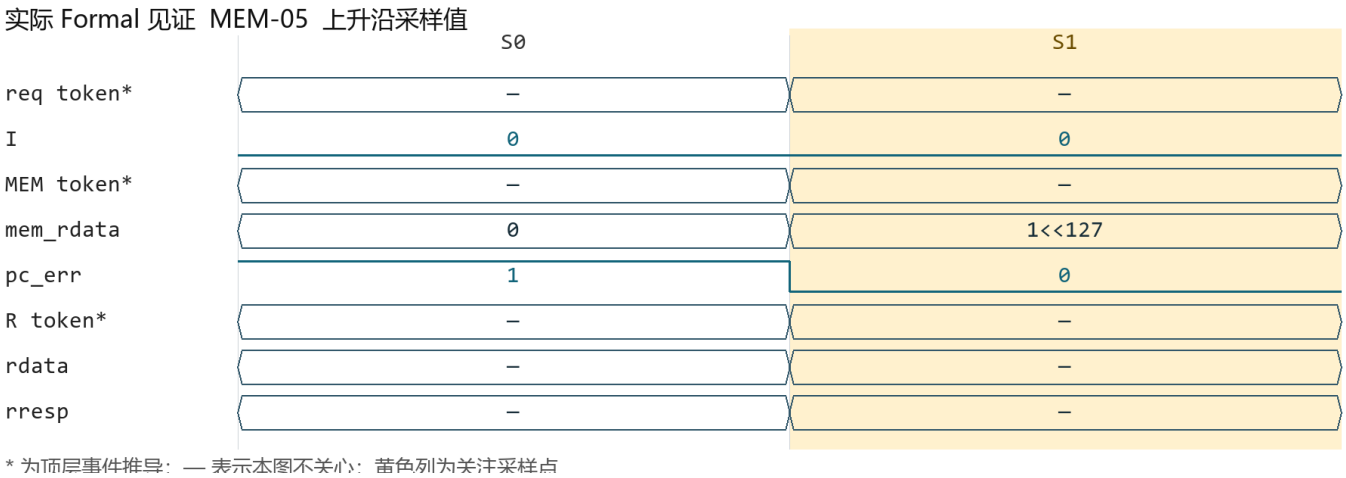


图 MEM-05 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_05_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    !mem_capture_due && mem_rdata!=h_prev_mem && pc_err!=h_prev_pc && used>0
);
```

证据 build/formal/mem/run/engine_0/trace0.vcd；日志 step=2，图示采样 S1。

关联属性 A_XAU_R_HAS_PENDING。

分支 1 个；本页已列出该场景 cover。

MEM 06 返回脉冲只有一拍

MEM 返回与 OTB | 合法业务

前提与动作 到期拍给特征数据与 pc_err；下一拍立即改变输入总线。

预期 OTB 保留到期拍值，晚 R 仍正确。

要拦住的错误 晚一拍采样或 R 直接连 MEM 总线。

实际 Formal 见证 MEM-06 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
req token*	–	T0	T1	–	T2	T2	–	T2
I	0	1	1	0	0	0	0	0
MEM token*	–	–	–	–	–	T0	T1	–
mem_rdata	0	0	0	0	0	0	0	1<<127
pc_err	0	0	0	0	0	0	0	1
R token*	–	–	–	–	–	–	–	T0
rdata	–	–	–	–	–	–	–	0
rresp	–	–	–	–	–	–	–	00

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-06 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=2/0/3，ready=111，pending_cnt=3。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && h_done_wait[rsp_e]>=1 && mem_rdata!=m_data[rsp_e] &&
        pc_err!=m_pc[rsp_e]
);
```

证据 build/formal/mem/run/engine_0/trace267.vcd；日志 step=9，图示采样 S8。

关联属性 A_XAU_R_RESP_ORDER。

分支 1 个；本页已列出该场景 cover。

覆盖差异 当前 cover 检查 R 时总线与保存值不同，没有限定在捕获后的紧邻下一拍就变化；本图到捕获后第二拍才变化，见附录 E。

MEM 07 R 不 ready 期间照常捕获

MEM 返回与 OTB | 合法业务

前提与动作 一个或多个源 rready=0；MEM 到期连续返回。

预期 仍逐项缓存；R 阻塞不能暂停返回日历。

要拦住的错误 将 R ready 用作 MEM 采样使能。

实际 Formal 见证 MEM-07 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
req token*	–	–	T0	T1	–	T2	T3	T2
I	0	0	1	1	0	0	1	0
MEM token*	–	–	–	–	–	–	T0	T1
mem_rdata	–	–	–	–	–	–	0	1<<127
pc_err	0	0	0	0	0	0	0	0
rready	000	000	111	111	011	010	111	000
R token*	–	–	–	–	–	–	–	–
rdata	–	–	–	–	–	–	–	–
rresp	–	–	–	–	–	–	–	–

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-07 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0010，N=0/1/4，ready=111，pending_cnt=2。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    mem_capture_due && h_prev_m && rready==0 && rvalid==0
);
```

证据 build/formal/mem/run/engine_0/trace264.vcd；日志 step=8，图示采样 S7。

关联属性 A_R_READY。

分支 1 个；本页已列出该场景 cover。

MEM 08 新 AR 与 MEM 同拍

MEM 返回与 OTB | 合法业务

前提与动作 A 为新事务，M 为四拍前成功的旧事务；两事件同拍，合法槽位互异。

预期 新槽预留与旧槽捕获各自完成，N 加 1；不能覆盖旧数据。

要拦住的错误 分配/捕获只允许执行一个全局分支。

实际 Formal 见证 MEM-08 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6
A	1	1	1	1	1	1	1
req token*	–	–	T0	T1	T1	T1	T1
I	0	0	1	0	0	0	0
MEM token*	–	–	–	–	–	–	T0
mem_rdata	–	–	–	–	–	–	1<<127
pc_err	0	0	0	0	0	0	0
R token*	–	–	–	–	–	–	–
rdata	–	–	–	–	–	–	–
rresp	–	–	–	–	–	–	–

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-08 实际正向见证；黄色为关键采样点。

关键采样 S6 A/I/M/R=1010，N=5/0/1，ready=111，pending_cnt=6。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && mem_capture_due && alloc_e!=mem_capture_slot
);
```

证据 build/formal/mem/run/engine_0/trace1.vcd；日志 step=7，图示采样 S6。

关联属性 M_STATE。

分支 1 个；本页已列出该场景 cover。

MEM 09 发射成功与旧 MEM 同拍

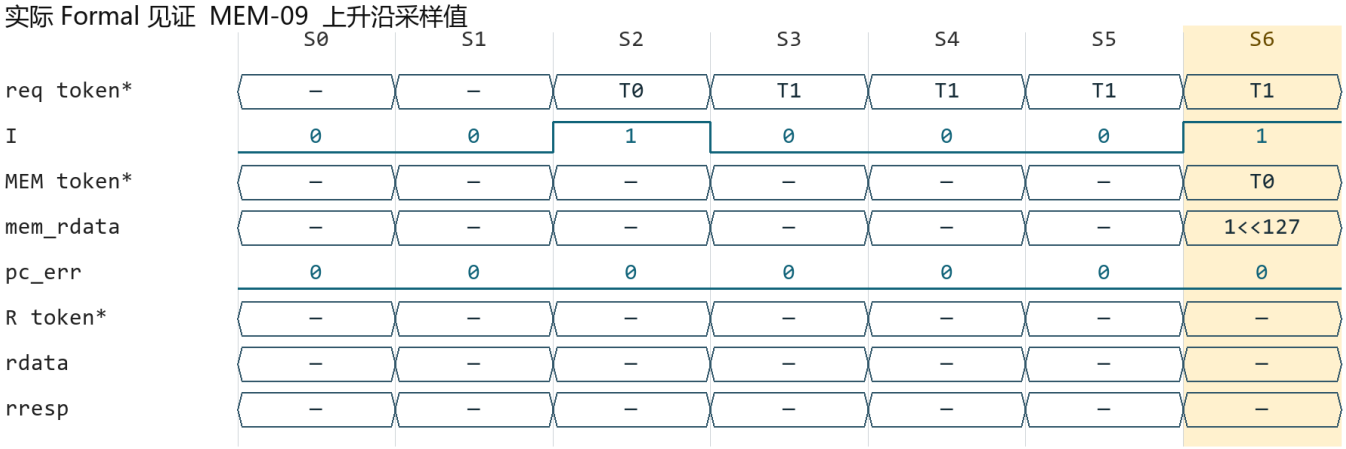
MEM 返回与 OTB | 合法业务

前提与动作 新 I 与四拍前 I 到期；同拍写日历头并读日历尾。

预期 两事件均保留，流水连续推进。

要拦住的错误 头尾更新次序覆盖尾标签。

实际 Formal 见证 MEM-09 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-09 实际正向见证；黄色为关键采样点。

关键采样 S6 A/I/M/R=1110，N=5/0/1，ready=111，pending_cnt=5。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_09_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && mem_capture_due && m_attempt_e!=mem_capture_slot
);
```

证据 build/formal/mem/run/engine_0/trace3.vcd；日志 step=7，图示采样 S6。

关联属性 C_FOUR_SUCCESS。

分支 1 个；本页已列出该场景 cover。

MEM 10 MEM 与 R 同拍不同命令

MEM 返回与 OTB | 合法业务

前提与动作 已完成 A 正在 R；B 本拍才 MEM；同拍捕获 B、退役 A。

预期 R 给 A 的旧缓存；B 最早下拍响应；两槽不得相同。

要拦住的错误 共享 data mux 直接给本拍 B。

实际 Formal 见证 MEM-10 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
req token*	–	–	T0	T1	–	T2	T2	–
I	0	0	1	1	0	0	0	0
MEM token*	–	–	–	–	–	–	T0	T1
mem_rdata	–	–	–	–	–	–	0	1<<127
pc_err	0	0	0	0	0	0	0	0
R token*	–	–	–	–	–	–	–	T0
rdata	–	–	–	–	–	–	–	0
rresp	–	–	–	–	–	–	–	00

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-10 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0011，N=2/0/2，ready=111，pending_cnt=2。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_10_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    mem_capture_due && h_response_matches && mem_capture_slot!=rsp_e
);
```

证据 build/formal/mem/run/engine_0/trace266.vcd；日志 step=8，图示采样 S7。

关联属性 A_XAU_R_NO_EARLY_BYPASS。

分支 1 个；本页已列出该场景 cover。

MEM 11 MEM 不能当拍旁路 R

MEM 返回与 OTB | 合法业务

前提与动作 命令首次到期，之前未 done，rready=1；到期拍观察 rvalid。

预期 本拍不能为该命令产生 R；下一拍才具备资格。

要拦住的错误 用 capture_valid 直接组成 R 候选。

实际 Formal 见证 MEM-11 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6
req token*	–	–	T0	T1	T1	T1	T1
I	0	0	1	0	0	0	0
MEM token*	–	–	–	–	–	–	T0
mem_rdata	–	–	–	–	–	–	1<<127
pc_err	0	0	0	0	0	0	0
rready	000	000	000	000	000	000	100
R token*	–	–	–	–	–	–	–
rdata	–	–	–	–	–	–	–
rresp	–	–	–	–	–	–	–

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-11 实际正向见证；黄色为关键采样点。

关键采样 S6 A/I/M/R=1010，N=5/0/1，ready=111，pending_cnt=6。

[REF_MODEL] 队头未完成时，年轻命令即使已完成，也不能让该源提前响应。

```
A_XAU_R_NO_EARLY_BYPASS: assert property (
    @(posedge clk) disable iff (!rst_n)
    (oldest_valid && !m_done[oldest])
    |-> (!rvalid[g])
);
```

```
C_CC_MEM_11_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    mem_capture_due && head_valid[m_src[mem_capture_slot]] &&
        head[m_src[mem_capture_slot]]==mem_capture_slot &&
        rready[m_src[mem_capture_slot]] && !rvalid[m_src[mem_capture_slot]]
);
```

证据 build/formal/mem/run/engine_0/trace2.vcd；日志 step=7，图示采样 S6。

关联属性 M_NO_BYPASS。

分支 1 个；本页已列出该场景 cover。

MEM 12 128 位数据全宽特征

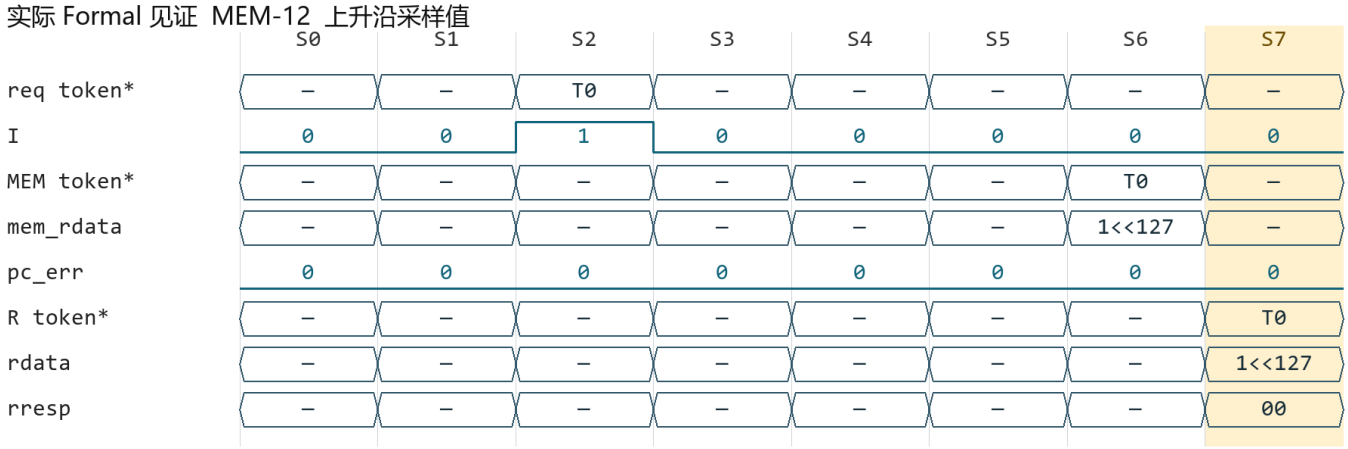
MEM 返回与 OTB | 合法业务

前提与动作 多条不同命令返回不同数据；全 0、全 1、交替位、walking-1、walking-0、随机；在各字节边界翻转。

预期 R 数据与该事务捕获值逐位一致。

要拦住的错误 只比较低 32 位；字节/位交换；高位截断。

实际 Formal 见证 MEM-12 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 MEM-12 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=2/1/0，ready=111，pending_cnt=2。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_MEM_12_W1_127: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<127)
);
```

证据 build/formal/mem/run/engine_0/trace263.vcd；日志 step=8，图示采样 S7。

关联属性 A_XAU_R_DATA_ORDER。

分支 260 个；其余分支完整代码见附录 B。

MEM 13 parity 错误交替

MEM 返回与 OTB | 合法业务

前提与动作 连续到期，pc_err=1/0/1/0；R 可延迟、跨源交错。

预期 pc_err 随对应 entry 保存，不能使用当前总线的错误位。

要拦住的错误 parity 与数据相差一拍。

实际 Formal 见证 MEM-13 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
req token*	T0	T1	T2	T3	T4	–	T4	T4
I	1	1	1	1	0	0	0	0
MEM token*	–	–	–	–	T0	T1	T2	T3
mem_rdata	–	–	–	–	0	0	0	0
pc_err	0	0	0	0	1	0	1	0
R token*	–	–	–	–	–	–	T0	–
rdata	–	–	–	–	–	–	0	–
rresp	–	–	–	–	–	–	01	–

* 为顶层事件推导：— 表示本图不关心；黄色列为关注采样点

图 MEM-13 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0010，N=2/0/3，ready=111，pending_cnt=2。

[REF_MODEL] R 错误状态来自预期事务；地址越界优先于奇偶校验异常。

```
A_R_RESP: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rresp == expected_resp)
);
```

```
C_CC_MEM_13_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_m_history[2:0]==3'b111 && mem_capture_due && h_pc_history==3'b101 && !pc_err
);
```

证据 build/formal/mem/run/engine_0/trace269.vcd；日志 step=10，图示采样 S9。

关联属性 A_XAU_R_RESP_ORDER。

分支 1 个；本页已列出该场景 cover。

MEM 14 MEM 提前/延后/丢返回

MEM 返回与 OTB | 环境违约

前提与动作 已成功事务按协议应在 n+4 返回；独立注入 n+3/n+5 或 n+4 给错误数据。

预期 无返回 valid/tag，DUT 按 n+4 无条件采样；不能承诺自动识别丢包或重定时。

要拦住的错误 把任意数据引起的错误归为 DUT 时间恢复能力缺陷。

注错或观测边界示意 未作为已命中的 Formal 波形

Cn 成功发射		Cn+4 唯一采样点		Cn+5 或之后响应
I=1	→	mem_rdata=D_bus	→	rdata=D_bus
外部预期 D_true		若 D_bus 错误须有外部真值		端口自比较不能发现丢返回

本图描述检查责任或拟注错路径，不声称该非法内部状态已经被注入。

图 MEM-14 未执行注错或不可直接观察的边界示意。

检查边界 MEM 无 valid/tag/独立存储器真值：仅当前顶层信号不能分辨延迟或错误数据；不得伪造 assume/cover。需外部 MEM 参考或注错记录。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

RSP 01 没有完成的源队头

R 保序与 RR | 合法业务

前提与动作 三路均无可返回队头；rready 任意。

预期 rvalid=000；无效共享负载不检查；RR 不推进。

要拦住的错误 未 done 就响应。

实际 Formal 见证 RSP-01 上升沿采样值



* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 RSP-01 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] 只允许响应已成功发射且已捕获 MEM 数据的 FIFO 队头。

```
A_XAU_R_HEAD_COMPLETE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && m_sent[oldest] && m_done[oldest]))
);
```

```
C_CC_RSP_01_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==0 && rvalid==0 && used>0
);
```

证据 build/formal_retry/rsp/run/engine_0/trace0.vcd；日志 step=2，图示采样 S1。

关联属性 A_RR_CHOICE。

分支 1 个；本页已列出该场景 cover。

RSP 02 完成但 rready 全零

R 保序与 RR | 合法业务

前提与动作 三路可有缓存完成项；rready=000 持续多拍。

预期 rvalid=000；缓存不变；RR 保持。

要拦住的错误 先拉 valid 再等待 ready，不符合本接口。

实际 Formal 见证 RSP-02 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	T5	T6	T7
req token*	–	–	T0	–	T1	T1	T4	T1
I	0	0	1	0	0	0	1	0
MEM token*	–	–	–	–	–	–	T0	–
rready	000	000	111	111	111	101	000	000
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	000	000
rvalid	000	000	000	000	000	000	000	000
R token*	–	–	–	–	–	–	–	–
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 RSP-02 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1000，N=1/2/4，ready=111，pending_cnt=6。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    (m_live&m_done)!=0 && rready==0 && rvalid==0 && h_prev_rready==0 && !h_prev_r
);
```

证据 build/formal_retry/rsp/run/engine_0/trace1.vcd；日志 step=8，图示采样 S7。

关联属性 A_R_DATA, A_R_READY。

分支 1 个；本页已列出该场景 cover。

RSP 03 只有一路候选

R 保序与 RR | 合法业务

前提与动作 其他两路空、未完成或不 ready；逐一覆盖候选 0/1/2。

预期 立即选择该候选；共享负载属于它，rvalid onehot。

要拦住的错误 按固定指针路等待而不跳过。

实际 Formal 见证 RSP-03 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	–	T2	–	T3	–	T4
req token*	–	–	T0	–	T1	T1	T2	T1
I	0	0	1	0	0	0	1	0
MEM token*	–	–	–	–	–	–	T0	–
rready	000	000	111	111	111	101	000	111
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	000	001
rvalid	000	000	000	000	000	000	000	001
R token*	–	–	–	–	–	–	–	T0
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；黄色列为关键采样点

图 RSP-03 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=2/1/1，ready=111，pending_cnt=3。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_03_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==(3'b1<<0) && rvalid==(3'b1<<0) && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace3.vcd；日志 step=8，图示采样 S7。

关联属性 A_R_METADATA, A_R_ONEHOT。

分支 3 个；其余分支完整代码见附录 B。

RSP 04 三路同时候选

R 保序与 RR | 合法业务

前提与动作 当前 RR 指向 0；各路持续可响应；连续三个 R 拍。

预期 顺序 0→1→2，下一轮回到 0。

要拦住的错误 指针无握手推进或从错误源起步。

实际 Formal 见证 RSP-04 上升沿采样值

	S4	S5	S6	S7	S8	S9	S10	S11
AR token*	T4	T5	–	–	T6	T7	–	–
req token*	T2	T3	T4	–	T5	–	T5	T5
I	1	1	1	0	0	0	0	0
MEM token*	–	–	T0	T1	T2	T3	T4	–
rready	111	111	111	101	100	111	111	111
RR start*	0	0	0	0	0	0	1	2
eligible*	000	000	000	000	000	111	111	111
rvalid	000	000	000	000	000	001	010	100
R token*	–	–	–	–	–	T1	T0	T2
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 RSP-04 实际正向见证；黄色为关键采样点。

关键采样 S11 A/I/M/R=0001，N=1/2/3，ready=111，pending_cnt=3。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_rr012_phase==2 && eligible==7 && rvalid==4 && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace34.vcd；日志 step=12，图示采样 S11。

关联属性 A_RR_BOUND。

分支 1 个；本页已列出该场景 cover。

RSP 05 两路候选跳过空缺

R 保序与 RR | 合法业务

前提与动作 分别候选 011/101/110，RR 可处于 0/1/2；每拍实际响应。

预期 按 RR 起点循环查找第一候选；不是比较谁的全局命令更老。

要拦住的错误 用全局年龄替代跨源 RR。

实际 Formal 见证 RSP-05 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	–	T2	–	–	T3	–	–
req token*	–	T0	T1	–	T2	–	T2	T2
I	0	1	1	0	0	0	0	0
MEM token*	–	–	–	–	–	T0	T1	–
rready	000	111	111	111	111	111	010	111
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	000	011
rvalid	000	000	000	000	000	000	000	001
R token*	–	–	–	–	–	–	–	T0
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；黄色列为关键采样点

图 RSP-05 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=1/2/1，ready=111，pending_cnt=2。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_05_M3RR0: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==3 && m_rr==0 && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace9.vcd；日志 step=9，图示采样 S8。

关联属性 C_READY_MASK_RR。

分支 9 个；其余分支完整代码见附录 B。

RSP 06 RR 到末尾回绕

R 保序与 RR | 合法业务

前提与动作 RR 指向 2，候选含 2/0；源 2 实际 R 后继续响应。

预期 下次从 0 开始扫描，不产生非法源 3。

要拦住的错误 指针二位自然加一未做模 3。

实际 Formal 见证 RSP-06 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	T2	T3	–	T4	T5	T6	–
req token*	–	T0	T1	T2	T3	–	T4	T5
I	0	1	1	1	1	0	1	0
MEM token*	–	–	–	–	–	T0	T1	T2
rready	000	110	110	111	111	111	111	111
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	100	001
rvalid	000	000	000	000	000	000	100	001
R token*	–	–	–	–	–	–	T0	T1
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；蓝色列为关注采样点

图 RSP-06 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0011，N=3/0/3，ready=111，pending_cnt=2。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_r && h_prev_rsrc==2 && m_rr==0 && rvalid[0] && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace11.vcd；日志 step=9，图示采样 S8。

关联属性 A_R_ONEHOT。

分支 1 个；本页已列出该场景 cover。

RSP 07 无响应的空拍

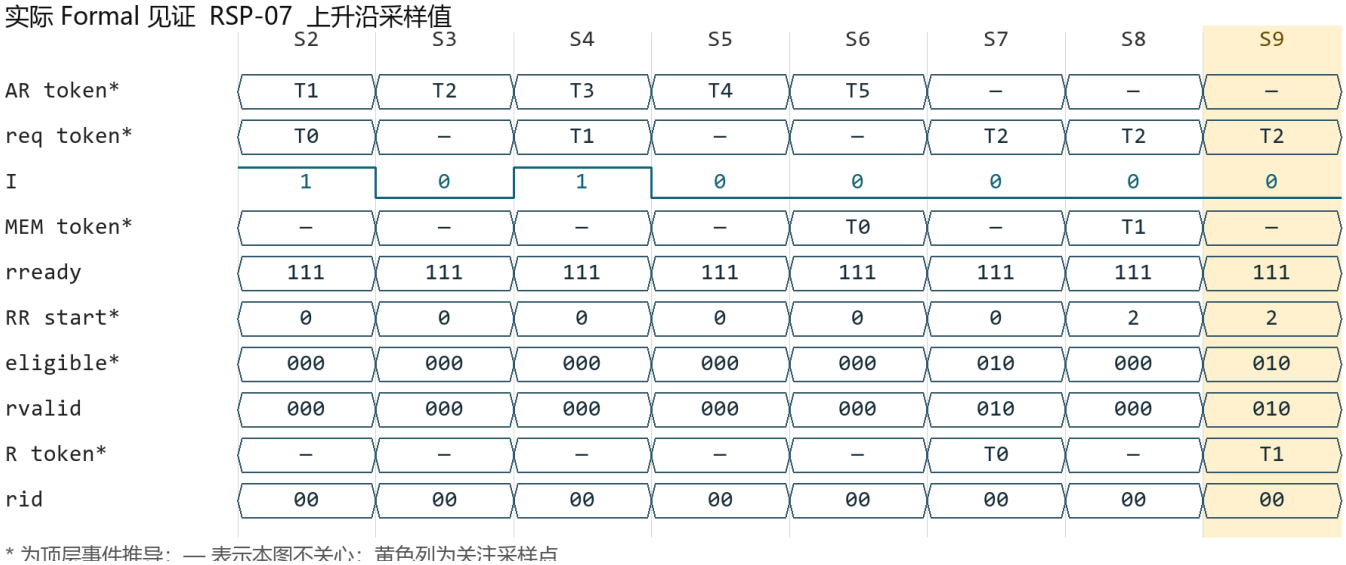
R 保序与 RR | 合法业务

前提与动作 中途所有候选消失一拍或多拍；后续候选恢复。

预期 RR 起点维持空拍前值；不因时间流逝旋转。

要拦住的错误 每拍无条件加 RR 指针。

实际 Formal 见证 RSP-07 上升沿采样值



* 为低层事件推导；— 表示本图不关心；蓄色列为关注采样点

图 RSP-07 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0001，N=2/2/1，ready=111，pending_cnt=4。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_rr_gap_seen && r_fire && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace25.vcd；日志 step=10，图示采样 S9。

分支 1 个；本页已列出该场景 cover。

RSP 08 rready 变化改变当拍候选

R 保序与 RR | 合法业务

前提与动作 某源已完成，另一路也可响应；在采样前切换 rready mask。

预期 只在 ready=1 源中选；没有 valid 锁住等待的状态。

要拦住的错误 沿用标准 AXI R 反压锁定算法。

实际 Formal 见证 RSP-08 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	–	T2	–	–	T3	–	–
req token*	–	T0	T1	–	T2	–	T2	T2
I	0	1	1	0	0	0	0	0
MEM token*	–	–	–	–	–	T0	T1	–
rready	000	111	111	111	111	111	011	111
RR start*	0	0	0	0	0	0	0	1
eligible*	000	000	000	000	000	000	001	010
rvalid	000	000	000	000	000	000	001	010
R token*	–	–	–	–	–	–	T0	T1
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；蓝色列为关注采样点

图 RSP-08 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=0/2/1，ready=111，pending_cnt=2。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_eligible!=0 && eligible!=0 && rready!=h_prev_rready && rvalid!=0 &&
        (rvalid&~rready)==0 && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace8.vcd；日志 step=9，图示采样 S8。

关联属性 A_R_READY。

分支 1 个；本页已列出该场景 cover。

RSP 09 同源 3 先发、先完成

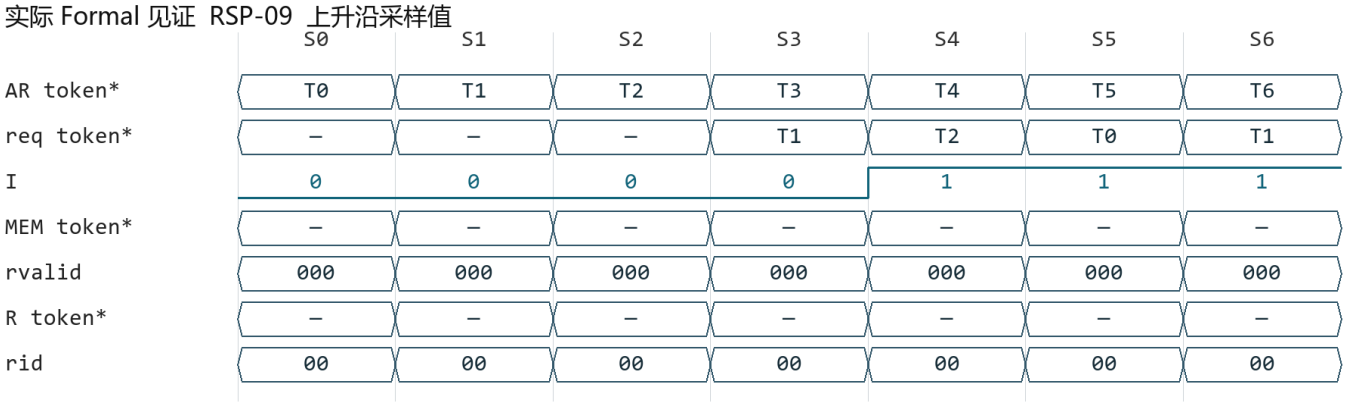
R 保序与 RR | 合法业务

前提与动作 同源 AR 顺序为 1/2/3，3 的 bank 先可用；3 先成功且 MEM 返回；1/2 尚未 R。

预期 源内不能先 R3；最终 R 顺序必须 1→2→3。

要拦住的错误 在同源完成集合中挑最老已完成项。

实际 Formal 见证 RSP-09 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；蓄色列为关注采样点

图 RSP-09 实际正向见证；黄色为关键采样点。下一页接续返回过程。

看图要点 T0、T1、T2 是同源前三条接收命令；T2 先成功并先完成，R 仍必须等待 T0、T1。后续页给出完整返回链。

[REF_MODEL] 只允许响应已成功发射且已捕获 MEM 数据的 FIFO 队头。

```
A_XAU_R_HEAD_COMPLETE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && m_sent[oldest] && m_done[oldest]))
);
```

```
C_CC_RSP_09_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[0].order_phase==5 && sources[0].source_r &&
        sources[0].oldest==sources[0].third_token && sources[0].head_payload_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace39.vcd；日志 step=13，图示采样 S12。

关联属性 C_XAU_EARLY_THIRD_BLOCKED, C_XAU_R_123_AFTER_ISSUE3_FIRST。

分支 3 个；其余分支完整代码见附录 B。

RSP 09 单源保序的返回过程

接上页。下图与上页来自同一条轨迹，采样编号连续；没有把不同运行的波形拼在一起。



图 RSP-09 续 已完成的第三条等待前两条完成 R，随后才能返回。

采样拍	AR	成功 I	捕获 M	返回 R
S0	T0	—	—	—
S1	T1	—	—	—
S2	T2	—	—	—
S4	T4	T2	—	—
S5	T5	T0	—	—
S6	T6	T1	—	—
S8	—	T3	T2	—
S9	T8	T5	T0	—
S10	T9	—	T1	T0
S11	T10	—	—	T1
S12	—	—	T3	T2

[REF_MODEL] 有效 R 的 rid 必须属于队头；不按 ID 数值排序，也不要求 ID 唯一。

```
A_XAU_R_ID_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rid == m_id[oldest]))
);
```

同一队头还要同时比较 raddr、port_id、128 位 rdata 和 rresp，不能分别在不同事务中找到匹配字段就算通过。对应 A_XAU_R_ADDR_ORDER、A_XAU_R_PORT_ORDER、A_XAU_R_DATA_ORDER、A_XAU_R_RESP_ORDER 的完整代码见附录 A。

RSP 10 同源第二条完成但第一条未发

R 保序与 RR | 合法业务

前提与动作 队头仍待发，次项已完成；rready=1。

预期 不得 R 次项；先找源队头再检查 done。

要拦住的错误 先过滤 done 再找该源最老。

实际 Formal 见证 RSP-10 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	–	T2	T3	–	–	T4	–
req token*	–	T0	T1	–	T2	–	T0	T0
I	0	0	1	0	1	0	0	0
MEM token*	–	–	–	–	–	–	T1	–
rready	000	111	111	111	111	111	111	111
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	000	000
rvalid	000	000	000	000	000	000	000	000
R token*	–	–	–	–	–	–	–	–
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；蓄色列为关注采样点

图 RSP-10 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0000，N=4/1/0，ready=111，pending_cnt=3。

[REF_MODEL] 队头未完成时，年轻命令即使已完成，也不能让该源提前响应。

```
A_XAU_R_NO_EARLY_BYPASS: assert property (
    @(posedge clk) disable iff (!rst_n)
    (oldest_valid && !m_done[oldest])
    |-> (!rvalid[g])
);
```

```
C_CC_RSP_10_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[0].order_count>=2 && !m_sent[sources[0].order_slot[0]] &&
        m_done[sources[0].order_slot[1]] && rready[0] && !rvalid[0]
);
```

证据 build/formal_retry/rsp/run/engine_0/trace16.vcd；日志 step=9，图示采样 S8。

关联属性 A_XAU_R_HEAD_COMPLETE, A_XAU_R_ID_ORDER。

分支 3 个；其余分支完整代码见附录 B。

RSP 11 不同 ID 仍要同源保序

R 保序与 RR | 合法业务

前提与动作 同源命令 ID 各不相同；乱序发射与完成。

预期 R 按 AR 接收顺序；不是仅同 ID 内保序。

要拦住的错误 错用 AXI 同 ID 规则放行别的 ID。

实际 Formal 见证 RSP-11 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	–	T2	T3	–	–	T4	–
arid	80	00	00	00	00	00	80	00
req token*	–	T0	T1	–	T2	–	T3	T3
MEM token*	–	–	–	–	–	T0	T1	–
rready	000	111	111	111	111	111	100	011
rvalid	000	000	000	000	000	000	000	001
R token*	–	–	–	–	–	–	–	T0
rid	00	00	00	00	00	00	00	00

* 为顶层事件推导；— 表示本图不关心；黄色列为关键采样点

图 RSP-11 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=4/1/0，ready=111，pending_cnt=2。

[REF_MODEL] 有效 R 的 rid 必须属于队头；不按 ID 数值排序，也不要求 ID 唯一。

```
A_XAU_R_ID_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rid == m_id[oldest]))
);
```

```
C_CC_RSP_11_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[0].order_count>=2 &&
        m_id[sources[0].order_slot[0]]!=m_id[sources[0].order_slot[1]] &&
        m_done[sources[0].order_slot[1]] && sources[0].source_r &&
        sources[0].head_payload_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace18.vcd；日志 step=9，图示采样 S8。

关联属性 A_XAU_R_ADDR_ORDER。

分支 3 个；其余分支完整代码见附录 B。

RSP 12 重复 ID 不合并事务

R 保序与 RR | 合法业务

前提与动作 同源或跨源 arid 重复，其他字段/数据不同；多条命令乱序完成。

预期 每次 R 返回对应源队头的完整记录，不能以 rid 查唯一事务。

要拦住的错误 参考模型用 ID 当唯一 key。

实际 Formal 见证 RSP-12 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	–	T2	T3	–	–	T4	–
arid	00	00	00	00	00	00	80	00
req token*	–	T0	T1	–	T2	–	T3	T3
MEM token*	–	–	–	–	–	T0	T1	–
rready	000	111	111	111	111	111	100	011
rvalid	000	000	000	000	000	000	000	001
R token*	–	–	–	–	–	–	–	T0
rid	00	00	00	00	00	00	00	00

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 RSP-12 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=4/1/0，ready=111，pending_cnt=2。

[REF_MODEL] 有效 R 的 rid 必须属于队头；不按 ID 数值排序，也不要求 ID 唯一。

```
A_XAU_R_ID_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rid == m_id[oldest]))
);
```

```
C_CC_RSP_12_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[0].order_count>=2 &&
        m_id[sources[0].order_slot[0]]==m_id[sources[0].order_slot[1]] &&
        m_data[sources[0].order_slot[0]]!=m_data[sources[0].order_slot[1]] &&
        m_done[sources[0].order_slot[1]] && sources[0].source_r &&
        sources[0].head_payload_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace17.vcd；日志 step=9，图示采样 S8。

关联属性 A_XAU_R_DATA_ORDER, C_XAU_REPEAT_ID_HEAD_R。

分支 3 个；其余分支完整代码见附录 B。

RSP 13 一源阻塞其他源仍推进

R 保序与 RR | 合法业务

前提与动作 XAU0 老队头未完成；XAU1/2 有已完成头且 ready；多拍 R 选择。

预期 可返回 XAU1/2，不要求全局 AR 顺序。

要拦住的错误 用单条全局响应 FIFO 阻塞所有源。

实际 Formal 见证 RSP-13 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	T5	T6	T7
req token*	–	–	T0	–	T1	T1	T4	T1
I	0	0	1	0	0	0	1	0
MEM token*	–	–	–	–	–	–	T0	–
rready	000	000	111	111	111	101	000	111
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	000	100
rvalid	000	000	000	000	000	000	000	100
R token*	–	–	–	–	–	–	–	T0
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；蓝色列为关注采样点

图 RSP-13 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=1/2/4，ready=111，pending_cnt=6。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_RSP_13_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[0] && !m_done[head[0]] && (rvalid&~(3'b1<<0))!=0 &&
        h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace2.vcd；日志 step=8，图示采样 S7。

关联属性 C_XAU_OTHER_SOURCE_PROGRESS。

分支 3 个；其余分支完整代码见附录 B。

RSP 14 同源队头 R 后立即换头

R 保序与 RR | 合法业务

前提与动作 某源前两条都已完成；本拍第一条 R，下一拍第二条仍 ready。

预期 第二条进入候选，可按 RR 获胜；元数据与数据同步切换。

要拦住的错误 弹头时多出一拍空泡或混用新旧头。

实际 Formal 见证 RSP-14 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	T2	–	–	T3	T4	–	–
req token*	–	T0	T1	T2	T2	–	T3	T3
I	0	1	1	0	1	0	0	0
MEM token*	–	–	–	–	–	T0	T1	–
rready	000	111	111	111	111	111	111	011
RR start*	0	0	0	0	0	0	0	1
eligible*	000	000	000	000	000	000	001	001
rvalid	000	000	000	000	000	000	001	001
R token*	–	–	–	–	–	–	T0	T1
rid	00	00	00	00	00	00	FE	00

* 为低层事件推导；— 表示本图不关心；蓝色列为关注采样点

图 RSP-14 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=0001，N=1/2/1，ready=111，pending_cnt=2。

[REF_MODEL] 有效 R 的 128 位数据必须来自同一个队头事务的 MEM 捕获记录。

```
A_XAU_R_DATA_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rdata == m_data[oldest]))
);
```

```
C_CC_RSP_14_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_r && h_prev_rsrc==0 && rvalid[0] && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace14.vcd；日志 step=9，图示采样 S8。

关联属性 A_XAU_R_ID_ORDER。

分支 3 个；其余分支完整代码见附录 B。

RSP 15 同源唯一旧项 R 与新 AR

R 保序与 RR | 合法业务

前提与动作 源队列仅一项且完成，AR 同源接收；同拍出入。

预期 返回旧项；新项变新队头但未完成，不能下拍凭旧 done 返回。

要拦住的错误 FIFO 空/非空同时更新错误。

实际 Formal 见证 RSP-15 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	–	T2	–	T3	–	T4
req token*	–	–	T0	–	T1	T1	T2	T1
I	0	0	1	0	0	0	1	0
MEM token*	–	–	–	–	–	–	T0	–
rready	000	000	111	111	111	101	000	111
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	000	001
rvalid	000	000	000	000	000	000	000	001
R token*	–	–	–	–	–	–	–	T0
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；蓝色列为关注采样点

图 RSP-15 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=1/2/1，ready=111，pending_cnt=3。

[NO_REF_MODEL_HISTORY] 只要该源 rvalid 有效，就必须有之前接受而尚未返回的命令。

```
A_CC_R_NONEMPTY: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid[cs] |-> (h_occ[cs]!=0)
);
```

```
C_CC_RSP_15_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[0].order_count==1 && sources[0].source_r && sources[0].source_ar &&
        sources[0].head_payload_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace4.vcd；日志 step=8，图示采样 S7。

关联属性 A_XAU_R_NO_EARLY_BYPASS, C_XAU_SAME_CYCLE_POP_PUSH。

分支 3 个；其余分支完整代码见附录 B。

RSP 16 持续候选的 RR 等待上界

R 保序与 RR | 合法业务

前提与动作 某源队头持续 done、rready 持续 1；其他两源也持续候选。

预期 至多先服务其他两次 R；最迟第三个连续选择拍轮到该源。

要拦住的错误 RR 公平性与上游 src 公平性混为一谈。

实际 Formal 见证 RSP-16 上升沿采样值

	S4	S5	S6	S7	S8	S9	S10	S11
AR token*	–	–	–	T4	T5	–	–	–
req token*	T2	T3	–	–	–	–	T4	T4
I	1	1	0	0	0	0	0	0
MEM token*	–	–	T0	T1	T2	T3	–	–
rready	111	110	111	001	101	111	111	111
RR start*	0	0	0	0	0	1	2	0
eligible*	000	000	000	000	001	011	101	001
rvalid	000	000	000	000	001	010	100	001
R token*	–	–	–	–	T1	T0	T3	T2
rid	00	00	00	00	80	00	00	00

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 RSP-16 实际正向见证；黄色为关键采样点。

关键采样 S11 A/I/M/R=0001，N=1/2/0，ready=111，pending_cnt=2。

[REF_MODEL] 该源持续可响应且已被另外两次响应越过时，本拍必须轮到它。

```
A_RR_BOUND: assert property (
    @(posedge clk) disable iff (!rst_n)
    (eligible[g] && bypass_count >= 2)
    |-> (rvalid[g])
);
```

```
C_CC_RSP_16_S0: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible[0] && sources[0].bypass_count==2 && rvalid[0] && h_response_matches
);
```

证据 build/formal_retry/rsp/run/engine_0/trace35.vcd；日志 step=12，图示采样 S11。

关联属性 A_RR_CHOICE。

分支 3 个；其余分支完整代码见附录 B。

RSP 17 R 共享负载只归一个源

R 保序与 RR | 合法业务

前提与动作 多个完成队头且 ready；同拍观察 rvalid 与共享 rdata/rid/port/raddr/rresp。

预期 rvalid 至多一位；所有字段来自同一获胜源的同一队头。

要拦住的错误 各字段按不同仲裁结果拼接。

实际 Formal 见证 RSP-17 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
AR token*	T2	T3	T4	T5	T6	T7	–	–
req token*	T0	T1	T2	–	–	T3	–	T4
I	1	1	1	0	0	1	0	0
MEM token*	–	–	–	–	T0	T1	T2	–
rready	111	111	111	111	111	101	001	111
RR start*	0	0	0	0	0	0	0	0
eligible*	000	000	000	000	000	000	000	111
rvalid	000	000	000	000	000	000	000	001
R token*	–	–	–	–	–	–	–	T2
rid	00	00	00	00	00	00	00	00

* 为低层事件推导；— 表示本图不关心；蓄色列为关注采样点

图 RSP-17 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0001，N=4/2/2，ready=111，pending_cnt=4。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_RSP_17_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==7 && h_response_matches && (rvalid==1 || rvalid==2 || rvalid==4)
);
```

证据 build/formal_retry/rsp/run/engine_0/trace30.vcd；日志 step=10，图示采样 S9。

关联属性 A_R_METADATA, A_R_ONEHOT, A_R_RESP。

分支 1 个；本页已列出该场景 cover。

RSP 18 无效 R 负载不约束

R 保序与 RR | 合法业务

前提与动作 rvalid=000；共享输出可能保持旧值或为零。

预期 只检查 rvalid；不得强制 rdata/rid/raddr/port/rresp 全零。

要拦住的错误 对无效负载 assert 清零导致假失败。

实际 Formal 见证 RSP-18 上升沿采样值

	S0	S1
AR token*	T0	–
req token*	–	–
I	0	0
MEM token*	–	–
rready	000	000
RR start*	0	0
eligible*	000	000
rvalid	000	000
R token*	–	–
rid	00	00

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 RSP-18 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

```
C_CC_RSP_18_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    rvalid==0 && used>0
);
```

证据 build/formal_retry/rsp/run/engine_0/trace0.vcd；日志 step=2，图示采样 S1。

关联属性 A_R_METADATA。

分支 1 个；本页已列出该场景 cover。

ERR 01 正常响应

字段与错误组合 | 合法业务

前提与动作 arresp=00，MEM pc_err=0；命令按任意合法时序完成。

预期 rresp=00，其他字段和数据属于原命令。

要拦住的错误 错误位未清或译码错误。

实际 Formal 见证 ERR-01 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	–	T1	T2	–	T3	T4	–
arresp	00	00	00	00	00	00	00	00
MEM token*	–	–	–	–	–	–	T0	–
pc_err	0	0	0	0	0	0	0	0
R token*	–	–	–	–	–	–	–	T0
rresp	–	–	–	–	–	–	–	00
rid	00	00	00	00	00	00	00	00
r_port_id	0	0	0	0	0	0	0	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ERR-01 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=0/2/3，ready=111，pending_cnt=4。

[REF_MODEL] R 错误状态来自预期事务；地址越界优先于奇偶校验异常。

```
A_R_RESP: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rresp == expected_resp)
);
```

```
C_CC_ERR_01_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_err[rsp_e]==0 && m_pc[rsp_e]==0 && rresp==0
);
```

证据 build/formal/err/run/engine_0/trace7.vcd；日志 step=8，图示采样 S7。

关联属性 A_XAU_R_RESP_ORDER。

分支 1 个；本页已列出该场景 cover。

ERR 02 仅地址越界

字段与错误组合 | 合法业务

前提与动作 arresp=11，pc_err=0；仍发射 MEM，再按源内顺序 R。

预期 rresp=11；不可绕过发射/四拍捕获提前返回。

要拦住的错误 直接在 AR 拍短路错误响应。

实际 Formal 见证 ERR-02 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	–	T1	T2	–	T3	T4	–
arresp	11	00	00	00	00	00	00	00
MEM token*	–	–	–	–	–	–	T0	–
pc_err	0	0	0	0	0	0	0	0
R token*	–	–	–	–	–	–	–	T0
rresp	–	–	–	–	–	–	–	11
rid	00	00	00	00	00	00	00	00
r_port_id	0	0	0	0	0	0	0	4

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ERR-02 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=0/2/3，ready=111，pending_cnt=4。

[REF_MODEL] R 错误状态来自预期事务；地址越界优先于奇偶校验异常。

```
A_R_RESP: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rresp == expected_resp)
);
```

```
C_CC_ERR_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_err[rsp_e]==3 && m_pc[rsp_e]==0 && rresp==3
);
```

证据 build/formal/err/run/engine_0/trace0.vcd；日志 step=8，图示采样 S7。

关联属性 A_REQ, A_XAU_R_NO_EARLY_BYPASS。

分支 1 个；本页已列出该场景 cover。

ERR 03 仅奇偶异常

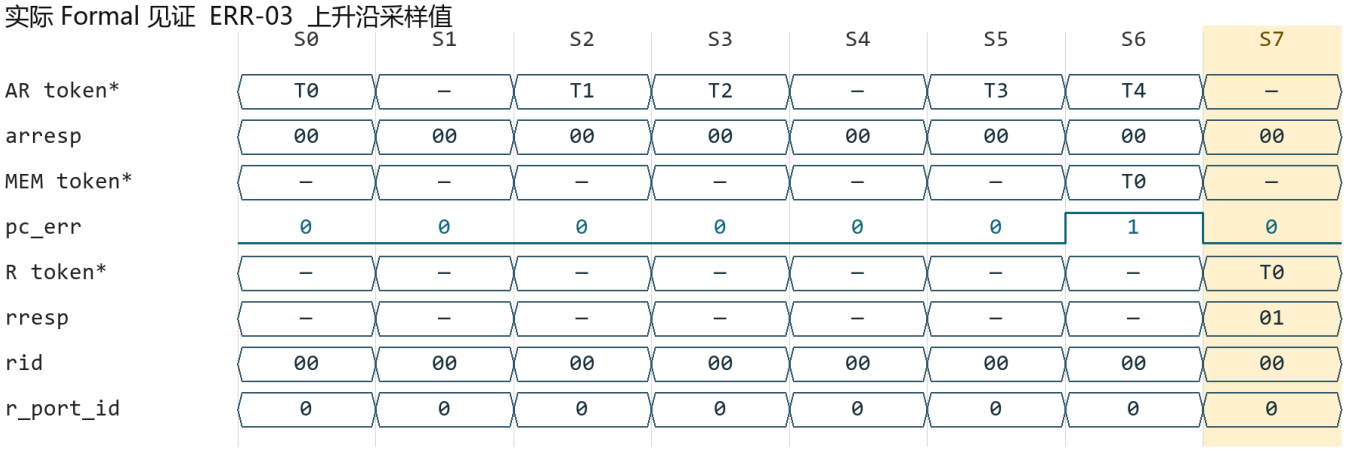
字段与错误组合 | 合法业务

前提与动作 arresp=00，pc_err=1；捕获后 R。

预期 rresp=01；保留返回数据；不丢弃命令。

要拦住的错误 错映射为 11 或丢 R。

实际 Formal 见证 ERR-03 上升沿采样值



* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ERR-03 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=0/2/3，ready=111，pending_cnt=4。

[REF_MODEL] R 错误状态来自预期事务；地址越界优先于奇偶校验异常。

```
A_R_RESP: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rresp == expected_resp)
);
```

```
C_CC_ERR_03_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_err[rsp_e]==0 && m_pc[rsp_e]==1 && rresp==1
);
```

证据 build/formal/err/run/engine_0/trace8.vcd；日志 step=8，图示采样 S7。

关联属性 A_XAU_R_RESP_ORDER。

分支 1 个；本页已列出该场景 cover。

ERR 04 两个错误同时发生

字段与错误组合 | 合法业务

前提与动作 arresp=11，pc_err=1；正常完成并返回。

预期 rresp=11；地址越界优先于奇偶异常。

要拦住的错误 判断优先级相反。

实际 Formal 见证 ERR-04 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	—	T1	T2	—	T3	T4	—
arresp	11	00	00	00	00	00	00	00
MEM token*	—	—	—	—	—	—	T0	—
pc_err	0	0	0	0	0	0	1	0
R token*	—	—	—	—	—	—	—	T0
rresp	—	—	—	—	—	—	—	11
rid	00	00	00	00	00	00	00	00
r_port_id	0	0	0	0	0	0	0	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ERR-04 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=0/2/3，ready=111，pending_cnt=4。

[REF_MODEL] R 错误状态来自预期事务；地址越界优先于奇偶校验异常。

```
A_R_RESP: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rresp == expected_resp)
);
```

```
C_CC_ERR_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_err[rsp_e]==3 && m_pc[rsp_e]==1 && rresp==3
);
```

证据 build/formal/err/run/engine_0/trace6.vcd；日志 step=8，图示采样 S7。

关联属性 C_RETURN_ERROR_PRIORITY。

分支 1 个；本页已列出该场景 cover。

ERR 05 未定义输入 arresp

字段与错误组合 | 环境违约

前提与动作 arvalid=1、arresp=01 或 10；独立负向输入。

预期 违反 E04；正常环境只允许 00/11；不替 DUT 指定新业务含义。

要拦住的错误 把非法码默认为正常并计为覆盖完成。

环境违规见证 ERR-05 上升沿采样值

	S0
AR token*	T0
arresp	10
MEM token*	–
pc_err	0
R token*	–
rresp	–
rid	00
r_port_id	0

* 为顶层事件推导；— 表示本图不关心；蓄色列为关注采样点

图 ERR-05 实际负向输入轨迹，监测断言按预期失败。

关键采样 S0 A/I/M/R=1000，N=0/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 有效请求携带未定义的 arresp；本项改为监测断言，预期检出。

```
V_ERR_05: assert property (
    @(posedge clk) disable iff (!rst_n)
    arvalid |-> (arresp==0 || arresp==3)
);
```

```
N_ERR_05: cover property (
    @(posedge clk) disable iff (!rst_n)
    arvalid && (arresp==1 || arresp==2)
);
```

证据 build/negative/err_05_cover/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 E04_ARRESP。

边界 关闭 E04 后，用 N_ERR_05 cover / V_ERR_05 assert 检查环境违规见证。

ERR 06 错误命令被正常命令夹住

字段与错误组合 | 合法业务

前提与动作 三条同源命令为正常/错误/正常；改变发射完成顺序。

预期 R 仍按接收顺序，错误只附着于中间命令。

要拦住的错误 使用全局错误寄存器污染前后项。

实际 Formal 见证 ERR-06 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
AR token*	T2	–	–	–	–	T3	T4	–
arresp	00	00	00	00	00	00	00	00
MEM token*	–	–	–	–	T0	T1	T2	–
pc_err	0	0	0	0	0	1	0	0
R token*	–	–	–	–	–	T0	T1	T2
rresp	–	–	–	–	–	00	01	00
rid	00	00	00	00	00	00	00	00
r_port_id	0	0	0	0	0	0	0	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ERR-06 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0001，N=3/0/0，ready=111，pending_cnt=2。

[REF_MODEL] 有效 R 的 128 位数据必须来自同一个队头事务的 MEM 捕获记录。

```
A_XAU_R_DATA_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rdata == m_data[oldest]))
);
```

```
C_CC_ERR_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_error_sandwich_phase==2 && h_response_matches && winner==0 && rresp==0
);
```

证据 build/formal/err/run/engine_0/trace10.vcd；日志 step=10，图示采样 S9。

关联属性 A_XAU_R_RESP_ORDER。

分支 1 个；本页已列出该场景 cover。

ERR 07 低地址位与 Bank/行地址分离

字段与错误组合 | 合法业务

前提与动作 命令只改变 araddr[2:0]；保持其余字段，分别完成。

预期 req 和 arb_addr 不变，raddr 按原三位变化。

要拦住的错误 将 raddr 错取 arb_addr 低三位。

实际 Formal 见证 ERR-07 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
AR token*	T1	–	T2	T3	T4	T5	T6	–
araddr	0000C	00000	00000	00000	00000	00000	00000	00000
req	00000010	00000000	00000010	00000000	00000001	00000001	00000001	00000001
arb_addr	000	–	000	–	000	000	000	000
R token*	–	–	–	–	–	–	T0	T1
raddr	0	0	0	0	0	0	0	4
rid	00	00	00	00	00	00	00	00
r_port_id	0	0	0	0	0	0	0	0

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ERR-07 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=0001，N=0/2/4，ready=111，pending_cnt=5。

[REF_MODEL] req 有效时检查完整下游地址 araddr[17:6]；无效时不检查负载。

```
A_ARB_ADDR: assert property (
    @(posedge clk) disable iff (!rst_n)
    (req != 8'b0)
    |-> (arb_addr == expected_addr)
);
```

```
C_CC_ERR_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_r && r_fire && winner==h_prev_rsrc &&
        m_addr[rsp_e][17:3]==h_prev_raddr_full[17:3] &&
        raddr!=h_prev_raddr_full[2:0] && h_response_matches
);
```

证据 build/formal/err/run/engine_0/trace9.vcd；日志 step=10，图示采样 S9。

关联属性 A_XAU_R_ADDR_ORDER。

分支 1 个；本页已列出该场景 cover。

ERR 08 port_id 直接透传

字段与错误组合 | 合法业务

前提与动作 同源或跨源使用不同 port_id，包括 0 和 31；乱序完成且 RR 交错。

预期 r_port_id 来自该命令 AR，不由 src、Bank 或 ID 重新编码。

要拦住的错误 混淆 src 与 port_id。

实际 Formal 见证 ERR-08 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	–	T1	T2	–	T3	T4	–
ar_port_id	0	0	0	0	0	0	0	0
arid	00	00	00	00	00	00	00	00
req token*	–	–	T0	–	T1	T1	T1	T1
MEM token*	–	–	–	–	–	–	T0	–
R token*	–	–	–	–	–	–	–	T0
r_port_id	0	0	0	0	0	0	0	0
rid	00	00	00	00	00	00	00	00

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 ERR-08 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=0001，N=0/2/3，ready=111，pending_cnt=4。

[REF_MODEL] R 握手时，ID、port_id、地址低三位必须属于同一预期事务。

```
A_R_METADATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> ((rid == m_id[rsp_e] && r_port_id == m_port[rsp_e] && raddr ==
        m_addr[rsp_e][2:0]))
);
```

```
C_CC_ERR_08_P0: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && r_port_id==0
);
```

证据 build/formal/err/run/engine_0/trace1.vcd；日志 step=8，图示采样 S7。

关联属性 A_XAU_R_PORT_ORDER。

分支 5 个；其余分支完整代码见附录 B。

CON 01 四事件全部发生

同拍事件与冲突 | 合法业务

前提与动作 新 A、当前 I、四拍前的 M、已完成项 R 均合法；四事件同拍；A 可复用 R 槽，其他事务互异。

预期 N_next=N，P_out=P；M 缓存、R 输出、I 日历与 A 分配全部完成。

要拦住的错误 全局 if/else 导致只处理一个事件。

实际 Formal 见证 CON-01 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	–	T5	T6
arready	111	111	111	111	111	111	111	111
req token*	–	–	T0	T1	–	T2	T2	T2
I	0	0	1	1	0	0	0	1
MEM token*	–	–	–	–	–	–	T0	T1
R token*	–	–	–	–	–	–	–	T0
N0/N1/N2*	0/0/0	1/0/0	1/0/1	2/0/1	2/0/2	2/0/3	2/0/3	2/1/3
pending_cnt	1	2	2	2	3	3	4	4
A/I/M/R*	1000	1000	1100	1100	1000	0000	1010	1111

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CON-01 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1111，N=2/1/3，ready=111，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CON_01_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && issue_fire && mem_capture_due && r_fire && h_response_matches
);
```

证据 build/formal/con/run/engine_0/trace2.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_PENDING, A_R_DATA, C_ALL_FOUR_EVENTS。

分支 1 个；本页已列出该场景 cover。

CON 02 满表四事件同拍

同拍事件与冲突 | 合法业务

前提与动作 N=9，存在一个待发、一个到期、一个已完成头；R 释放槽，A 选择合法来源，同时 I/M。

预期 同拍复用 R 槽；不得借用 I 或 M 槽；N 保持 9。

要拦住的错误 空闲掩码含 issued/done 项而覆盖旧事务。

实际 Formal 见证 CON-02 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
AR token*	T2	T3	T4	T5	T6	T7	T8	T9
arready	111	111	111	111	111	111	111	111
req token*	–	T0	T2	T3	T1	T1	T1	T1
I	0	1	1	1	0	0	0	1
MEM token*	–	–	–	–	–	T0	T2	T3
R token*	–	–	–	–	–	–	–	T0
N0/N1/N2*	0/2/0	0/3/0	0/3/1	1/3/1	2/3/1	3/3/1	4/3/1	4/3/2
pending_cnt	3	3	3	3	4	5	6	6
A/I/M/R*	1000	1100	1100	1100	1000	1010	1010	1111

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CON-02 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=1111，N=4/3/2，ready=111，pending_cnt=6。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CON_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && ar_fire && issue_fire && mem_capture_due && r_fire && alloc_e==rsp_e
        && mem_capture_slot!=rsp_e && m_attempt_e!=rsp_e && h_response_matches
);
```

证据 build/formal/con/run/engine_0/trace4.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_DATA, C_ALL_FOUR_EVENTS, C_FULL_REUSE。

分支 1 个；本页已列出该场景 cover。

CON 03 零事件保持

同拍事件与冲突 | 合法业务

前提与动作 无 A/I/M/R；可有失败 req；等待多拍。

预期 占用不变；数据与年龄保持；失败项可重选；日历仍按拍移位。

要拦住的错误 空拍停止整条流水或意外清零。

实际 Formal 见证 CON-03 上升沿采样值

	S0	S1	S2	S3	S4
AR token*	T0	–	–	–	–
arready	111	111	111	111	111
req token*	–	–	T0	T0	T0
I	0	0	0	0	0
MEM token*	–	–	–	–	–
R token*	–	–	–	–	–
N0/N1/N2*	0/0/0	0/0/1	0/0/1	0/0/1	0/0/1
pending_cnt	1	1	1	1	1
A/I/M/R*	1000	0000	0000	0000	0000

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CON-03 实际正向见证；黄色为关键采样点。

关键采样 S4 A/I/M/R=0000，N=0/0/1，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CON_03_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_quiet_run>=3 && !ar_fire && !issue_fire && !mem_capture_due && !r_fire &&
        used>0
);
```

证据 build/formal/con/run/engine_0/trace1.vcd；日志 step=5，图示采样 S4。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_PENDING, A_REQ, A_R_DATA。

分支 1 个；本页已列出该场景 cover。

CON 04 同拍新 AR 不能立刻 I/M/R

同拍事件与冲突 | 合法业务

前提与动作 当前只有一个全新接收事务，无旧事务；尝试把多个事件解释为同一命令。

预期 A→I 至少三阶段，I→M 四拍，M→R 至少下一拍；事件组合需对应历史事务。

要拦住的错误 只看事件数，不查事务身份与历史。

实际 Formal 见证 CON-04 上升沿采样值

	S0
AR token*	T0
arready	111
req token*	–
I	0
MEM token*	–
R token*	–
N0/N1/N2*	0/0/0
pending_cnt	1
A/I/M/R*	1000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CON-04 实际正向见证；黄色为关键采样点。

关键采样 S0 A/I/M/R=1000，N=0/0/0，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

```
C_CC_CON_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==0 && ar_fire && !issue_fire && !mem_capture_due && !r_fire
);
```

证据 build/formal/con/run/engine_0/trace0.vcd；日志 step=1，图示采样 S0。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_REQ, A_XAU_R_NO_EARLY_BYPASS。

分支 1 个；本页已列出该场景 cover。

CON 05 物理槽冲突注错

同拍事件与冲突 | 内部注错

前提与动作 内部把同拍 I/M、M/R、A/M 等指向同一存活事务槽；观察双重写入或错误响应。

预期 合法历史中这些别名不成立；应由内部断言/注错及端口后果检出。

要拦住的错误 将不可能的内部事件强行作为输入 assume。

注错或观测边界示意 未作为已命中的 Formal 波形

Cn 同拍事件		沿后内部注错		Cn+k 外部后果
A / I / M / R 可共存	→	错误写到同一物理槽	→	新事务被旧事务覆盖
仅 A 与 R 可复用退役槽		物理地址不在顶层接口		最终由有效 R 负载检查

本图描述检查责任或拟注错路径，不声称该非法内部状态已经被注入。

图 CON-05 未执行注错或不可直接观察的边界示意。

检查边界 端口数据/顺序断言检查有外部后果的槽别名；无外部后果的内部冲突需白盒断言。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_DATA, A_XAU_R_HAS_PENDING。

CON 06 计数不变不等于无动作

同拍事件与冲突 | 合法业务

前提与动作 同拍 A/R=1，可能 I/M=1；连续多拍总 N 恒定。

预期 字段、队头、年龄、数据和 RR 均可能变化；需逐事务检查。

要拦住的错误 仅按总数守恒就认为全部正确。

实际 Formal 见证 CON-06 上升沿采样值

	S2	S3	S4	S5	S6	S7	S8	S9
AR token*	T2	–	–	–	T3	T4	T5	T6
arready	111	111	111	111	111	111	111	111
req token*	T0	T1	T2	–	–	–	–	T3
I	1	1	1	0	0	0	0	0
MEM token*	–	–	–	–	T0	T1	T2	–
R token*	–	–	–	–	–	T0	T1	T2
N0/N1/N2*	1/0/1	1/1/1	1/1/1	1/1/1	1/1/1	2/1/1	2/1/1	2/1/1
pending_cnt	2	1	0	0	1	2	3	4
A/I/M/R*	1100	0100	0100	0000	1010	1011	1011	1001

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 CON-06 实际正向见证；黄色为关键采样点。

关键采样 S9 A/I/M/R=1001，N=2/1/1，ready=111，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    aready==h_ready
);
```

```
C_CC_CON_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_ar_r_run>=2 && ar_fire && r_fire && h_response_matches
);
```

证据 build/formal/con/run/engine_0/trace3.vcd；日志 step=10，图示采样 S9。

关联属性 A_CC_BUDGET, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_RR_CHOICE, A_R_DATA, A_R_METADATA。

分支 1 个；本页已列出该场景 cover。

BIZ 01 单源持续满速流

持续业务与边界 | 合法业务

前提与动作 同源 ready 时每拍提交；Bank/grant 条件允许最早 I；R ready 持续 1；最短 C1 AR→C3 I→C7 M→C8 R，持续填充和回收。

预期 首次返回后可同拍接收并稳态 1 AR/拍；这是条件吞吐，不是无条件保证。

要拦住的错误 7 项容量边界插空泡；把仲裁等待算成 DUT 吞吐错误。

实际 Formal 见证 BIZ-01 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	–	–	–	T1	–	–	T2
arready	111	111	111	111	111	111	111	111
req token*	–	–	T0	–	–	–	T1	–
I	0	0	1	0	0	0	1	0
MEM token*	–	–	–	–	–	–	T0	–
R token*	–	–	–	–	–	–	–	T0
N0/N1/N2*	0/0/0	1/0/0	1/0/0	1/0/0	1/0/0	1/0/1	1/0/1	1/0/1
pending_cnt	1	1	0	0	1	1	0	1
A/I/M/R*	1000	0000	0100	0000	1000	0000	0110	1001

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-01 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1001，N=1/0/1，ready=111，pending_cnt=1。

[NO_REF_MODEL_HISTORY] 空来源按 1 项保留计算，三源总预算不得超过 9。

```
A_CC_BUDGET: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_budget<=9
);
```

```
C_CC_BIZ_01_FIRST_R: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_single_start[6] && h_response_matches
);
```

证据 build/formal_retry/biz/run/engine_0/trace1.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_PENDING, A_READY, A_R_DATA。

分支 2 个；其余分支完整代码见附录 B。

BIZ 02 三源混合长流

持续业务与边界 | 合法业务

前提与动作 上游选择不同 src，维持合法握手；随机 Bank/half、ID 重复、R ready mask 和 grant 失败。

预期 长期 AR-R 守恒，各源不超过 7，字段与数据不串单。

要拦住的错误 仅短序列正确，年龄复用后失效。

实际 Formal 见证 BIZ-02 上升沿采样值

	S9	S10	S11	S12	S13	S14	S15	S16
AR token*	T9	T10	T11	T12	T13	T14	T15	T16
arready	111	111	111	111	111	111	111	111
req token*	–	T7	–	T8	T8	T8	T8	T8
I	0	1	0	0	0	0	0	0
MEM token*	T3	T4	T5	T6	–	T7	–	–
R token*	T2	T3	–	T4	T6	T5	T7	–
N0/N1/N2*	1/2/4	1/3/3	1/4/2	2/4/2	2/4/2	2/3/3	3/3/2	4/2/2
pending_cnt	3	3	4	5	6	7	8	9
A/I/M/R*	1011	1111	1010	1011	1001	1011	1001	1000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-02 实际正向见证；黄色为关键采样点。

关键采样 S16 A/I/M/R=1000，N=4/2/2，ready=111，pending_cnt=9。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_BIZ_02_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total_a>=16 && h_total_r>=8 && h_ar_sources==7 && h_seen_reject &&
        h_seen_partial_ready && ar_fire && h_duplicate_id
);
```

证据 build/formal_retry/biz/run/engine_0/trace10.vcd；日志 step=17，图示采样 S16。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_DATA_ORDER。

分支 1 个；本页已列出该场景 cover。

BIZ 03 同 Bank 热点竞争

持续业务与边界 | 合法业务

前提与动作 许多命令指向同一 Bank 或 half；status 忙闲变化，失败多拍，夹杂其他 Bank 命令。

预期 只在窗口内绕过忙项；成功规则与年龄不变；无吞吐下限承诺。

要拦住的错误 误认为有 9 项缓存就能消除热点限制。

实际 Formal 见证 BIZ-03 上升沿采样值

	S0	S1	S2	S3	S4
C2 chosen*	–	T0	T0	T0	T0
req	00000000	00000000	10000000	10000000	10000000
arb_addr	–	–	000	000	000
status B7[1:0]	00	00	00	00	00
ar_rd_gnt	00000000	00000000	00000000	00000000	00000000
rw_rd_gnt	00000000	00000000	00000000	00000000	00000000
I	0	0	0	0	0
pending_cnt	1	2	2	3	3

* 为证据事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-03 实际正向见证；黄色为关键采样点。

关键采样 S4 A/I/M/R=0000，N=2/0/1，ready=111，pending_cnt=3。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_BIZ_03_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting>=3 && h_hotspot && h_reject_run>=2 && select_valid
);
```

证据 build/formal_retry/biz/run/engine_0/trace0.vcd；日志 step=5，图示采样 S4。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, C_RETRY_NEXT_CYCLE。

分支 1 个；本页已列出该场景 cover。

BIZ 04 前五永久忙

持续业务与边界 | 合法业务

前提与动作 前五待发项始终忙，第六项及以后空闲；长时间无成功发射。

预期 可以永久不发射；容量耗尽后阻塞合法；无 bank 公平假设不能证明排空。

要拦住的错误 强加所有请求最终发射的无条件 assert。

实际 Formal 见证 BIZ-04 上升沿采样值

	S5	S6	S7	S8	S9	S10	S11	S12
AR token*	T5	–	–	–	T6	–	T7	–
pending_cnt	6	6	6	6	7	7	8	8
C2 window*	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4	0/1/2/3/4
idle rank 1→5*	00000	00000	00000	00000	00000	00000	00000	00000
6th outside*	–	T5 idle	T5 idle	T5 idle	T5 idle	T5 idle	T5 idle	T5 idle
C2 chosen*	–	–	–	–	–	–	–	–
req token*	–	–	–	–	–	–	–	–
req	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
I	0	0	0	0	0	0	0	0

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-04 实际正向见证；黄色为关键采样点。

关键采样 S12 A/I/M/R=0000，N=4/0/4，ready=010，pending_cnt=8。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_BIZ_04_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    blocked_six && blocked_streak>=6 && req==0
);
```

证据 build/formal_retry/biz/run/engine_0/trace6.vcd；日志 step=13，图示采样 S12。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, C_BLOCKED_SIX。

分支 1 个；本页已列出该场景 cover。

本图仅见证有限连续等待；没有输入公平性条件，不将它解释为永久行为的完整证明。

BIZ 05 仲裁持续拒绝

持续业务与边界 | 合法业务

前提与动作 可用候选每次 req，但双 grant 不成功；任意长失败序列。

预期 不丢命令、不减 pending，可连续重试或重选；不保证最终成功。

要拦住的错误 把无限失败误判为 DUT 死锁。

实际 Formal 见证 BIZ-05 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	–	–	–	T1	–	–	–	T2
arready	111	111	111	111	111	111	111	111
req token*	–	T0	T0	T0	T0	T0	T0	T0
I	0	0	0	0	0	0	0	0
MEM token*	–	–	–	–	–	–	–	–
R token*	–	–	–	–	–	–	–	–
N0/N1/N2*	0/0/1	0/0/1	0/0/1	0/0/1	0/0/2	0/0/2	0/0/2	0/0/2
pending_cnt	1	1	1	2	2	2	2	3
A/I/M/R*	0000	0000	0000	1000	0000	0000	0000	1000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-05 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=1000，N=0/0/2，ready=111，pending_cnt=3。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_BIZ_05_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_reject_run>=6 && req!=0 && !issue_fire
);
```

证据 build/formal_retry/biz/run/engine_0/trace3.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_PENDING。

分支 1 个；本页已列出该场景 cover。

本图仅见证有限连续等待；没有输入公平性条件，不将它解释为永久行为的完整证明。

BIZ 06 某源永不 R ready

持续业务与边界 | 合法业务

前提与动作 该源占据若干已完成项，其他源继续；rready[该源]=0 长期不变。

预期 该源项不释放；其他源在容量和窗口许可范围推进；整体不保证排空。

要拦住的错误 无条件要求所有 entry 最终清空。

实际 Formal 见证 BIZ-06 上升沿采样值

	S6	S7	S8	S9	S10	S11	S12	S13
AR token*	T5	T6	T7	T8	T9	–	T10	–
arready	111	111	111	111	111	111	111	111
req token*	T2	T4	T3	–	–	–	T6	T6
I	1	1	1	0	0	0	0	0
MEM token*	T0	T1	–	–	T2	T4	T3	–
R token*	–	–	T1	–	–	T2	–	T3
N0/N1/N2*	2/0/3	2/1/3	2/2/3	2/2/3	2/2/4	2/2/5	2/2/4	2/2/5
pending_cnt	3	3	3	4	5	5	6	6
A/I/M/R*	1110	1110	1101	1000	1010	0011	1010	0001

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-06 实际正向见证；黄色为关键采样点。

关键采样 S13 A/I/M/R=0001，N=2/2/5，ready=111，pending_cnt=6。

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

```
C_CC_BIZ_06_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_notready_run[0]>=6 && !rready[0] && (rvalid&6)!=0 && h_response_matches
);
```

证据 build/formal_retry/biz/run/engine_0/trace8.vcd；日志 step=14，图示采样 S13。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_R_READY。

分支 1 个；本页已列出该场景 cover。

本图仅见证有限连续等待；没有输入公平性条件，不将它解释为永久行为的完整证明。

BIZ 07 源切换不公平

持续业务与边界 | 集成边界

前提与动作 三路有业务，但上游长期不选某一路；仅观察共享 AR 与 arready。

预期 可证明该路首项容量许可；无法证明未呈现请求的服务时间。

要拦住的错误 将 ready 保证提升为上游公平保证。

实际 Formal 见证 BIZ-07 上升沿采样值

	S0	S1	S2	S3	S4	S5	S6	S7
AR token*	T0	T1	T2	T3	T4	T5	T6	T7
arready	111	111	111	111	111	111	111	110
req token*	–	–	–	T0	T1	T2	T4	–
I	0	0	0	1	1	1	1	0
MEM token*	–	–	–	–	–	–	–	T0
R token*	–	–	–	–	–	–	–	–
N0/N1/N2*	0/0/0	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0
pending_cnt	1	2	3	3	3	3	3	4
A/I/M/R*	1000	1000	1000	1100	1100	1100	1100	1010

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-07 实际正向见证；黄色为关键采样点。

关键采样 S7 A/I/M/R=1010，N=7/0/0，ready=110，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 空来源按 1 项保留计算，三源总预算不得超过 9。

```
A_CC_BUDGET: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_budget<=9
);
```

```
C_CC_BIZ_07_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total_a>=7 && h_ar_sources==1 && arready[1] && arready[2]
);
```

证据 build/formal_retry/biz/run/engine_0/trace2.vcd；日志 step=8，图示采样 S7。

关联属性 A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY。

分支 1 个；本页已列出该场景 cover。

边界 有限的不公平选择；未呈现给共享 AR 的请求无法观察。

BIZ 08 停止注入后排空

持续业务与边界 | 集成边界

前提与动作 AR 停止；所有待发目标最终可用且获 grant；各源最终 ready；继续完整 MEM 返回。

预期 在这些服务条件下应逐项排空；若需固定周期界，需给出各条件的有界版本。

要拦住的错误 未给服务上界却断言固定超时。

实际 Formal 见证 BIZ-08 上升沿采样值

	S7	S8	S9	S10	S11	S12	S13	S14
AR token*	–	–	–	–	–	–	–	–
arready	111	111	111	111	111	111	111	111
req token*	T5	–	–	–	–	–	–	–
I	1	0	0	0	0	0	0	0
MEM token*	T1	T2	T3	T4	T5	–	–	–
R token*	T0	T1	T2	T3	T4	T5	–	–
N0/N1/N2*	4/0/2	4/0/1	3/0/1	2/0/1	1/0/1	0/0/1	0/0/0	0/0/0
pending_cnt	0	0	0	0	0	0	0	0
A/I/M/R*	0111	0011	0011	0011	0011	0001	0000	0000

* 为顶层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-08 实际正向见证；黄色为关键采样点。

关键采样 S14 A/I/M/R=0000，N=0/0/0，ready=111，pending_cnt=0。

[NO_REF_MODEL_HISTORY] 空来源按 1 项保留计算，三源总预算不得超过 9。

```
A_CC_BUDGET: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_budget<=9
);
```

```
C_CC_BIZ_08_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_was_drained && used==0 && h_total_r>=6 && !arvalid
);
```

证据 build/formal_retry/biz/run/engine_0/trace9.vcd；日志 step=15，图示采样 S14。

关联属性 A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_R_DATA, A_XAU_R_HAS_PENDING。

分支 1 个；本页已列出该场景 cover。

边界 存在排空轨迹；不添加公平性 assume，不声明所有轨迹有界排空。

BIZ 09 错序返回叠加容量限制

持续业务与边界 | 合法业务

前提与动作 同源年轻项先完成；老项阻塞；占用达到 7；另两路各一项有业务。

预期 老项未 R 前该源不再接收；另两源可各保留一项；年轻完成不归还额度。

要拦住的错误 done 代替 R 释放造成配额和保序同时失效。

实际 Formal 见证 BIZ-09 上升沿采样值

	S1	S2	S3	S4	S5	S6	S7	S8
AR token*	T1	T2	T3	T4	T5	T6	–	T7
arready	111	111	111	111	111	111	110	110
req token*	–	T0	T1	–	T2	T3	T5	T4
I	0	0	1	0	1	1	1	0
MEM token*	–	–	–	–	–	–	T1	–
R token*	–	–	–	–	–	–	–	–
N0/N1/N2*	1/0/0	2/0/0	3/0/0	4/0/0	5/0/0	6/0/0	7/0/0	7/0/0
pending_cnt	2	3	3	4	4	4	3	4
A/I/M/R*	1000	1000	1100	1000	1100	1100	0110	1000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-09 实际正向见证；黄色为关键采样点。

关键采样 S8 A/I/M/R=1000，N=7/0/0，ready=110，pending_cnt=4。

[NO_REF_MODEL_HISTORY] 空来源按 1 项保留计算，三源总预算不得超过 9。

```
A_CC_BUDGET: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_budget<=9
);
```

```
C_CC_BIZ_09_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0]==7 && !m_done[head[0]] && h_other_done && !r_fire && arready[1] &&
        arready[2] && ar_fire && src!=0
);
```

证据 build/formal_retry/biz/run/engine_0/trace4.vcd；日志 step=9，图示采样 S8。

关联属性 A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_READY, A_XAU_R_HEAD_COMPLETE。

分支 1 个；本页已列出该场景 cover。

BIZ 10 长时间全空后重新启动

持续业务与边界 | 合法业务

前提与动作 事务全部 R，空闲许多拍，MEM 总线仍变化；再发送新命令。

预期 仅新事务产生 req/R，RR 保持上次实际响应后的指针。

要拦住的错误 空闲自动把 RR 置零或残留日历触发幽灵返回。

实际 Formal 见证 BIZ-10 上升沿采样值

	S6	S7	S8	S9	S10	S11	S12	S13
AR token*	–	–	–	–	–	–	–	T1
arready	111	111	111	111	111	111	111	111
req token*	–	–	–	–	–	–	–	–
I	0	0	0	0	0	0	0	0
MEM token*	T0	–	–	–	–	–	–	–
R token*	–	T0	–	–	–	–	–	–
N0/N1/N2*	0/0/1	0/0/1	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
pending_cnt	0	0	0	0	0	0	0	1
A/I/M/R*	0010	0001	0000	0000	0000	0000	0000	1000

* 为低层事件推导；— 表示本图不关心；黄色列为关注采样点

图 BIZ-10 实际正向见证；黄色为关键采样点。

关键采样 S13 A/I/M/R=1000，N=0/0/0，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_BIZ_10_MAIN: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==0 && h_was_drained && h_empty_run>=5 && ar_fire && mem_rdata!=h_prev_mem
);
```

证据 build/formal_retry/biz/run/engine_0/trace7.vcd；日志 step=14，图示采样 S13。

关联属性 A_CC_BUDGET, A_CC_READY, A_CC_R_NONEMPTY, A_CC_SRC_LIMIT, A_CC_TOTAL, A_RR_CHOICE, A_R_DATA。

分支 1 个；本页已列出该场景 cover。

ENV 06 数组连接方向

接入与验证边界 | 集成边界

前提与动作 上层与模块都使用 bank-major [1:0] status [7:0]；对 16 个独立 bank/half 定向置位。

预期 一一对应 status[bank][half]；适配器若必须展开需显式定义映射。

要拦住的错误 用 [1:0][7:0] packed 直接接导致转置。

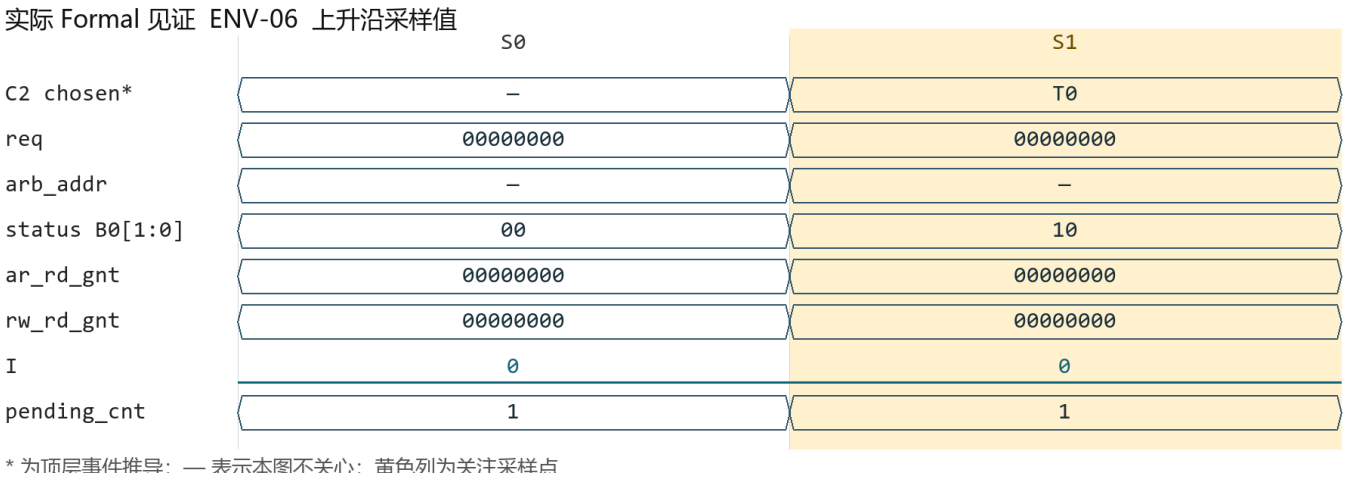


图 ENV-06 实际正向见证；黄色为关键采样点。

关键采样 S1 A/I/M/R=0000，N=1/0/0，ready=111，pending_cnt=1。

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

```
C_CC_ENV_06_B0H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==0 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<0))
);
```

证据 build/formal/env/run/engine_0/trace0.vcd；日志 step=2，图示采样 S1。

分支 16 个；其余分支完整代码见附录 B。

边界 逐位解码见证；实际 SoC 接线仍需在该 SoC 边界连接检查器。

ENV 07 仅端口可观察的证明边界

接入与验证边界 | 集成边界

前提与动作 仅连接顶层接口运行 SVA；检查数据、计数、顺序与后续响应。

预期 可查外部行为；不能单凭参考模型自检证明物理 entry 恰有 9 项、OTB 位已清零。

要拦住的错误 将 M_* 模型性质当 RTL 物理结构证明。

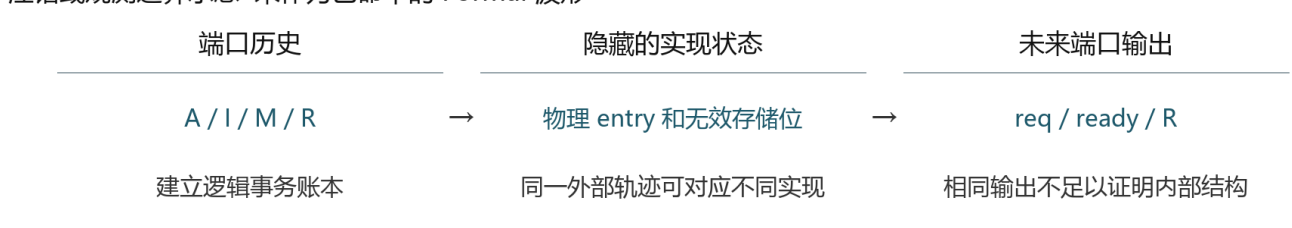


图 ENV-07 未执行注错或不可直接观察的边界示意。

检查边界 只使用顶层端口，不能证明物理存储数量、原始位清零或不可观察内部事件。

本项没有独立的当前端口 cover。上面的断言仅用于检查可观察后果，不能当作物理注错已发生的证据。

关联属性 M_QUOTA, M_RANGE, M_STATE。

ENV 08 环境约束不能假设 DUT 结果

接入与验证边界 | 集成边界

前提与动作 构建 formal harness；检查 E03/E04/E05/E06 及新增 assume。

预期 只约束合法输入关系；ready/req/R 正确性应 assert，不能 assume 掉设计错误。

要拦住的错误 assume rvalid 正确保序、假设 ready 公式等循环证明。

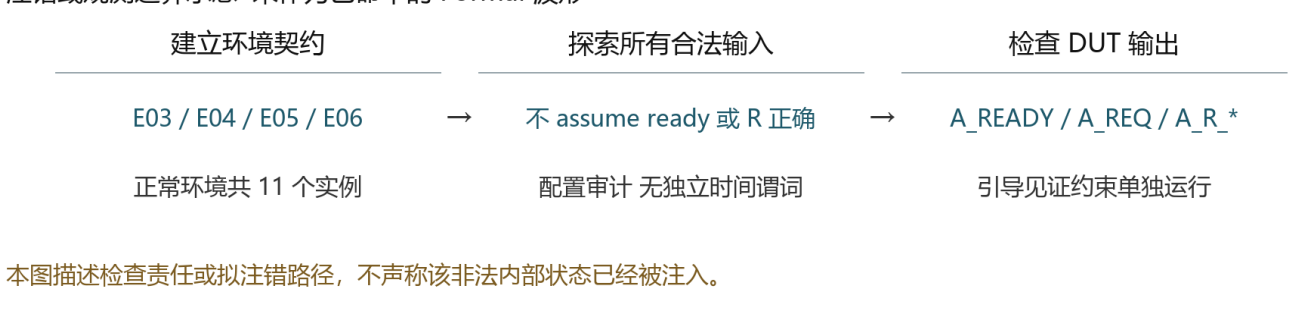


图 ENV-08 未执行注错或不可直接观察的边界示意。

检查边界 机械核对 assume 只有 E03/E04/E05/E06；环境关系不可替代 A_READY/A_REQ/A_R_*。

[REF_MODEL] aready 必须符合容量预留规则，并计入本拍 R 回收。

```
A_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    aready == expected_ready
);
```

本项以配置审计验收：正常环境只含四类 assume，不为审计结果伪造时序 cover。

关联属性 E03_SRC, E04_ARRESP, E05_ZERO_READY, E06_SUCCESS_BUSY。

附录 A 公共 assume 与 assert

以下为当前工程原生属性，标签和表达式保持不变，只调整换行。g、cs 表示来源 0 到 2，bank_env 表示 Bank 0 到 7；slots 中的 g 为逻辑槽编号 0 到 8，h 为另一个逻辑槽编号。作用域与辅助定义保留在工程源文件中。

正常环境的四类 assume

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 有效 AR 必须来自 XAU0、XAU1 或 XAU2；无效拍不约束 src。

```
E03_SRC: assume property (
    @(posedge clk) disable iff (!rst_n)
    (arvalid)
    |-> (src < 2'd3)
);
```

源码 cmdbuf_rd_port_sva.sv 第 93 行

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 有效 AR 的上游状态只允许正常 00 或地址越界 11。

```
E04_ARRESP: assume property (
    @(posedge clk) disable iff (!rst_n)
    (arvalid)
    |-> (arresp == 2'b00 || arresp == 2'b11)
);
```

源码 cmdbuf_rd_port_sva.sv 第 104 行

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 三个来源都不能接收时，上游不得发送有效 AR。

```
E05_ZERO_READY: assume property (
    @(posedge clk) disable iff (!rst_n)
    (arready == 3'b000)
    |-> (!arvalid)
);
```

源码 cmdbuf_rd_port_sva.sv 第 115 行

```
wire requested_half_busy = rd_mem_status[bank_env][arb_addr[0]];
```

[NO_REF_MODEL_ENV] [NO_REF_MODEL_PORTS] 本拍该 Bank 双 grant 成功时，被访问的奇偶半区本拍必须报告占用。

```
E06_SUCCESS_BUSY: assume property (
    @(posedge clk) disable iff (!rst_n)
    (req[bank_env] && ar_rd_gnt[bank_env] && rw_rd_gnt[bank_env])
    |-> (requested_half_busy)
);
```

源码 cmdbuf_rd_port_sva.sv 第 131 行

仅端口与端口历史的 DUT 断言

[NO_REF_MODEL_PORTS] 下游每拍最多请求一个 Bank；全零合法。

```
A_REQ_ONEHOT: assert property (
    @(posedge clk) disable iff (!rst_n)
    (req & (req-8'd1)) == 0
);
```

源码 cmdbuf_rd_port_sva.sv 第 159 行

[NO_REF_MODEL_PORTS] 共享 R 负载每拍最多发往一个 XAU；全零合法。

```
A_R_ONEHOT: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid & (rvalid-3'd1)) == 0
);
```

源码 cmdbuf_rd_port_sva.sv 第 169 行

[NO_REF_MODEL_PORTS] rvalid 只能在对应 rready 已经为 1 的通道拉高。

```
A_R_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid & ~rready) == 0
);
```

源码 cmdbuf_rd_port_sva.sv 第 179 行

[NO_REF_MODEL_PORTS] 待发命令数不能超过共享容量 9。

```
A_PENDING_RANGE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt <= 9
);
```

源码 cmdbuf_rd_port_sva.sv 第 189 行

[NO_REF_MODEL_HISTORY] 本拍计数 = 上拍计数 + 本拍 AR 握手 - 本拍成功发射。

```
A_PENDING_RECURRENCE: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (last_pending+{3'b0,ar_fire}-{3'b0,issue_fire})
);
```

源码 cmdbuf_rd_port_sva.sv 第 200 行

[NO_REF_MODEL_HISTORY] AR 累计接收减 R 累计返回：全局最多仍在途 9 条。

```
A_CC_TOTAL: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_total<=9
);
```

源码 cmdbuf_rd_corner_assertions.svh 第 6 行

[NO_REF_MODEL_HISTORY] 空来源按 1 项保留计算，三源总预算不得超过 9。

```
A_CC_BUDGET: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_budget<=9
);
```

源码 cmdbuf_rd_corner_assertions.svh 第 12 行

[NO_REF_MODEL_HISTORY] 三位 ready 必须等于按实际 AR/R 历史及本拍 R 计算的接收许可。

```
A_CC_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    arready==h_ready
);
```

源码 cmdbuf_rd_corner_assertions.svh 第 19 行

[NO_REF_MODEL_HISTORY] 任一 XAU 尚未返回的命令不能超过 7 条。

```
A_CC_SRC_LIMIT: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_occ[cs]<=7
);
```

源码 cmdbuf_rd_corner_assertions.svh 第 26 行

[NO_REF_MODEL_HISTORY] 只要该源 rvalid 有效，就必须有之前接受而尚未返回的命令。

```
A_CC_R_NONEMPTY: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid[cs] |-> (h_occ[cs]!=0)
);
```

源码 cmdbuf_rd_corner_assertions.svh 第 33 行

事务参考记录与单源保序断言

[REF_MODEL] aready 必须符合容量预留规则，并计入本拍 R 回收。

```
A_READY: assert property (
    @(posedge clk) disable iff (!rst_n)
    aready == expected_ready
);
```

源码 cmdbuf_rd_port_sva.sv 第 214 行

[REF_MODEL] 待发数含本拍新 AR，扣除本拍双 grant 成功；仲裁失败不扣除。

```
A_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    pending_cnt == (waiting+(ar_fire?1:0)-(issue_fire?1:0))
);
```

源码 cmdbuf_rd_port_sva.sv 第 225 行

[REF_MODEL] 当前 req 必须对应上一拍窗口内选出的最老可发命令。

```
A_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    req == expected_req
);
```

源码 cmdbuf_rd_port_sva.sv 第 236 行

[REF_MODEL] req 有效时检查完整下游地址 araddr[17:6]；无效时不检查负载。

```
A_ARB_ADDR: assert property (
    @(posedge clk) disable iff (!rst_n)
    (req != 8'b0)
    |-> (arb_addr == expected_addr)
);
```

源码 cmdbuf_rd_port_sva.sv 第 246 行

[REF_MODEL] 只在 ready 且最老队头已完成的来源间按 RR 选择。

```
A_RR_CHOICE: assert property (
    @(posedge clk) disable iff (!rst_n)
    rvalid == expected_rvalid
);
```

源码 cmdbuf_rd_port_sva.sv 第 258 行

[REF_MODEL] R 握手时，ID、port_id、地址低三位必须属于同一预期事务。

```
A_R_METADATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> ((rid == m_id[rsp_e] && r_port_id == m_port[rsp_e] && raddr ==
        m_addr[rsp_e][2:0]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 269 行

[REF_MODEL] R 错误状态来自预期事务；地址越界优先于奇偶校验异常。

```
A_R_RESP: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rresp == expected_resp)
);
```

源码 cmdbuf_rd_port_sva.sv 第 280 行

[REF_MODEL] R 数据必须等于预期事务在成功发射后第 4 拍捕获的数据。

```
A_R_DATA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> (rdata == m_data[rsp_e])
);
```

源码 cmdbuf_rd_port_sva.sv 第 291 行

[REF_MODEL] 该源持续可响应且已被另外两次响应越过时，本拍必须轮到它。

```
A_RR_BOUND: assert property (
    @(posedge clk) disable iff (!rst_n)
    (eligible[g] && bypass_count >= 2)
    |-> (rvalid[g])
);
```

源码 cmdbuf_rd_port_sva.sv 第 310 行

[REF_MODEL] 该 XAU 的 rvalid 有效时，必须存在已接受但尚未返回的最老事务。

```
A_XAU_R_HAS_PENDING: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> (oldest_valid)
);
```

源码 cmdbuf_rd_port_sva.sv 第 321 行

[REF_MODEL] 只允许响应已成功发射且已捕获 MEM 数据的 FIFO 队头。

```
A_XAU_R_HEAD_COMPLETE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && m_sent[oldest] && m_done[oldest]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 332 行

[REF_MODEL] 队头未完成时，年轻命令即使已完成，也不能让该源提前响应。

```
A_XAU_R_NO_EARLY_BYPASS: assert property (
    @(posedge clk) disable iff (!rst_n)
    (oldest_valid && !m_done[oldest])
    |-> (!rvalid[g])
);
```

源码 cmdbuf_rd_port_sva.sv 第 343 行

[REF_MODEL] 有效 R 的 raddr 必须等于该源接收 FIFO 队头的 araddr[2:0]。

```
A_XAU_R_ADDR_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && raddr == m_addr[oldest][2:0]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 354 行

[REF_MODEL] 有效 R 的 rid 必须属于队头；不按 ID 数值排序，也不要求 ID 唯一。

```
A_XAU_R_ID_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rid == m_id[oldest]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 365 行

[REF_MODEL] 有效 R 的 port_id 必须透传同一个队头命令的 port_id。

```
A_XAU_R_PORT_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && r_port_id == m_port[oldest]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 376 行

[REF_MODEL] 有效 R 的 128 位数据必须来自同一个队头事务的 MEM 捕获记录。

```
A_XAU_R_DATA_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rdata == m_data[oldest]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 387 行

[REF_MODEL] 有效 R 的错误状态必须由同一个队头事务的 arresp 和 pc_err 形成。

```
A_XAU_R_RESP_ORDER: assert property (
    @(posedge clk) disable iff (!rst_n)
    (rvalid[g])
    |-> ((oldest_valid && rresp == oldest_resp))
);
```

源码 cmdbuf_rd_port_sva.sv 第 398 行

[REF_MODEL] 上一拍 C2 有选择，本拍必须按该选择发 req 和地址。

```
A_CC_C2_TO_C3: assert property (
    @(posedge clk) disable iff (!rst_n)
    h_selected_prev |-> (req==h_selected_req && arb_addr==h_selected_addr)
);
```

源码 cmdbuf_rd_corner_assertions.svh 第 43 行

[REF_MODEL] 上一拍没有 C2 选择，本拍不得凭空发起请求。

```
A_CC_NO_C2_NO_REQ: assert property (
    @(posedge clk) disable iff (!rst_n)
    (!h_selected_prev) |-> (req==0)
);
```

源码 cmdbuf_rd_corner_assertions.svh 第 49 行

模型自洽检查

这些 M_* 属性检查验证记录是否自洽，不作为 DUT 物理 entry 数量、存储清零或内部写冲突的证明。

[MODEL_SELF_CHECK] 独立接收 FIFO 的长度必须与事务表该源占用数一致，且不超过 7。

```
M_XAU_FIFO_COUNT: assert property (
    @(posedge clk) disable iff (!rst_n)
    order_count == occ[g] && order_count <= 7
);
```

源码 cmdbuf_rd_port_sva.sv 第 410 行

[MODEL_SELF_CHECK] 独立 FIFO 队头必须与事务表按年龄计算的最老事务一致。

```
M_XAU_FIFO_HEAD: assert property (
    @(posedge clk) disable iff (!rst_n)
    (order_count != 0) == head_valid[g] &&
    (order_count == 0 || (oldest_valid &&
    oldest == head[g]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 420 行

[MODEL_SELF_CHECK] 模型接收 AR 时必须找到合法逻辑槽；不检查 DUT 物理分配编号。

```
M_ALLOC: assert property (
    @(posedge clk) disable iff (!rst_n)
    (ar_fire)
    |-> ((free_found && alloc_e<9))
);
```

源码 cmdbuf_rd_port_sva.sv 第 492 行

[MODEL_SELF_CHECK] 实际 R 握手必须对应模型的合法、已发射、已完成事务。

```
M_RETIRE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (r_fire)
    |-> ((model_retire && rsp_e<9 && m_live[rsp_e] && m_sent[rsp_e] &&
        m_done[rsp_e]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 503 行

[MODEL_SELF_CHECK] 实际发射成功必须对应模型当前请求中的未发射事务。

```
M_SUCCESS: assert property (
    @(posedge clk) disable iff (!rst_n)
    (issue_fire)
    |-> ((m_attempt && m_attempt_e<9 && m_live[m_attempt_e] &&
        !m_sent[m_attempt_e]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 514 行

[MODEL_SELF_CHECK] 返回日历到期必须对应仍在途且尚未捕获的事务。

```
M_DUE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (mem_capture_due)
    |-> ((mem_capture_slot<9 &&
          m_live[mem_capture_slot] &&
          m_sent[mem_capture_slot] &&
          !m_done[mem_capture_slot]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 525 行

[MODEL_SELF_CHECK] 本拍刚捕获 MEM 的事务不得在同拍 R 返回，最快下一拍。

```
M_NO_BYPASS: assert property (
    @(posedge clk) disable iff (!rst_n)
    (mem_capture_due && r_fire)
    |-> (mem_capture_slot != rsp_e)
);
```

源码 cmdbuf_rd_port_sva.sv 第 539 行

[MODEL_SELF_CHECK] 模型完成集合是已发射集合的子集；已发射集合是有效集合的子集。

```
M_STATE: assert property (
    @(posedge clk) disable iff (!rst_n)
    ((m_sent & ~m_live) == 0 && (m_done & ~m_sent) == 0)
);
```

源码 cmdbuf_rd_port_sva.sv 第 550 行

[MODEL_SELF_CHECK] 模型待发数不超过占用数，占用数不超过 9。

```
M_RANGE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (waiting >= 0 && waiting <= used && used <= 9)
);
```

源码 cmdbuf_rd_port_sva.sv 第 560 行

[MODEL_SELF_CHECK] 模型的共享池与空源保留预算均不得超限。

```
M_QUOTA: assert property (
    @(posedge clk) disable iff (!rst_n)
    (shared_cost <= 6 && quota_cost <= 9)
);
```

源码 cmdbuf_rd_port_sva.sv 第 570 行

[MODEL_SELF_CHECK] 模型请求流水项必须指向合法、有效、未成功发射的事务。

```
M_ATTEMPT: assert property (
    @(posedge clk) disable iff (!rst_n)
    (m_attempt)
    |-> ((m_attempt_e<9 && m_live[m_attempt_e] && !m_sent[m_attempt_e]))
);
```

源码 cmdbuf_rd_port_sva.sv 第 580 行

[MODEL_SELF_CHECK] 每个有效逻辑槽的年龄排名必须落在当前有效事务范围内。

```
M_AGE_RANGE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (m_live[g])
    |-> (m_age[g]<used)
);
```

源码 cmdbuf_rd_port_sva.sv 第 599 行

[MODEL_SELF_CHECK] 每个已发射未完成事务在 4 拍返回日历中恰好有一项。

```
M_CALENDAR: assert property (
    @(posedge clk) disable iff (!rst_n)
    calendar_count == {3'b0,(m_live[g] && m_sent[g] && !m_done[g])}
);
```

源码 cmdbuf_rd_port_sva.sv 第 610 行

[MODEL_SELF_CHECK] 任意两个不同的有效逻辑槽不得使用相同年龄排名。

```
M_AGE_UNIQUE: assert property (
    @(posedge clk) disable iff (!rst_n)
    (m_live[g] && m_live[h])
    |-> (m_age[g] != m_age[h])
);
```

源码 cmdbuf_rd_port_sva.sv 第 622 行

附录 B 其余 cover 分支

正文已经给出每个场景的一个代表 cover。这里按原编号补齐其他分支，正文与本附录合计覆盖全部 547 个 Corner cover 标签。代码中的来源、Bank、half 和数据位均保持工程中的具体展开值。所有这些分支已有原生回放命中记录，不能据此宣称整项无界证明。

AR 01 空表首次接收

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_AR_01_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==0 && !r_fire && ar_fire && src==1 && arready==7
);
```

```
C_CC_AR_01_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==0 && !r_fire && ar_fire && src==2 && arready==7
);
```

AR 02 无请求时切换 src

本场景共 4 个分支；下列 3 个与正文代表分支共同组成完整集合。

```
C_CC_AR_02_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    !h_prev_valid && !arvalid && !h_prev_r && !r_fire && h_prev_n==used && src==1 &&
        src!=h_prev_src && arready==h_prev_ready
);
```

```
C_CC_AR_02_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    !h_prev_valid && !arvalid && !h_prev_r && !r_fire && h_prev_n==used && src==2 &&
        src!=h_prev_src && arready==h_prev_ready
);
```

```
C_CC_AR_02_S3: cover property (
    @(posedge clk) disable iff (!rst_n)
    !h_prev_valid && !arvalid && !h_prev_r && !r_fire && h_prev_n==used && src==3 &&
        src!=h_prev_src && arready==h_prev_ready
);
```

AR 03 选中可接收来源

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_AR_03_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && src==1
);
```

```
C_CC_AR_03_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && src==2
);
```

AR 16 有效时字段的边界值

本场景共 14 个分支；下列 13 个与正文代表分支共同组成完整集合。

```
C_CC_AR_16_S0_EXT1: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_src[rsp_e]==0 && m_addr[rsp_e]==18'h3ffff && rid==255 &&
        r_port_id==31
);
C_CC_AR_16_S1_EXT0: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_src[rsp_e]==1 && m_addr[rsp_e]==18'h00000 && rid==0 &&
        r_port_id==0
);
C_CC_AR_16_S1_EXT1: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_src[rsp_e]==1 && m_addr[rsp_e]==18'h3ffff && rid==255 &&
        r_port_id==31
);
C_CC_AR_16_S2_EXT0: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_src[rsp_e]==2 && m_addr[rsp_e]==18'h00000 && rid==0 &&
        r_port_id==0
);
C_CC_AR_16_S2_EXT1: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && m_src[rsp_e]==2 && m_addr[rsp_e]==18'h3ffff && rid==255 &&
        r_port_id==31
);
C_CC_AR_16_LOW0: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==0
);
C_CC_AR_16_LOW1: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==1
);
C_CC_AR_16_LOW2: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==2
);
C_CC_AR_16_LOW3: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==3
);
C_CC_AR_16_LOW4: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==4
);
C_CC_AR_16_LOW5: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==5
);
C_CC_AR_16_LOW6: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==6
);
C_CC_AR_16_LOW7: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && raddr==7
);
```

CAP 01 总量 0 到 6

本场景共 7 个分支；下列 6 个与正文代表分支共同组成完整集合。

```
C_CC_CAP_01_N1: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total==1 && !r_fire && arready==7
);
```

```
C_CC_CAP_01_N2: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total==2 && !r_fire && arready==7
);
```

```
C_CC_CAP_01_N3: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total==3 && !r_fire && arready==7
);
```

```
C_CC_CAP_01_N4: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total==4 && !r_fire && arready==7
);
```

```
C_CC_CAP_01_N5: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total==5 && !r_fire && arready==7
);
```

```
C_CC_CAP_01_N6: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_total==6 && !r_fire && arready==7
);
```

CAP 09 满表回收唯一项源

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_CAP_09_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && r_fire && winner==1 && occ[1]==1 && arready==(3'b1<<1)
);
```

```
C_CC_CAP_09_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    used==9 && r_fire && winner==2 && occ[2]==1 && arready==(3'b1<<2)
);
```

CAP 18 八种 ready 码全部出现

本场景共 8 个分支；下列 7 个与正文代表分支共同组成完整集合。

```
C_CC_CAP_18_READY001: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==1 && used>0
);
```

```
C_CC_CAP_18_READY010: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==2 && used>0
);
```

```
C_CC_CAP_18_READY011: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==3 && used>0
);
```

```
C_CC_CAP_18_READY100: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==4 && used>0
);
C_CC_CAP_18_READY101: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==5 && used>0
);
C_CC_CAP_18_READY110: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==6 && used>0
);
C_CC_CAP_18_READY111: cover property (
    @(posedge clk) disable iff (!rst_n)
    arready==7 && used>0
);
```

CAP 19 来源对称性

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_CAP_19_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[1]==7 && used==7 && !r_fire && arready==(3'b111^(3'b1<<1))
);
C_CC_CAP_19_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[2]==7 && used==7 && !r_fire && arready==(3'b111^(3'b1<<2))
);
```

CAP 22 非空并不代表可回收

本场景共 6 个分支；下列 5 个与正文代表分支共同组成完整集合。

```
C_CC_CAP_22_NOTREADY0: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[0] && m_done[head[0]] && !rready[0] && !rvalid[0]
);
C_CC_CAP_22_UNDONE1: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[1] && !m_done[head[1]] && rready[1] && !rvalid[1]
);
C_CC_CAP_22_NOTREADY1: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[1] && m_done[head[1]] && !rready[1] && !rvalid[1]
);
C_CC_CAP_22_UNDONE2: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[2] && !m_done[head[2]] && rready[2] && !rvalid[2]
);
C_CC_CAP_22_NOTREADY2: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[2] && m_done[head[2]] && !rready[2] && !rvalid[2]
);
```

WIN 03 窗口不足五项

本场景共 5 个分支；下列 4 个与正文代表分支共同组成完整集合。

```
C_CC_WIN_03_P1: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==1 && !issue_fire && (h_rank_valid>>1)==0
);
C_CC_WIN_03_P2: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==2 && !issue_fire && (h_rank_valid>>2)==0
);
C_CC_WIN_03_P3: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==3 && !issue_fire && (h_rank_valid>>3)==0
);
C_CC_WIN_03_P4: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==4 && !issue_fire && (h_rank_valid>>4)==0
);
```

WIN 09 六到九项窗口仍只取五

本场景共 4 个分支；下列 3 个与正文代表分支共同组成完整集合。

```
C_CC_WIN_09_P7: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==7 && blocked_six && blocked_streak>=1 && req==0
);
C_CC_WIN_09_P8: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==8 && blocked_six && blocked_streak>=1 && req==0
);
C_CC_WIN_09_P9: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==9 && blocked_six && blocked_streak>=1 && req==0
);
```

ARB 01 八个 Bank 编码边界

本场景共 8 个分支；下列 7 个与正文代表分支共同组成完整集合。

```
C_CC_ARB_01_B1: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<1) && arb_addr==m_addr[m_attempt_e][17:6]
);
C_CC_ARB_01_B2: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<2) && arb_addr==m_addr[m_attempt_e][17:6]
);
C_CC_ARB_01_B3: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<3) && arb_addr==m_addr[m_attempt_e][17:6]
);
C_CC_ARB_01_B4: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<4) && arb_addr==m_addr[m_attempt_e][17:6]
);
C_CC_ARB_01_B5: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<5) && arb_addr==m_addr[m_attempt_e][17:6]
);
```

```
C_CC_ARB_01_B6: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<6) && arb_addr==m_addr[m_attempt_e][17:6]
);
C_CC_ARB_01_B7: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_fire && req==(8'b1<<7) && arb_addr==m_addr[m_attempt_e][17:6]
);
```

ARB 02 奇偶半区独立

本场景共 16 个分支；下列 15 个与正文代表分支共同组成完整集合。

```
C_CC_ARB_02_B0H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==0 && m_addr[select_e][6]==1 &&
        rd_mem_status[0]==1
);
C_CC_ARB_02_B1H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==1 && m_addr[select_e][6]==0 &&
        rd_mem_status[1]==2
);
C_CC_ARB_02_B1H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==1 && m_addr[select_e][6]==1 &&
        rd_mem_status[1]==1
);
C_CC_ARB_02_B2H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==2 && m_addr[select_e][6]==0 &&
        rd_mem_status[2]==2
);
C_CC_ARB_02_B2H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==2 && m_addr[select_e][6]==1 &&
        rd_mem_status[2]==1
);
C_CC_ARB_02_B3H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==3 && m_addr[select_e][6]==0 &&
        rd_mem_status[3]==2
);
C_CC_ARB_02_B3H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==3 && m_addr[select_e][6]==1 &&
        rd_mem_status[3]==1
);
C_CC_ARB_02_B4H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==4 && m_addr[select_e][6]==0 &&
        rd_mem_status[4]==2
);
C_CC_ARB_02_B4H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==4 && m_addr[select_e][6]==1 &&
        rd_mem_status[4]==1
);
```

```
C_CC_ARB_02_B5H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==5 && m_addr[select_e][6]==0 &&
        rd_mem_status[5]==2
);
C_CC_ARB_02_B5H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==5 && m_addr[select_e][6]==1 &&
        rd_mem_status[5]==1
);
C_CC_ARB_02_B6H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==6 && m_addr[select_e][6]==0 &&
        rd_mem_status[6]==2
);
C_CC_ARB_02_B6H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==6 && m_addr[select_e][6]==1 &&
        rd_mem_status[6]==1
);
C_CC_ARB_02_B7H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==7 && m_addr[select_e][6]==0 &&
        rd_mem_status[7]==2
);
C_CC_ARB_02_B7H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==7 && m_addr[select_e][6]==1 &&
        rd_mem_status[7]==1
);
```

ARB 03 Bank 两半均空闲/均忙

本场景共 2 个分支；下列 1 个与正文代表分支共同组成完整集合。

```
C_CC_ARB_03_BUSY: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting>0 && h_status==16'hffff && !select_valid
);
```

ARB 04 发送地址保留 half

本场景共 4 个分支；下列 3 个与正文代表分支共同组成完整集合。

```
C_CC_ARB_04_ADDR1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && arb_addr==1 && arb_addr==m_addr[m_attempt_e][17:6]
);
C_CC_ARB_04_ADDR4094: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && arb_addr==4094 && arb_addr==m_addr[m_attempt_e][17:6]
);
C_CC_ARB_04_ADDR4095: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && arb_addr==4095 && arb_addr==m_addr[m_attempt_e][17:6]
);
```

ARB 05 四种双 grant 组合

本场景共 4 个分支；下列 3 个与正文代表分支共同组成完整集合。

```
C_CC_ARB_05_G01: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && ((req&ar_rd_gnt)!=0)==0 && ((req&rw_rd_gnt)!=0)==1 && issue_fire==0
);
C_CC_ARB_05_G10: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && ((req&ar_rd_gnt)!=0)==1 && ((req&rw_rd_gnt)!=0)==0 && issue_fire==0
);
C_CC_ARB_05_G11: cover property (
    @(posedge clk) disable iff (!rst_n)
    req!=0 && ((req&ar_rd_gnt)!=0)==1 && ((req&rw_rd_gnt)!=0)==1 && issue_fire==1
);
```

ARB 11 成功当拍目标 busy

本场景共 16 个分支；下列 15 个与正文代表分支共同组成完整集合。

```
C_CC_ARB_11_B0H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[0] && ar_rd_gnt[0] && rw_rd_gnt[0] && arb_addr[0]==1 && rd_mem_status[0][1]
);
C_CC_ARB_11_B1H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[1] && ar_rd_gnt[1] && rw_rd_gnt[1] && arb_addr[0]==0 && rd_mem_status[1][0]
);
C_CC_ARB_11_B1H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[1] && ar_rd_gnt[1] && rw_rd_gnt[1] && arb_addr[0]==1 && rd_mem_status[1][1]
);
C_CC_ARB_11_B2H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[2] && ar_rd_gnt[2] && rw_rd_gnt[2] && arb_addr[0]==0 && rd_mem_status[2][0]
);
C_CC_ARB_11_B2H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[2] && ar_rd_gnt[2] && rw_rd_gnt[2] && arb_addr[0]==1 && rd_mem_status[2][1]
);
C_CC_ARB_11_B3H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[3] && ar_rd_gnt[3] && rw_rd_gnt[3] && arb_addr[0]==0 && rd_mem_status[3][0]
);
C_CC_ARB_11_B3H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[3] && ar_rd_gnt[3] && rw_rd_gnt[3] && arb_addr[0]==1 && rd_mem_status[3][1]
);
C_CC_ARB_11_B4H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[4] && ar_rd_gnt[4] && rw_rd_gnt[4] && arb_addr[0]==0 && rd_mem_status[4][0]
);
C_CC_ARB_11_B4H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[4] && ar_rd_gnt[4] && rw_rd_gnt[4] && arb_addr[0]==1 && rd_mem_status[4][1]
);
C_CC_ARB_11_B5H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[5] && ar_rd_gnt[5] && rw_rd_gnt[5] && arb_addr[0]==0 && rd_mem_status[5][0]
);
```

```
C_CC_ARB_11_B5H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[5] && ar_rd_gnt[5] && rw_rd_gnt[5] && arb_addr[0]==1 && rd_mem_status[5][1]
);
C_CC_ARB_11_B6H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[6] && ar_rd_gnt[6] && rw_rd_gnt[6] && arb_addr[0]==0 && rd_mem_status[6][0]
);
C_CC_ARB_11_B6H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[6] && ar_rd_gnt[6] && rw_rd_gnt[6] && arb_addr[0]==1 && rd_mem_status[6][1]
);
C_CC_ARB_11_B7H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[7] && ar_rd_gnt[7] && rw_rd_gnt[7] && arb_addr[0]==0 && rd_mem_status[7][0]
);
C_CC_ARB_11_B7H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[7] && ar_rd_gnt[7] && rw_rd_gnt[7] && arb_addr[0]==1 && rd_mem_status[7][1]
);
```

CNT 01 四种 A/I 组合

本场景共 4 个分支；下列 3 个与正文代表分支共同组成完整集合。

```
C_CC_CNT_01_A0I1: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire==0 && issue_fire==1 && waiting>0 && pending_cnt==waiting+0-1
);
C_CC_CNT_01_A1I0: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire==1 && issue_fire==0 && waiting>0 && pending_cnt==waiting+1-0
);
C_CC_CNT_01_A1I1: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire==1 && issue_fire==1 && waiting>0 && pending_cnt==waiting+1-1
);
```

CNT 06 pending 上界与下界

本场景共 2 个分支；下列 1 个与正文代表分支共同组成完整集合。

```
C_CC_CNT_06_FULL: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting==9 && !ar_fire && !issue_fire && pending_cnt==9
);
```

CNT 07 失败与 MEM 不减 pending

本场景共 2 个分支；下列 1 个与正文代表分支共同组成完整集合。

```
C_CC_CNT_07_MEM: cover property (
    @(posedge clk) disable iff (!rst_n)
    mem_capture_due && !issue_fire && !ar_fire && pending_cnt==waiting
);
```

CNT 08 R 仅间接影响 pending

本场景共 2 个分支；下列 1 个与正文代表分支共同组成完整集合。

```
C_CC_CNT_08_A1: cover property (
    @(posedge clk) disable iff (!rst_n)
    r_fire && ar_fire==1 && !issue_fire && pending_cnt==waiting+1
);
```

MEM 12 128 位数据全宽特征

本场景共 260 个分支；下列 259 个与正文代表分支共同组成完整集合。

```
C_CC_MEM_12_ZERO: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==128'b0
);
C_CC_MEM_12_ONES: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata=={128{1'b1}}
);
C_CC_MEM_12_AA: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata=={16{8'haa}}
);
C_CC_MEM_12_55: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata=={16{8'h55}}
);
C_CC_MEM_12_W1_000: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<0)
);
C_CC_MEM_12_W0_000: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<0)
);
C_CC_MEM_12_W1_001: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<1)
);
C_CC_MEM_12_W0_001: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<1)
);
C_CC_MEM_12_W1_002: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<2)
);
C_CC_MEM_12_W0_002: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<2)
);
C_CC_MEM_12_W1_003: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<3)
);
C_CC_MEM_12_W0_003: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<3)
);
```

```
C_CC_MEM_12_W1_004: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<4)
);
C_CC_MEM_12_W0_004: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<4)
);
C_CC_MEM_12_W1_005: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<5)
);
C_CC_MEM_12_W0_005: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<5)
);
C_CC_MEM_12_W1_006: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<6)
);
C_CC_MEM_12_W0_006: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<6)
);
C_CC_MEM_12_W1_007: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<7)
);
C_CC_MEM_12_W0_007: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<7)
);
C_CC_MEM_12_W1_008: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<8)
);
C_CC_MEM_12_W0_008: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<8)
);
C_CC_MEM_12_W1_009: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<9)
);
C_CC_MEM_12_W0_009: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<9)
);
C_CC_MEM_12_W1_010: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<10)
);
C_CC_MEM_12_W0_010: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<10)
);
```

```
C_CC_MEM_12_W1_011: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<11)
);
C_CC_MEM_12_W0_011: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<11)
);
C_CC_MEM_12_W1_012: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<12)
);
C_CC_MEM_12_W0_012: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<12)
);
C_CC_MEM_12_W1_013: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<13)
);
C_CC_MEM_12_W0_013: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<13)
);
C_CC_MEM_12_W1_014: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<14)
);
C_CC_MEM_12_W0_014: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<14)
);
C_CC_MEM_12_W1_015: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<15)
);
C_CC_MEM_12_W0_015: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<15)
);
C_CC_MEM_12_W1_016: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<16)
);
C_CC_MEM_12_W0_016: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<16)
);
C_CC_MEM_12_W1_017: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<17)
);
C_CC_MEM_12_W0_017: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<17)
);
```

```
C_CC_MEM_12_W1_018: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<18)
);
C_CC_MEM_12_W0_018: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<18)
);
C_CC_MEM_12_W1_019: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<19)
);
C_CC_MEM_12_W0_019: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<19)
);
C_CC_MEM_12_W1_020: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<20)
);
C_CC_MEM_12_W0_020: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<20)
);
C_CC_MEM_12_W1_021: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<21)
);
C_CC_MEM_12_W0_021: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<21)
);
C_CC_MEM_12_W1_022: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<22)
);
C_CC_MEM_12_W0_022: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<22)
);
C_CC_MEM_12_W1_023: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<23)
);
C_CC_MEM_12_W0_023: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<23)
);
C_CC_MEM_12_W1_024: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<24)
);
C_CC_MEM_12_W0_024: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<24)
);
```

```
C_CC_MEM_12_W1_025: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<25)
);
C_CC_MEM_12_W0_025: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<25)
);
C_CC_MEM_12_W1_026: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<26)
);
C_CC_MEM_12_W0_026: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<26)
);
C_CC_MEM_12_W1_027: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<27)
);
C_CC_MEM_12_W0_027: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<27)
);
C_CC_MEM_12_W1_028: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<28)
);
C_CC_MEM_12_W0_028: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<28)
);
C_CC_MEM_12_W1_029: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<29)
);
C_CC_MEM_12_W0_029: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<29)
);
C_CC_MEM_12_W1_030: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<30)
);
C_CC_MEM_12_W0_030: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<30)
);
C_CC_MEM_12_W1_031: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<31)
);
C_CC_MEM_12_W0_031: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<31)
);
```

```
C_CC_MEM_12_W1_032: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<32)
);
C_CC_MEM_12_W0_032: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<32)
);
C_CC_MEM_12_W1_033: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<33)
);
C_CC_MEM_12_W0_033: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<33)
);
C_CC_MEM_12_W1_034: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<34)
);
C_CC_MEM_12_W0_034: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<34)
);
C_CC_MEM_12_W1_035: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<35)
);
C_CC_MEM_12_W0_035: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<35)
);
C_CC_MEM_12_W1_036: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<36)
);
C_CC_MEM_12_W0_036: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<36)
);
C_CC_MEM_12_W1_037: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<37)
);
C_CC_MEM_12_W0_037: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<37)
);
C_CC_MEM_12_W1_038: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<38)
);
C_CC_MEM_12_W0_038: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<38)
);
```

```
C_CC_MEM_12_W1_039: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<39)
);
C_CC_MEM_12_W0_039: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<39)
);
C_CC_MEM_12_W1_040: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<40)
);
C_CC_MEM_12_W0_040: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<40)
);
C_CC_MEM_12_W1_041: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<41)
);
C_CC_MEM_12_W0_041: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<41)
);
C_CC_MEM_12_W1_042: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<42)
);
C_CC_MEM_12_W0_042: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<42)
);
C_CC_MEM_12_W1_043: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<43)
);
C_CC_MEM_12_W0_043: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<43)
);
C_CC_MEM_12_W1_044: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<44)
);
C_CC_MEM_12_W0_044: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<44)
);
C_CC_MEM_12_W1_045: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<45)
);
C_CC_MEM_12_W0_045: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<45)
);
```

```
C_CC_MEM_12_W1_046: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<46)
);
C_CC_MEM_12_W0_046: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<46)
);
C_CC_MEM_12_W1_047: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<47)
);
C_CC_MEM_12_W0_047: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<47)
);
C_CC_MEM_12_W1_048: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<48)
);
C_CC_MEM_12_W0_048: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<48)
);
C_CC_MEM_12_W1_049: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<49)
);
C_CC_MEM_12_W0_049: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<49)
);
C_CC_MEM_12_W1_050: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<50)
);
C_CC_MEM_12_W0_050: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<50)
);
C_CC_MEM_12_W1_051: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<51)
);
C_CC_MEM_12_W0_051: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<51)
);
C_CC_MEM_12_W1_052: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<52)
);
C_CC_MEM_12_W0_052: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<52)
);
```

```
C_CC_MEM_12_W1_053: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<53)
);
C_CC_MEM_12_W0_053: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<53)
);
C_CC_MEM_12_W1_054: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<54)
);
C_CC_MEM_12_W0_054: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<54)
);
C_CC_MEM_12_W1_055: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<55)
);
C_CC_MEM_12_W0_055: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<55)
);
C_CC_MEM_12_W1_056: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<56)
);
C_CC_MEM_12_W0_056: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<56)
);
C_CC_MEM_12_W1_057: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<57)
);
C_CC_MEM_12_W0_057: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<57)
);
C_CC_MEM_12_W1_058: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<58)
);
C_CC_MEM_12_W0_058: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<58)
);
C_CC_MEM_12_W1_059: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<59)
);
C_CC_MEM_12_W0_059: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<59)
);
```

```
C_CC_MEM_12_W1_060: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<60)
);
C_CC_MEM_12_W0_060: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<60)
);
C_CC_MEM_12_W1_061: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<61)
);
C_CC_MEM_12_W0_061: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<61)
);
C_CC_MEM_12_W1_062: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<62)
);
C_CC_MEM_12_W0_062: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<62)
);
C_CC_MEM_12_W1_063: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<63)
);
C_CC_MEM_12_W0_063: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<63)
);
C_CC_MEM_12_W1_064: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<64)
);
C_CC_MEM_12_W0_064: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<64)
);
C_CC_MEM_12_W1_065: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<65)
);
C_CC_MEM_12_W0_065: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<65)
);
C_CC_MEM_12_W1_066: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<66)
);
C_CC_MEM_12_W0_066: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<66)
);
```

```
C_CC_MEM_12_W1_067: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<67)
);
C_CC_MEM_12_W0_067: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<67)
);
C_CC_MEM_12_W1_068: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<68)
);
C_CC_MEM_12_W0_068: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<68)
);
C_CC_MEM_12_W1_069: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<69)
);
C_CC_MEM_12_W0_069: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<69)
);
C_CC_MEM_12_W1_070: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<70)
);
C_CC_MEM_12_W0_070: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<70)
);
C_CC_MEM_12_W1_071: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<71)
);
C_CC_MEM_12_W0_071: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<71)
);
C_CC_MEM_12_W1_072: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<72)
);
C_CC_MEM_12_W0_072: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<72)
);
C_CC_MEM_12_W1_073: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<73)
);
C_CC_MEM_12_W0_073: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<73)
);
```

```
C_CC_MEM_12_W1_074: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<74)
);
C_CC_MEM_12_W0_074: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<74)
);
C_CC_MEM_12_W1_075: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<75)
);
C_CC_MEM_12_W0_075: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<75)
);
C_CC_MEM_12_W1_076: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<76)
);
C_CC_MEM_12_W0_076: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<76)
);
C_CC_MEM_12_W1_077: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<77)
);
C_CC_MEM_12_W0_077: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<77)
);
C_CC_MEM_12_W1_078: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<78)
);
C_CC_MEM_12_W0_078: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<78)
);
C_CC_MEM_12_W1_079: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<79)
);
C_CC_MEM_12_W0_079: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<79)
);
C_CC_MEM_12_W1_080: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<80)
);
C_CC_MEM_12_W0_080: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<80)
);
```

```
C_CC_MEM_12_W1_081: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<81)
);
C_CC_MEM_12_W0_081: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<81)
);
C_CC_MEM_12_W1_082: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<82)
);
C_CC_MEM_12_W0_082: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<82)
);
C_CC_MEM_12_W1_083: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<83)
);
C_CC_MEM_12_W0_083: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<83)
);
C_CC_MEM_12_W1_084: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<84)
);
C_CC_MEM_12_W0_084: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<84)
);
C_CC_MEM_12_W1_085: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<85)
);
C_CC_MEM_12_W0_085: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<85)
);
C_CC_MEM_12_W1_086: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<86)
);
C_CC_MEM_12_W0_086: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<86)
);
C_CC_MEM_12_W1_087: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<87)
);
C_CC_MEM_12_W0_087: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<87)
);
```

```
C_CC_MEM_12_W1_088: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<88)
);
C_CC_MEM_12_W0_088: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<88)
);
C_CC_MEM_12_W1_089: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<89)
);
C_CC_MEM_12_W0_089: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<89)
);
C_CC_MEM_12_W1_090: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<90)
);
C_CC_MEM_12_W0_090: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<90)
);
C_CC_MEM_12_W1_091: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<91)
);
C_CC_MEM_12_W0_091: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<91)
);
C_CC_MEM_12_W1_092: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<92)
);
C_CC_MEM_12_W0_092: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<92)
);
C_CC_MEM_12_W1_093: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<93)
);
C_CC_MEM_12_W0_093: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<93)
);
C_CC_MEM_12_W1_094: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<94)
);
C_CC_MEM_12_W0_094: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<94)
);
```

```
C_CC_MEM_12_W1_095: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<95)
);
C_CC_MEM_12_W0_095: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<95)
);
C_CC_MEM_12_W1_096: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<96)
);
C_CC_MEM_12_W0_096: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<96)
);
C_CC_MEM_12_W1_097: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<97)
);
C_CC_MEM_12_W0_097: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<97)
);
C_CC_MEM_12_W1_098: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<98)
);
C_CC_MEM_12_W0_098: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<98)
);
C_CC_MEM_12_W1_099: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<99)
);
C_CC_MEM_12_W0_099: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<99)
);
C_CC_MEM_12_W1_100: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<100)
);
C_CC_MEM_12_W0_100: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<100)
);
C_CC_MEM_12_W1_101: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<101)
);
C_CC_MEM_12_W0_101: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<101)
);
```

```
C_CC_MEM_12_W1_102: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<102)
);
C_CC_MEM_12_W0_102: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<102)
);
C_CC_MEM_12_W1_103: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<103)
);
C_CC_MEM_12_W0_103: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<103)
);
C_CC_MEM_12_W1_104: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<104)
);
C_CC_MEM_12_W0_104: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<104)
);
C_CC_MEM_12_W1_105: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<105)
);
C_CC_MEM_12_W0_105: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<105)
);
C_CC_MEM_12_W1_106: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<106)
);
C_CC_MEM_12_W0_106: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<106)
);
C_CC_MEM_12_W1_107: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<107)
);
C_CC_MEM_12_W0_107: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<107)
);
C_CC_MEM_12_W1_108: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<108)
);
C_CC_MEM_12_W0_108: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<108)
);
```

```
C_CC_MEM_12_W1_109: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<109)
);
C_CC_MEM_12_W0_109: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<109)
);
C_CC_MEM_12_W1_110: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<110)
);
C_CC_MEM_12_W0_110: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<110)
);
C_CC_MEM_12_W1_111: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<111)
);
C_CC_MEM_12_W0_111: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<111)
);
C_CC_MEM_12_W1_112: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<112)
);
C_CC_MEM_12_W0_112: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<112)
);
C_CC_MEM_12_W1_113: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<113)
);
C_CC_MEM_12_W0_113: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<113)
);
C_CC_MEM_12_W1_114: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<114)
);
C_CC_MEM_12_W0_114: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<114)
);
C_CC_MEM_12_W1_115: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<115)
);
C_CC_MEM_12_W0_115: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<115)
);
```

```
C_CC_MEM_12_W1_116: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<116)
);
C_CC_MEM_12_W0_116: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<116)
);
C_CC_MEM_12_W1_117: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<117)
);
C_CC_MEM_12_W0_117: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<117)
);
C_CC_MEM_12_W1_118: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<118)
);
C_CC_MEM_12_W0_118: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<118)
);
C_CC_MEM_12_W1_119: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<119)
);
C_CC_MEM_12_W0_119: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<119)
);
C_CC_MEM_12_W1_120: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<120)
);
C_CC_MEM_12_W0_120: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<120)
);
C_CC_MEM_12_W1_121: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<121)
);
C_CC_MEM_12_W0_121: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<121)
);
C_CC_MEM_12_W1_122: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<122)
);
C_CC_MEM_12_W0_122: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<122)
);
```

```
C_CC_MEM_12_W1_123: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<123)
);
C_CC_MEM_12_W0_123: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<123)
);
C_CC_MEM_12_W1_124: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<124)
);
C_CC_MEM_12_W0_124: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<124)
);
C_CC_MEM_12_W1_125: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<125)
);
C_CC_MEM_12_W0_125: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<125)
);
C_CC_MEM_12_W1_126: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==(128'b1<<126)
);
C_CC_MEM_12_W0_126: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<126)
);
C_CC_MEM_12_W0_127: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && rdata==~(128'b1<<127)
);
```

RSP 03 只有一路候选

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_03_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==(3'b1<<1) && rvalid==(3'b1<<1) && h_response_matches
);
C_CC_RSP_03_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==(3'b1<<2) && rvalid==(3'b1<<2) && h_response_matches
);
```

RSP 05 两路候选跳过空缺

本场景共 9 个分支；下列 8 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_05_M3RR1: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==3 && m_rr==1 && h_response_matches
);
```

```
C_CC_RSP_05_M3RR2: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==3 && m_rr==2 && h_response_matches
);
C_CC_RSP_05_M5RR0: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==5 && m_rr==0 && h_response_matches
);
C_CC_RSP_05_M5RR1: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==5 && m_rr==1 && h_response_matches
);
C_CC_RSP_05_M5RR2: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==5 && m_rr==2 && h_response_matches
);
C_CC_RSP_05_M6RR0: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==6 && m_rr==0 && h_response_matches
);
C_CC_RSP_05_M6RR1: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==6 && m_rr==1 && h_response_matches
);
C_CC_RSP_05_M6RR2: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible==6 && m_rr==2 && h_response_matches
);
```

RSP 09 同源 3 先发、先完成

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_09_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[1].order_phase==5 && sources[1].source_r &&
        sources[1].oldest==sources[1].third_token && sources[1].head_payload_matches
);
C_CC_RSP_09_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[2].order_phase==5 && sources[2].source_r &&
        sources[2].oldest==sources[2].third_token && sources[2].head_payload_matches
);
```

RSP 10 同源第二条完成但第一条未发

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_10_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[1].order_count>=2 && !m_sent[sources[1].order_slot[0]] &&
        m_done[sources[1].order_slot[1]] && rready[1] && !rvalid[1]
);
C_CC_RSP_10_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[2].order_count>=2 && !m_sent[sources[2].order_slot[0]] &&
        m_done[sources[2].order_slot[1]] && rready[2] && !rvalid[2]
);
```

RSP 11 不同 ID 仍要同源保序

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_11_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[1].order_count>=2 &&
        m_id[sources[1].order_slot[0]]!=m_id[sources[1].order_slot[1]] &&
        m_done[sources[1].order_slot[1]] && sources[1].source_r &&
        sources[1].head_payload_matches
);
```

```
C_CC_RSP_11_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[2].order_count>=2 &&
        m_id[sources[2].order_slot[0]]!=m_id[sources[2].order_slot[1]] &&
        m_done[sources[2].order_slot[1]] && sources[2].source_r &&
        sources[2].head_payload_matches
);
```

RSP 12 重复 ID 不合并事务

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_12_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[1].order_count>=2 &&
        m_id[sources[1].order_slot[0]]==m_id[sources[1].order_slot[1]] &&
        m_data[sources[1].order_slot[0]]!=m_data[sources[1].order_slot[1]] &&
        m_done[sources[1].order_slot[1]] && sources[1].source_r &&
        sources[1].head_payload_matches
);
```

```
C_CC_RSP_12_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[2].order_count>=2 &&
        m_id[sources[2].order_slot[0]]==m_id[sources[2].order_slot[1]] &&
        m_data[sources[2].order_slot[0]]!=m_data[sources[2].order_slot[1]] &&
        m_done[sources[2].order_slot[1]] && sources[2].source_r &&
        sources[2].head_payload_matches
);
```

RSP 13 一源阻塞其他源仍推进

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_13_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[1] && !m_done[head[1]] && (rvalid&~(3'b1<<1))!=0 &&
        h_response_matches
);
```

```
C_CC_RSP_13_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    head_valid[2] && !m_done[head[2]] && (rvalid&~(3'b1<<2))!=0 &&
        h_response_matches
);
```

RSP 14 同源队头 R 后立即换头

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_14_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_r && h_prev_rsrc==1 && rvalid[1] && h_response_matches
);
C_CC_RSP_14_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_prev_r && h_prev_rsrc==2 && rvalid[2] && h_response_matches
);
```

RSP 15 同源唯一旧项 R 与新 AR

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_15_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[1].order_count==1 && sources[1].source_r && sources[1].source_ar &&
        sources[1].head_payload_matches
);
C_CC_RSP_15_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    sources[2].order_count==1 && sources[2].source_r && sources[2].source_ar &&
        sources[2].head_payload_matches
);
```

RSP 16 持续候选的 RR 等待上界

本场景共 3 个分支；下列 2 个与正文代表分支共同组成完整集合。

```
C_CC_RSP_16_S1: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible[1] && sources[1].bypass_count==2 && rvalid[1] && h_response_matches
);
C_CC_RSP_16_S2: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible[2] && sources[2].bypass_count==2 && rvalid[2] && h_response_matches
);
```

ERR 08 port_id 直接透传

本场景共 5 个分支；下列 4 个与正文代表分支共同组成完整集合。

```
C_CC_ERR_08_P1: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && r_port_id==1
);
C_CC_ERR_08_P15: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && r_port_id==15
);
C_CC_ERR_08_P16: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && r_port_id==16
);
C_CC_ERR_08_P31: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_response_matches && r_port_id==31
);
```

BIZ 01 单源持续满速流

本场景共 2 个分支；下列 1 个与正文代表分支共同组成完整集合。

```
C_CC_BIZ_01_STEADY4: cover property (
    @(posedge clk) disable iff (!rst_n)
    h_ar_r_run>=3 && ar_fire && r_fire && src==winner && (h_ar_sources==1 ||
        h_ar_sources==2 || h_ar_sources==4) && h_response_matches
);
```

ENV 06 数组连接方向

本场景共 16 个分支；下列 15 个与正文代表分支共同组成完整集合。

```
C_CC_ENV_06_B0H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==0 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<1))
);
```

```
C_CC_ENV_06_B1H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==1 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<2))
);
```

```
C_CC_ENV_06_B1H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==1 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<3))
);
```

```
C_CC_ENV_06_B2H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==2 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<4))
);
```

```
C_CC_ENV_06_B2H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==2 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<5))
);
```

```
C_CC_ENV_06_B3H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==3 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<6))
);
```

```
C_CC_ENV_06_B3H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==3 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<7))
);
```

```
C_CC_ENV_06_B4H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==4 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<8))
);
```

```
C_CC_ENV_06_B4H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==4 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<9))
);
```

```
C_CC_ENV_06_B5H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==5 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<10))
);
C_CC_ENV_06_B5H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==5 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<11))
);
C_CC_ENV_06_B6H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==6 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<12))
);
C_CC_ENV_06_B6H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==6 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<13))
);
C_CC_ENV_06_B7H0: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==7 && m_addr[select_e][6]==0 &&
        h_status==(16'hffff^(16'b1<<14))
);
C_CC_ENV_06_B7H1: cover property (
    @(posedge clk) disable iff (!rst_n)
    select_valid && m_addr[select_e][5:3]==7 && m_addr[select_e][6]==1 &&
        h_status==(16'hffff^(16'b1<<15))
);
```

附录 C 源码接入与辅助变量

本册是阅读用的场景与代码对照，编译时仍使用工程 native.f。它选入当前三份 SV RTL 和完整检查器，保留 generate、参考模型及历史记录。不要只复制一条使用 m_* 的断言而漏掉相应定义。

路径	内容
formal/corner_20261021/native.f	当前原生文件清单，启用 CMDBUF_CORNER_SUITE。
formal/protocol_20261021/cmdbuf_rd_port_sva.sv	主检查器、顶层事件、环境约束、DUT 断言及基础 cover。
formal/protocol_20261021/cmdbuf_rd_reference_model.svh	九项事务记录、年龄、预选流水、固定四拍日历、期望响应。
formal/protocol_20261021/cmdbuf_rd_xau_order_model.svh	三个独立接收 FIFO，按 AR 接收和本源 R 出队。
formal/corner_20261021/cmdbuf_rd_corner_history.svh	独立握手计数 h_occ、上一拍快照和场景历史。
formal/corner_20261021/cmdbuf_rd_corner_sequences.svh	长场景见证进度，均由端口事件推进。
formal/corner_20261021/cmdbuf_rd_corner_assertions.svh	A_CC_* 容量和 C2/C3 检查。
formal/corner_20261021/cmdbuf_rd_corner_properties.svh	本册全部 547 个 Corner cover。
formal/corner_20261021/negative/*.sv	四个独立负向环境监测器，不能混入正常作业。

同名属性在 generate 中按来源或槽位展开。正文的 A_CC_SRC_LIMIT 中 cs=0/1/2；A_XAU_R_* 中 g=0/1/2。order_slot 和 oldest 指向验证侧 token，不是读取 DUT 的物理分配。

附录 C 变量读法与证据定位

变量	来源与用途
ar_fire / issue_fire / r_fire	当前顶层端口组合事件，即图中 A / I / R。
h_occ / h_total / h_budget	只数端口 AR/R；h_budget=sum(max(h_occ[s],1))。
h_after / h_ready	先扣本拍实际 R 后，算本拍各源接收资格。
waiting / used / occ	验证侧的待发总量、存活总量和各源存活量。
pool / ranks / window / idle	本拍成功项扣除后的待发集合、年龄排名、最老五项、空闲 half 判断。
select_valid / select_e	C2 预选结果；保存一拍后成为 m_attempt / m_attempt_e。
h_selected_prev / h_selected_req	上拍 C2 选择快照，用于直接检查本拍 C3 请求。
mem_capture_due / mem_capture_slot	成功发射后第 4 拍的验证日历到期事件和对应 token。
m_live / m_sent / m_done	验证事务的存活、成功发射、已捕获 MEM 状态。
m_addr / m_id / m_port / m_data / m_pc	该验证事务保存的 AR 元数据和到期拍 MEM 数据。
sources[s].oldest / order_count	本源独立 AR FIFO 的队头和长度；R 只允许从队头返回。
eligible / m_rr / expected_rvalid	可响应源集合、期望 RR 起点和期望 R 选择。
h_prev_* / h_*_history / h_*_run	上拍值、最近若干拍事件历史、连续运行计数，用于描述多拍 cover。

证据索引 verification/corner_20261021/case_property_results.json 给出每项所有 cover、实际轨迹、日志 step 和边界。VCD 位于同目录 vm_evidence.tar.gz 中，场景页所列路径是该包内路径。资料包以 SHA256 清单校验，不改写历史轨迹。

正常执行来源与证明范围见 summary.json、audit.json。全文的 SVA 取自当前源文件，未使用 SBY 的布尔降级执行副本。编译入口保持原生 |->、posedge clk 和 disable iff；辅助状态由完整工程负责建立。

附录 D 关联的基础 cover

场景页引用的下列 C_* 属性来自主检查器，属于当前 Corner 扩展之外的基础覆盖。它们与 547 个 C_CC_* 分支分别计数。这里补齐完整代码，便于按标签检索。

[REF_MODEL] 命令 3 先捕获，命令 1 未完成；即使 ready=1，该源仍无响应。

```
C_XAU_EARLY_THIRD_BLOCKED: cover property (
    @(posedge clk) disable iff (!rst_n)
    order_phase == 3 && m_done[third_token] && !m_done[first_token] && rready[g] &&
        !rvalid[g]
);
```

源码 cmdbuf_rd_port_sva.sv 第 433 行

[REF_MODEL] 完整见证：3 先发射及捕获，但 R 按 1、2、3 返回对应整组负载。

```
C_XAU_R_123_AFTER_ISSUE3_FIRST: cover property (
    @(posedge clk) disable iff (!rst_n)
    order_phase == 5 && source_r && oldest == third_token && head_payload_matches
);
```

源码 cmdbuf_rd_port_sva.sv 第 443 行

[REF_MODEL] 该源因旧命令未完成而等待时，另一个 XAU 仍能返回。

```
C_XAU_OTHER_SOURCE_PROGRESS: cover property (
    @(posedge clk) disable iff (!rst_n)
    order_phase == 3 && m_done[third_token] && !m_done[first_token] && (|(rvalid &
        ~(3'b1<<g)))
);
```

源码 cmdbuf_rd_port_sva.sv 第 453 行

[REF_MODEL] 同一个 XAU 同拍返回旧命令、接受新命令。

```
C_XAU_SAME_CYCLE_POP_PUSH: cover property (
    @(posedge clk) disable iff (!rst_n)
    source_ar && source_r && order_count>0 && head_payload_matches
);
```

源码 cmdbuf_rd_port_sva.sv 第 463 行

[REF_MODEL] 队列中 ID 重复且年轻项已完成时，响应仍匹配最老队头。

```
C_XAU_REPEAT_ID_HEAD_R: cover property (
    @(posedge clk) disable iff (!rst_n)
    order_count >= 2 &&
    m_id[order_slot[0]] == m_id[order_slot[1]] &&
    m_done[order_slot[1]] &&
    source_r &&
    head_payload_matches
);
```

源码 cmdbuf_rd_port_sva.sv 第 473 行

[REF_MODEL] 仲裁失败后本拍重选同一命令，下一拍继续发出该请求。

```
C_RETRY_NEXT_CYCLE: cover property (
    @(posedge clk) disable iff (!rst_n)
    retry_selected && req == last_retry_req && arb_addr == last_retry_addr && req !=
        0
);
```

源码 cmdbuf_rd_port_sva.sv 第 640 行

[REF_MODEL] 当前请求失败且对应半区忙，本拍选中窗口内另一条命令。

```
C_FAILURE_SELECT_OTHER: cover property (
    @(posedge clk) disable iff (!rst_n)
    m_attempt && !issue_fire && !idle[m_attempt_e] && select_valid && select_e !=
        m_attempt_e
);
```

源码 cmdbuf_rd_port_sva.sv 第 650 行

[REF_MODEL] 成功当拍补入原第 6 条命令，下一拍请求它。

```
C_REFILL_NEXT_CYCLE: cover property (
    @(posedge clk) disable iff (!rst_n)
    refill_selected && req == last_refill_req && arb_addr == last_refill_addr
);
```

源码 cmdbuf_rd_port_sva.sv 第 660 行

[REF_MODEL] 窗口非队头成功后，原第 6 条同拍补入并于下一拍请求。

```
C_NONHEAD_REFILL: cover property (
    @(posedge clk) disable iff (!rst_n)
    nonhead_refill_selected && req == last_refill_req && arb_addr ==
        last_refill_addr
);
```

源码 cmdbuf_rd_port_sva.sv 第 670 行

[REF_MODEL] 窗口五条均不匹配、窗口外第六条匹配，连续等待仍不能越窗发射。

```
C_BLOCKED_SIX: cover property (
    @(posedge clk) disable iff (!rst_n)
    blocked_six && blocked_streak >= 3
);
```

源码 cmdbuf_rd_port_sva.sv 第 680 行

[REF_MODEL] 原待发 5 条，本拍只成功发射一条，pending_cnt 本拍为 4。

```
C_PENDING5_DEC: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting == 5 && !ar_fire && issue_fire && pending_cnt == 4
);
```

源码 cmdbuf_rd_port_sva.sv 第 689 行

[REF_MODEL] 原待发 5 条，本拍同收同发，pending_cnt 本拍仍为 5。

```
C_PENDING5_BOTH: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting == 5 && ar_fire && issue_fire && pending_cnt == 5
);
```

源码 cmdbuf_rd_port_sva.sv 第 698 行

[REF_MODEL] 原待发 5 条，本拍只接收一条，pending_cnt 本拍为 6。

```
C_PENDING5_ADD: cover property (
    @(posedge clk) disable iff (!rst_n)
    waiting == 5 && ar_fire && !issue_fire && pending_cnt == 6
);
```

源码 cmdbuf_rd_port_sva.sv 第 707 行

[REF_MODEL] 9 项满占用时，同拍 R 回收并接收新 AR。

```
C_FULL_REUSE: cover property (
    @(posedge clk) disable iff (!rst_n)
    used == 9 && ar_fire && r_fire
);
```

源码 cmdbuf_rd_port_sva.sv 第 717 行

[REF_MODEL] 同拍发生 AR 接收、成功发射、MEM 捕获和 R 返回。

```
C_ALL_FOUR_EVENTS: cover property (
    @(posedge clk) disable iff (!rst_n)
    ar_fire && issue_fire && mem_capture_due && r_fire
);
```

源码 cmdbuf_rd_port_sva.sv 第 727 行

[REF_MODEL] 占用为 7/0/0 时，仍允许其余两个 XAU 各接收第一条命令。

```
C_CAPACITY700: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0] == 7 && occ[1] == 0 && occ[2] == 0 && !r_fire && aready == 6
);
```

源码 cmdbuf_rd_port_sva.sv 第 737 行

[REF_MODEL] 占用为 4/4/0 时，最后空源仍可接收，已有两个源暂不可接收。

```
C_CAPACITY440: cover property (
    @(posedge clk) disable iff (!rst_n)
    occ[0] == 4 && occ[1] == 4 && occ[2] == 0 && !r_fire && aready == 4
);
```

源码 cmdbuf_rd_port_sva.sv 第 747 行

[REF_MODEL] 同一事务越界与校验异常同时存在，最终响应越界 11。

```
C_RETURN_ERROR_PRIORITY: cover property (
    @(posedge clk) disable iff (!rst_n)
    r_fire && m_err[rsp_e] == 3 && m_pc[rsp_e] && rresp == 3
);
```

源码 cmdbuf_rd_port_sva.sv 第 756 行

[REF_MODEL] 部分通道不 ready 时，RR 仍能选出可响应来源。

```
C_READY_MASK_RR: cover property (
    @(posedge clk) disable iff (!rst_n)
    eligible != 0 && rready != 7 && r_fire
);
```

源码 cmdbuf_rd_port_sva.sv 第 765 行

[NO_REF_MODEL_HISTORY] 连续四拍成功发射，仅使用成功发射事件历史。

```
C_FOUR_SUCCESS: cover property (
    @(posedge clk) disable iff (!rst_n)
    issue_streak >= 3 && issue_fire
);
```

源码 cmdbuf_rd_port_sva.sv 第 775 行

[NO_REF_MODEL_PORTS] 逐 Bank、逐奇偶半区命中成功发射且状态报告占用。

```
C_SUCCESS_BUSY_TARGET: cover property (
    @(posedge clk) disable iff (!rst_n)
    req[cb] && ar_rd_gnt[cb] && rw_rd_gnt[cb] && arb_addr[0] == ch && this_half_busy
);
```

源码 cmdbuf_rd_port_sva.sv 第 790 行

附录 E 已识别的覆盖条件差异

以下差异来自本次文字、SVA 与实际波形的交叉核对。对应 cover 已命中，但条件比文字场景宽，不能把该命中解释为完整场景已经闭合。本次只如实记录，未修改 SVA、补跑求解器或改变封存证据；这也不是对 DUT 缺陷的判定。

ARB-13：其他 Bank 状态变化

已有条件是 h_status!=h_prev_status，表示任意一个状态位变化。它同时检查当前 req/arb_addr 仍为上一拍选中的值，且本拍没有发射成功；没有要求被请求的 half 状态保持不变。代表轨迹中 Bank0 从 00 变为 01，目标位也改变了。

后续应补：固定候选集合及相关候选 half 的状态，仅改变不相关 Bank/half 的状态，再检查预选和下一拍请求。激励限制只描述输入，不应把待验证的输出正确性放入 assume。

MEM-03：连续返回时每拍的数据区分

已有条件要求过去三拍及当前拍均捕获，只有当前 mem_rdata 与上一拍、当前 pc_err 与上一拍不等。代表轨迹的四次捕获数据为 0、0、0、1<<127，pc_err 为 0、0、0、1；前三次没有区分。

后续应补：在连续捕获的每个相邻拍都记录并比较数据/校验位；若目标要求四条数据两两不同，应进一步明确并检查两两差异。已有捕获、保存与 R 数据一致性断言仍按其各自的验证范围解释。

MEM-06：捕获后的下一拍总线变化

已有条件要求 R 对应保存事务、完成后已等待至少一拍，并且当前 MEM 数据/校验位与保存值不同。代表轨迹在 S6 捕获，S7 总线仍保持原值，到 S8 返回 R 时才改变，因此没有见证“捕获后紧邻下一拍就变走”。

后续应补：显式关联捕获事件与 M+1 的数据/校验位变化，并在该事务后续 R 时检查保存值。本接口没有 MEM valid，不应额外虚构一个有效脉冲；需要约束和观察的是四拍到期窗口的数据。

阅读结论：147 项有实际关联轨迹，9 项使用边界示意；547 个已实现 Corner cover 均命中。以上三项的更强文字目标仍待补充，不能称 156 个场景全部验证闭合。